
A Greedy PDE Router for Blending Neural Operators and Classical Methods

Anonymous Author(s)

Affiliation

Address

email

1 **Revision note.** All text, tables and figures added in this revision are typeset in red (new figures
2 carry a red frame); everything in black is unchanged from the submitted version. The iteration-based
3 experiments of the submitted version on 31^2 and 63^2 grids with the LSTM router, and their appendix
4 analyses, have been removed: they are superseded by the wall-clock study of Sections E and 7,
5 which covers the same solvers and equations (plus anisotropic and variable-coefficient diffusion) on
6 128^2 – 512^2 grids and also reports the iteration-based metrics. Only the Dirichlet experiment of the
7 submitted version is kept (Section D).

8 **Status of the confirmatory re-run (draft for review).** In response to the two reviews, every
9 wall-clock result is being regenerated under a frozen protocol (compiled classical kernels, a fresh
10 test seed, three warmed-up timed replays, the complete fixed-schedule family; Section E.1). At
11 the time of this draft the following are *final*: the theory corrections (Sections 4 and 5 and their
12 proofs), the exact short-horizon verification of Theorem 4.1 and the assumption tables of Section E.10
13 (provisional: computed in the development run with the numpy kernels; the operators are identical
14 and only the macro-action sizes change), the discretisation-error study, the 128^2 pairwise results
15 for all three periodic equations (with the classical baselines for Poisson and convection–diffusion),
16 and the development-versus-confirmatory comparison for those cells (Section E.9). Everything
17 marked **[pending]** in the text or shown as a **[results pending]** table is still running, in this order:
18 the nested and larger ensembles at 128^2 ; all 256^2 cells with their ensembles and baselines; all 512^2
19 cells; usage traces, seed trials, overheads, and the confirmatory assumption and screening tables;
20 variable-coefficient diffusion at 128^2 and 256^2 (Section E.5); and the decision-granularity ablation
21 (Section E.6). Summary ranges quoted in the abstract and in Section 7 are computed from the cells
22 available at build time and will change as cells are added. Figures that depend on pending runs (usage
23 traces on the larger grids, the variable-coefficient usage figure) show the development-run version or
24 a placeholder.

Abstract

25 When solving PDEs, classical numerical solvers are often computationally ex-
26 pensive, while machine learning methods can suffer from spectral bias, failing to
27 capture high-frequency components. Designing an optimal hybrid iterative solver–
28 where, at each iteration, a solver is selected from an ensemble of solvers to leverage
29 their complementary strengths–poses a challenging combinatorial problem. While
30 greedy selection is desirable for its constant-factor approximation guarantee to the
31 optimal solution under Lipschitz and **shared-eigenbasis** assumptions, it requires
32 knowledge of the true error at each step, which is unavailable in practice. We
33 address this by proposing an approximate greedy router that efficiently mimics
34 a greedy approach to solver selection. Empirical results on the Poisson, convec-
35 tion–diffusion and anisotropic diffusion equations show that our method reduces
36 the final error and the area-under-the-curve (AUC) of the error trajectory relative to
37 single-solver baselines and existing hybrid approaches such as HINTS in nearly
38 every setting, reaching comparable error levels in substantially fewer iterations
39 with more stable error decay. **With a cost-aware instantiation of the greedy rule, a**

lightweight router, and a band-limited scale-equivariant corrector, we also measure end-to-end wall-clock time on 128^2 to 512^2 grids with compiled classical kernels, four equations (Poisson, convection–diffusion, anisotropic and variable-coefficient diffusion) and six classical solvers including geometric multigrid, against HINTS, the best fixed schedule chosen per setting on the test set, multigrid, preconditioned Krylov methods and direct solves. At truncation-level accuracy the learned router matches, without any search, the best static schedule found by a test-set search (one corrector call, then smoothing) and is $1.69\times$ – $5.94\times$ faster than HINTS with its published period; below truncation level it tracks the cost-aware oracle and beats HINTS and the one-shot schedule by large factors, while a compiled geometric multigrid remains the strongest classical method there; routing over ensembles of classical solvers is strictly better than every single solver where the error passes through regimes that favour different solvers.

1 Introduction

Natural phenomena and engineered systems are often governed by ordinary and partial differential equations (PDEs). Solving these equations enables a variety of tasks - predicting the evolution of the system through simulation (e.g., forecasting weather using the Navier-Stokes equations) [Kalnay, 2003, Bauer et al., 2015, Staniforth and Côté, 1991], addressing control problems (e.g., optimizing heat shield design for spacecrafts using heat transfer equations) [Anderson, 1989, Tröltzsch, 2010], and tackling inverse problems based on external measurements (e.g., reconstructing brain activity from EEG data using electrophysiological models) [Baillet et al., 2001, Grech et al., 2008].

Traditionally, PDEs are solved using finite difference methods [Smith, 1985, LeVeque, 2007], which discretize the spatial and/or temporal domain and approximate the solution by solving a system of equations at the discrete grid points. Similarly, finite element methods [Hughes, 2003, Bathe, 2006, Brenner, 2008] construct a system of equations based on local basis functions, and then rely on numerical linear algebra techniques—such as the Jacobi and Gauss-Seidel method [Saad, 2003]—to compute the solution. These iterative methods, however, **must be re-run from scratch for every new** initial condition, boundary condition, or forcing function, as even minor changes to any of these parameters require re-solving the entire system of equations from scratch.

These challenges have motivated the development of neural operators [Kovachki et al., 2023]—a class of machine learning models that aim to learn the solution operator directly, enabling fast inference across a range of parameters. Neural operators lift the classical linear layer in neural networks to an operator—typically a kernel integral operator—between function spaces. Despite strong approximation guarantees [Chen and Chen, 1995], neural operators are prone to spectral bias [Liu and Cai, 2021, Liu et al., 2024, You et al., 2024, Khodakarami et al., 2025, Xu et al., 2025], similar to traditional neural networks [Rahaman et al., 2019, Xu et al., 2019, Luo et al., 2019]. As a result, they favor learning low-frequency components of the solution function and often struggle to capture high-frequency features that iterative solvers excel at.

Recognizing this complementarity, Zhang et al. [2024] proposed HINTS, a hybrid solver that interleaves a neural operator pass every τ^{th} iteration of a classical solver, with τ fixed a priori. While HINTS reports significant empirical gains in convergence and accuracy over the standalone neural operator and Jacobi solver, this fixed schedule can be detrimental: a poorly timed neural correction may increase the error and undo recent progress.

Although state-dependent solver schedules are desirable, this naturally suggest a reinforcement learning formulation for learning routing policies. However, in our setting, the structure of iterative PDE solvers allows for a simpler approach: each solver induces a known update, and its effect can be evaluated through error-based signals. Leveraging this, we adopt a greedy rule that selects, at each iteration, the solver with the largest immediate error reduction. Such a rule can be near-optimal when the final error satisfies (weak) supermodularity—so that applying a beneficial solver earlier is at least as valuable as applying it later. To support this approach, we make the following contributions:

- We present a general hybrid PDE solver in which a routing rule chooses, at each iteration, from a set of classical solvers and neural operators. With oracle access to the true error at every step, we show that a greedy routing rule achieves a constant-factor approximation to

the optimal strategy for linear PDEs, provided the updates are error-reducing (Lipschitz with constant < 1) and zero-preserving—conditions under which weak supermodularity holds—and share an orthogonal eigenbasis, which gives the postfix monotonicity the guarantee also needs.

- Given that the true error is not observed at test time, a convex surrogate loss is introduced, which, when minimized, enables the learned model to imitate the greedy router without access to true error information. This alignment is guaranteed through Bayes consistency.
- We empirically demonstrate that our approximate greedy solver achieves faster convergence than HINTS and individual solvers on the Poisson, convection-diffusion, anisotropic and variable-coefficient diffusion equations, in iterations and in end-to-end wall-clock time on 128^2 to 512^2 grids with compiled classical kernels; below truncation-level accuracy it is also faster than every fixed schedule, than geometric multigrid alone and than multigrid-preconditioned Krylov methods, and it extends to ensembles of classical solvers.

2 Background

Consider the following linear PDE:

$$\mathcal{L}_x^a(u) = f, x \in D; \quad \mathcal{B}_x(u) = b, x \in \partial D \quad (1)$$

where $\mathcal{L}_x^a$ is a differential operator with respect to the spatial variable $x \in D$ parameterized by the coefficient function a , f is a forcing function, u is the PDE solution, $\mathcal{B}_x$ is the boundary operator, and b is the boundary term. Such PDEs can be solved numerically using various discretization strategies, such as finite differences [Smith, 1985, LeVeque, 2007], finite element, and spectral methods [Boyd, 2001, Canuto et al., 2006]. Here, we focus on finite differences, which replace derivatives with difference quotients (e.g., $\partial_x u(x) \approx (u(x+h) - u(x))/h$) on a uniform grid.

Formally, let h be the grid size and $G_h(D)$ be a uniform grid with spacing h over D . For a function $g : D \rightarrow \mathbb{R}$, we denote its restriction to $G_h(D)$ by $g_h \in \mathbb{R}^N$, where $N = |G_h(D)|$. If $\mathcal{L}_h^a \in \mathbb{R}^{N \times N}$ is the discretized operator, the PDE reduces to a linear system of equations

$$\mathcal{L}_h^a u_h = f_h, \quad (2)$$

with the boundary conditions incorporated in $\mathcal{L}_h^a$. Unlike much of the neural operator literature, which retains function space formulations for discretization invariance, we follow the conventions of classical numerical analysis and represent functions and operators as vectors and matrices. Discretization invariance is not essential here, since we focus on fixed-grid problems.

2.1 Iterative PDE Solvers

Direct methods like Gaussian elimination and Thomas Algorithm can be computationally expensive in high-dimensional domains when solving systems like Equation (2). In contrast, iterative methods like Jacobi and Gauss-Seidel offer computational speedups by iteratively updating the solution, gradually converging to the true solution for the PDE. The general iterative update is

$$u^{(t+1)} = u^{(t)} + C \left(f_h - \mathcal{L}_h^a u^{(t)} \right), \quad (3)$$

where $u^{(t)}$ is the t^{th} iterate of the solution and C is a preconditioning matrix. Jacobi uses $C = D^{-1}$, where D is the diagonal of $\mathcal{L}_h^a$, and Gauss-Seidel uses $C = (D + L)^{-1}$, where L denotes the strict lower triangular part. However, these methods tend to damp low frequency parts of the error slowly.

2.2 Neural Operators

Suppose we observe samples $\{(a_i, f_i, u_i)\}_{i=1}^N$ such that $(a_i, f_i) \sim \mathcal{P}$ are i.i.d. samples, and $u_i = \mathcal{G}^*(a_i, f_i)$ be generated by a deterministic solution operator $\mathcal{G}^*$. A neural operator (NO) $\mathcal{G}_\theta$ seeks to approximate $\mathcal{G}^*$ by minimizing the expected squared $L^2(D)$ error:

$$\mathcal{R}_{\text{NO}}(\mathcal{G}_\theta) = \mathbb{E}_{(a,f) \sim \mathcal{P}} \left[\int_D \|\mathcal{G}^*(a, f)(x) - \mathcal{G}_\theta(a, f)(x)\|_2^2 dx \right]$$

Neural operators use discretization-invariant layers. Prominent instantiations include Fourier Neural Operators (FNO), which use spectral transforms [Li et al., 2020a], Graph Neural Operators, which perform graph-based aggregation over sampled points [Li et al., 2020b], and DeepONets, which employ a trunk-branch decomposition to map input functions to output functions [Lu et al., 2021].

2.3 Greedy Optimization

Consider maximizing $g(S)$ over $S \subseteq \Omega$ with $|S| \leq T$ where $g : 2^\Omega \rightarrow \mathbb{R}$ is a set function defined on subsets of a set Ω . An exhaustive search is infeasible, so an efficient alternative is the greedy algorithm. It builds the solution subset iteratively by adding ω^* that yields the largest marginal gain, i.e. $\omega^* = \arg \max_{\omega \in \Omega \setminus S} g(S \cup \omega)$. When g is non-negative, monotone (adding elements never decreases the value), and submodular (diminishing returns), the greedy algorithm achieves a $(1 - e^{-1})$ approximation to the optimal solution [Nemhauser et al., 1978]. Formally, a function g is submodular if $g(A \cup \{\omega\}) - g(A) \geq g(B \cup \{\omega\}) - g(B)$ for all $A \subseteq B \subseteq \Omega$ and $\omega \in \Omega \setminus B$; intuitively, delaying an addition cannot increase its benefit. For set function minimization problems, supermodularity, where the inequality flips, plays an analogous role, and the greedy rule achieves constant-factor approximation guarantees [Liberty and Sviridenko, 2017].

The suboptimality of the greedy algorithm has been extensively studied for both set-function minimization [Bounia and Koriche, 2023] and maximization [Das and Kempe, 2018, Feige et al., 2011, Bian et al., 2017, Harshaw et al., 2019] when standard assumptions—non-negativity, monotonicity, and sub/supermodularity—are weakened. In contrast, results on greedy sequence maximization [Streeter and Golovin, 2008, Alaei et al., 2021, Zhang et al., 2015, Bernardini et al., 2020, Van Over et al., 2024, Tschischek et al., 2017]—where the ordering of the elements affects the function—have led to sequential analogues of submodularity and monotonicity. We leverage these tools to analyze the suboptimality of the greedy solution to minimizing the final error.

3 General framework for Hybrid Solvers

To solve Equation (2), consider the following hybrid iterative update:

$$u_h^{(t+1)} = u_h^{(t)} + C_{S_t} \left(f_h - \mathcal{L}_h^a u_h^{(t)} \right) \quad (4)$$

where $\mathcal{C} = \{C_j\}_{j=1}^K$ is a set of preconditioning functions and $S_t \in [K] = \{1, \dots, K\}$ indexes the function chosen at step t . Here, we use “preconditioning function” broadly to include classical linear updates ($C_j(x) = C_j x$ where C_j is a matrix), and learned models, such as neural operators. This update generalizes the classical iterative update in Equation (3), by allowing C_j to be non-linear and enabling the preconditioning function to be adaptively chosen at every step. $\mathcal{C}$ can also accommodate parameterized solver families (e.g., different Jacobi relaxation weights), enabling adaptive selection of solver parameters. As the number of solvers K grows, the chance of selecting a more effective update increases. Furthermore, HINTS [Zhang et al., 2024] is a special case of this hybrid solver with $K = 2$, where C_1 is a neural operator and C_2 is a classical preconditioner. Its routing rule is given by $S_t = \mathbf{1}_{(t+1) \bmod \tau > 0} + 1$, which selects C_1 every τ steps (first at iteration τ , after $\tau - 1$ classical steps, as in Algorithm S3 of Zhang et al. [2024]) and C_2 otherwise.

Let $e_h^{(t)} = u_h - u_h^{(t)}$ denote the error at step t . Then,

$$e_h^{(t+1)} = u_h - u_h^{(t)} - C_{S_t} \left(\mathcal{L}_h^a u_h - \mathcal{L}_h^a u_h^{(t)} \right) = (I - C_{S_t} \circ \mathcal{L}_h^a) (e_h^{(t)})$$

where I is the identity map and $I - C_j \circ \mathcal{L}_h^a$ is the error propagation function for solver j . Here, “ $\circ$ ” denotes function composition (matrix multiplication for linear updates). The objective is to select a sequence $S = (S_1, \dots, S_T)$ that minimizes the error norm after T steps or $\|e_h^{(T)}\|_2^2$:

$$\min_{S_t \in [K], |S| \leq T} h(S) := \left\| (I - C_{S_{|S|}} \circ \mathcal{L}_h^a) \circ \dots \circ (I - C_{S_1} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \quad (5)$$

with compositions applied from right to left, so that the S_1 update acts on the initial error.

4 Greedy Algorithm

Equation (5) defines a combinatorial optimization problem with a search space exponential in T , making exact optimization intractable for large T or K . As a starting point, we consider an “omniscient” greedy algorithm that assumes access to the true initial error $e_h^{(0)}$. This is unrealistic since knowledge of $e_h^{(0)}$ could directly recover the solution via $u_h = u_h^{(0)} + e_h^{(0)}$, but it provides a

179 useful benchmark. In Section 5, we relax this assumption using a practical learning strategy that is
 180 Bayes consistent with this omniscient rule, thereby recovering the guarantees shown below.

181 As discussed in Section 2.3, when supermodularity and monotonicity hold, the greedy rule—such
 182 as the one described in Algorithm 1—enjoys constant-factor approximation guarantees. However,
 183 classical results focus on *set* functions, with only recent extensions made to sequences. Building on
 184 this line, we introduce a sequence-based notion of weak supermodularity.

Algorithm 1 Greedy Algorithm for a Hybrid PDE solver

Require: $\{C_j\}_{j=1}^K, T, \mathcal{L}_h^a, e_h^{(0)}$
 $S^0 \leftarrow \emptyset$
for $t < T$ **do**
 $S^{t+1} \leftarrow S^t \oplus \arg \min_{j \in [K]} \|(I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)})\|_2^2$
 $e_h^{(t+1)} \leftarrow (I - C_{S^{t+1}} \circ \mathcal{L}_h^a)(e_h^{(t)})$
end for
return S^T

185 Let Ω^* denote the space of sequences with elements in Ω . For $S, S' \in \Omega^*$, we denote their
 186 concatenation as $S \oplus S'$. S is a prefix of S' or $S \preceq S'$ if $S' = S \oplus L$ for some $L \in \Omega^*$. A function
 187 $g : \Omega^* \rightarrow \mathbb{R}$ is prefix monotonically non-increasing if $g(S \oplus S') \leq g(S)$ for all $S, S' \in \Omega^*$, and
 188 postfix monotonically non-increasing if $g(S' \oplus S) \leq g(S')$ for all $S, S' \in \Omega^*$. A prefix non-increasing
 189 function g is sequence supermodular if, for all $S', S \in \Omega^* : S \preceq S'$, it holds that

$$g(S) - g(S \oplus \omega) \geq g(S') - g(S' \oplus \omega), \quad \forall \omega \in \Omega \quad (6)$$

190 However, $h(S)$ (defined in Equation (5)) may not satisfy this property in general. Therefore, we
 191 introduce weak sequence supermodularity. A prefix non-increasing function g is weakly supermodular
 192 with respect to $S' \in \Omega^*$ if, for any $S \in \Omega^*$, there exists $\alpha(S') \geq 1$ such that

$$g(S) - g(S \oplus S') \leq \alpha(S') \sum_{i \in [|S'|]} g(S) - g(S \oplus S'_i) \quad (7)$$

193 The parameter $\alpha(S')$ or the supermodularity ratio quantifies deviation from exact sequence super-
 194 modularity. Expanding the $g(S) - g(S \oplus S')$ as a telescoping sum $\sum_{i=1}^{|S'|} g(S \oplus (S'_1, \dots, S'_{i-1})) -$
 195 $g(S \oplus (S'_1, \dots, S'_i))$ shows that the marginal decrease from appending S'_i after its predecessors is
 196 controlled by the effect of appending S'_i directly to S . Thus, postponing the inclusion of S'_i cannot
 197 yield a significantly larger benefit than adding it earlier. The supermodularity ratio $\alpha(S)$ quantifies
 198 the extent to which future gains from delays may exceed immediate gains.

199 **Postfix monotonicity enters the analysis only through the greedy prefixes, so we also allow a slack:**
 200 **g is μ -postfix monotone with respect to O along a sequence S^T if $g(S^t \oplus O) \leq \mu g(O)$ for every**
 201 **prefix $S^t, t < T$; a postfix non-increasing function satisfies this with $\mu = 1$, and μ can be computed**
 202 **exactly for short horizons (Section E.10).** Having introduced these notions, we now characterize the
 203 suboptimality of greedy solutions of weakly supermodular and postfix monotonic sequence functions
 204 in Theorem 4.1.

205 **Theorem 4.1.** *Let $g : \Omega^* \rightarrow \mathbb{R}$ be a non-negative, prefix non-increasing function that is weakly*
 206 *supermodular with respect to the optimal solution $O = \arg \min_{S \in \Omega^T} g(S)$ with a supermodularity*
 207 *ratio of $\alpha(O)$, and let the greedy solution of length T be S^T . Suppose g is μ -postfix monotone with*
 208 *respect to O along S^T ($\mu = 1$ if g is postfix non-increasing). If $\phi_T(\alpha) = (1 - \frac{1}{\alpha T})^T$, then*

$$g(S^T) \leq \mu (1 - \phi_T(\alpha(O))) g(O) + \phi_T(\alpha(O)) g(\emptyset), \quad \text{i.e.,} \quad g(\emptyset) - g(S^T) \geq (1 - \phi_T(\alpha(O))) (g(\emptyset) - \mu g(O)).$$

209 The proof of Theorem 4.1 appears in Section A.2. As $T \rightarrow \infty$, the factor $1 - \phi_T(\alpha(O))$ decreases to
 210 $1 - e^{-1/\alpha(O)}$, so the worst-case performance of the greedy rule saturates rather than degrades with
 211 horizon. **The bound concerns the reduction of the error over T steps relative to the best sequence**
 212 **of T steps: with $\alpha = \mu = 1$ it guarantees a $(1 - e^{-1})$ fraction of the optimal reduction, uniformly**
 213 **over ensembles, but it is not a convergence rate—a single convergent solver applied T times already**
 214 **reduces the error by ρ^{2T} , which for large T is far below the $e^{-1}g(\emptyset)$ term. In the settings of**

Section E the greedy sequence is in fact optimal or within a few percent of the exact optimum over short horizons on most instances (Section E.10), much better than the worst case. Additionally, larger $\alpha(O)$, which indicates higher reward for delayed inclusions, loosens the suboptimality bound. Theorem 4.1 requires that the sequence objective h be weakly supermodular with respect to the optimal solution and postfix monotone—properties established in Proposition 4.2. We defer the proof to Section A.3.

Proposition 4.2. *Suppose that for all $j \in [K]$, the error propagation function $I - C_j \circ \mathcal{L}_h^a$ is ρ_j -Lipschitz continuous with $\rho_j < 1$, and that $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$. Then, the function h is weakly supermodular with respect to the optimal solution O , with*

$$\alpha(O) = \max \left\{ \frac{1}{T - \sum_{i=1}^T \rho_{O_i}^2}, 1 \right\}$$

Furthermore, if the error propagation maps are linear and simultaneously diagonalizable by an orthogonal matrix with eigenvalues of modulus at most one (the setting of Theorem 4.3), h is also postfix non-increasing; invertibility alone does not suffice (Section A.3).

The conditions of Proposition 4.2 are natural. For classical solvers, the Lipschitz constant is $\|I_N - C_j \mathcal{L}_h^a\|$, which is typically less than 1 for well-posed linear elliptic PDEs (i.e., the update is damping errors). For neural networks, Lipschitz continuity can be enforced via weight regularization [Gouk et al., 2021] and a sufficiently trained model should approximate $(\mathcal{L}_h^a)^{-1}$ well enough to make the Lipschitz constant small. The requirement $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$ is both natural and desirable: it precludes spurious updates when the residual $f_h - \mathcal{L}_h^a u_h^{(t)}$ is 0. This holds by design for classical schemes, and it can be enforced for any learned model by excluding bias terms.

The form of $\alpha(O)$ in Proposition 4.2 highlights that the suboptimality factor in Theorem 4.1 is governed by the collective contraction factors of the solvers chosen by the optimal solution: as $\sum_{i=1}^T \rho_{O_i}^2 \rightarrow T$, $\alpha(O)$ grows, resulting in a weaker bound. Postfix monotonicity ($\mu = 1$) holds when the error propagation maps share an orthogonal eigenbasis—constant-coefficient periodic operators with Jacobi-type smoothers and band-limited spectral correctors—but it can fail for non-commuting contractions even when all of them are invertible, and a Gauss–Seidel sweep or a learned corrector need not be invertible at all. For the non-commuting solvers of our experiments no a priori bound on μ is available; Section E.10 computes μ , $\alpha(O)$ and the bound of Theorem 4.1 exactly for short horizons by exhaustive search over sequences, and finds $\mu \leq 1.01$ and $\alpha(O) = 1$ on every ensemble of the study, including those with Gauss–Seidel, SOR and multigrid.

Theorem 4.1 thus indicates that the approximation guarantee is strongest when the sequence is nearly supermodular ($\alpha(O) \approx 1$). Generally, if all error propagation maps share an eigenbasis, supermodularity is established. This occurs, for example, for linear, constant-coefficient PDEs with periodic boundary conditions when the solver ensemble consists of (damped) Jacobi iterations and band-limited spectral correctors such as a single-layer linear Fourier neural operator, all of which are diagonal in the Fourier basis; lexicographic Gauss–Seidel and SOR are not, because their sweep order breaks translation invariance. The proof of Theorem 4.3 is deferred to Section A.5.

Proposition 4.3. *Let $\|I_N - C_j \mathcal{L}_h^a\| \leq 1$ for all $j \in [K]$ and $(I - C_j \mathcal{L}_h^a) = P \Lambda_j P^{-1}$ with a common orthogonal matrix P . Then, h is supermodular.*

Cost-aware greedy routing. Algorithm 1 treats every update as equally expensive, whereas in practice a neural-operator evaluation and a classical sweep have very different costs, and the quantity of interest is the error reached per unit of wall-clock time. We make the greedy rule cost-aware without changing the analysis: let c_j denote the (measured) cost of one application of C_j , take the cost u of one neural-operator call as the unit, and replace each preconditioner by the macro-action $C_j^{(m_j)}$ that applies C_j for $m_j = \max\{1, \text{round}(u/c_j)\}$ consecutive iterations, so that all actions cheaper than the unit cost approximately u ; an action dearer than the unit (a multigrid cycle, say) is applied once and its error reduction is compared per unit of cost, $(\|e_j\|/\|e^{(t)}\|)^{u/(m_j c_j)}$, which reduces to the plain comparison when the costs match. For a linear update, m consecutive sweeps are again a stationary iteration, $(I - C_j \circ \mathcal{L}_h^a)^m = I - C_j^{(m)} \mathcal{L}_h^a$ with $C_j^{(m)} = \sum_{i=0}^{m-1} (I - C_j \mathcal{L}_h^a)^i C_j$, and for a nonlinear C_j the macro-action is the m -fold composition of its error propagation map. Hence $\{C_j^{(m_j)}\}_{j=1}^K$ is again a set of preconditioning functions in the sense of Section 4: if $I - C_j \circ \mathcal{L}_h^a$ is

ρ_j -Lipschitz and zero-preserving, then so is its m_j -fold composition (with constant $\rho_j^{m_j}$), a shared eigenbasis is inherited by powers, and Theorem 4.1, Propositions 4.2 and 4.3, and the consistency result of Section 5 apply verbatim to an ensemble of macro-actions that are equalised to the unit cost. Running Algorithm 1 on such macro-actions selects, at every decision epoch, the action with the largest error reduction per unit cost, and a horizon of T macro-actions corresponds to a wall-clock budget of $\approx Tu$. Two caveats: the equalisation is up to rounding (the measured $m_j c_j / u$ lies in $[0.77, 1.18]$ for Jacobi, damped Jacobi, GS and SOR across all grids of Section E; symmetric GS is dearer than the unit on the larger grids and is compared per unit of cost like multigrid), and for an action dearer than the unit the per-unit-cost comparison is a geometric-rate heuristic rather than an equal-cost step, so the multigrid pairings of Section E are covered empirically, not by the equal-cost analysis. Section E uses this cost-aware instantiation throughout.

5 Approximate Greedy Router

The results in Section 4 indicate that the error reduction from the greedy solution closely matches that of the optimal sequence, but this is predicated on having access to the initial error, $e_h^{(0)}$. Inaccurate error estimates can result in poor solver decisions, causing errors to amplify. To remedy this, we learn a router r that selects solvers myopically, as in Algorithm 1, without access to true errors.

We adopt the following learning setup. Let $\mathcal{A}, \mathcal{F}, \mathcal{U} \subseteq \mathbb{R}^N$ denote the spaces of coefficient, forcing, and solution functions on the grid $G_h(D)$. We assume an application-specific data distribution $\mathcal{P}_{\mathcal{A} \times \mathcal{F}}$ over $\mathcal{A} \times \mathcal{F}$ that reflects test time conditions. During training, (a_h, f_h) is drawn from $\mathcal{P}_{\mathcal{A} \times \mathcal{F}}$, and a high-accuracy reference solution u_h is computed, providing the true per-step error. The router r is learned offline on this data; at test time, it operates without access to true errors.

At iteration t , router r selects a solver using the coefficient $a_h \in \mathcal{A}$, forcing $f_h \in \mathcal{F}$, and the current iterate $u_h^{(t)} \in \mathcal{U}$. If $r(a_h, f_h, u_h^{(t)}) = j$, the next iterate is computed with solver j , resulting in an error of $\|(I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)})\|_2^2$. Learning such a router requires minimizing the following loss:

$$l_{\text{route}}(r, a_h, f_h, u_h^{(t)}, u_h) = \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a)(u_h - u_h^{(t)}) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)})=j} \quad (8)$$

with risk $\mathcal{R}_{\text{route}}(r) = \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} [l_{\text{route}}(r, a_h, f_h, u_h^{(t)}, u_h)]$. For analysis, we fix the iterate generation via teacher-forcing [Williams and Zipser, 1989, Lamb et al., 2016]: during training, the iterates fed to the router are produced by an oracle greedy rollout, not by the router's own past choices. Formally, with $u_h^{(0)} = 0_N$, for $t \geq 1$,

$$j_t^* \in \operatorname{argmin}_{j \in [K]} \left\| (I - C_j \circ \mathcal{L}_h^a)(e_h^{(t-1)}) \right\|_2^2, \quad u_h^{(t)} = u_h^{(t-1)} + C_{j_t^*} \left(f_h - \mathcal{L}_h^a u_h^{(t-1)} \right)$$

Thus, $\{u_h^{(t)}\}$ is a deterministic function of (a_h, f_h) . At each t , l_{route} is evaluated on $r(a_h, f_h, u_h^{(t)})$ which takes in the teacher-forced iterate. Finally, the greedy choice j_{t+1}^* is used to advance $u_h^{(t+1)}$. Thus the input distribution seen by r depends on (a_h, f_h) , not on the router's predictions. However, full teacher forcing induces a distributional mismatch at test time (exposure bias). In experiments, we mitigate this with scheduled sampling [Bengio et al., 2015]; see Section C for details.

Minimizing $\mathcal{R}_{\text{route}}(r)$ yields the following Bayes-Optimal Router:

$$r^*(a_h, f_h, u_h^{(t)}) \in \operatorname{argmin}_{j \in [K]} \left\| (I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)}) \right\|_2^2 \quad (9)$$

which aligns with the update in Algorithm 1. Hence, l_{route} is consistent with learning the greedy rule.

5.1 Surrogate Loss

Since Equation (8) is discontinuous and non-convex, direct minimization is intractable in practice. We instead introduce a surrogate loss that (1) is convex, enabling efficient optimization; (2) upper bounds the original loss; and (3) is Bayes consistent, ensuring the Bayes optimal decision of the original loss is preserved upon minimization. Formally, a surrogate ϕ is considered to be Bayes consistent with respect to the loss l if

$$\lim_{n \rightarrow \infty} \mathcal{R}_\phi(f_n) - \mathcal{R}_\phi^* \implies \lim_{n \rightarrow \infty} \mathcal{R}_l(f_n) - \mathcal{R}_l^*$$

where $\mathcal{R}_l(f) = \mathbb{E}[l(f(X), Y)]$ and $\mathcal{R}_l^* = \inf_f \mathcal{R}_l(f)$. In other words, in the limit of infinite data, if the risk of a sequence of learned hypotheses $\{f_n\}$ converges to the optimal risk under ϕ , it also converges to the optimal risk with respect to the original loss l .

To define a surrogate loss for the routing problem, consider a set of scoring functions $\mathbf{g} = \{g_j\}_{j=1}^K$ with $g_j : \mathcal{A} \times \mathcal{F} \times \mathcal{U} \rightarrow \mathbb{R}$ and we define the router as $r(a, f, u^{(t)}) = \operatorname{argmax}_{j \in [K]} g_j(a, f, u^{(t)})$. For example, $\mathbf{g}$ can be a neural network with K outputs. Then, minimizing the following surrogate loss yields the same decision as minimizing Equation (8):

$$\Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h) = - \sum_{j=1}^K \sum_{k=1}^K \tilde{c}_k(a_h, u_h^{(t)}, u_h) \mathbf{1}_{k \neq j} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{m=1}^K \exp(g_m(a_h, f_h, u_h^{(t)}))} \right) \quad (10)$$

where $\tilde{c}_j(a_h, u_h^{(t)}, u_h) = \|(I - C_j \circ \mathcal{L}_h^a)(u_h - u_h^{(t)})\|_2^2$. The Ψ -risk is denoted by $\mathcal{R}_\Psi(\mathbf{g}) = \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}}[\Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)]$. The convexity of Ψ with respect to $\mathbf{g}$ follows from the convexity of log-softmax function in its inputs. Moreover, Ψ upper bounds l_{route} up to a constant factor, and we refer the reader to Section B.2 for the proof. Finally, Theorem 5.1 shows that Ψ achieves Bayes consistency with respect to l_{route} ; the proof can be found in Section B.3.

Theorem 5.1. *Let $\tilde{c}_j(a_h, u_h^{(t)}, u_h) < \bar{E} < \infty$ for all $j \in [K]$. If there exists $j \in [K]$ such that $\tilde{c}_j(a_h, u_h^{(t)}, u_h) > E_{\min} > 0$, then, for any collection of solvers $\{C_j\}_{j=1}^K$ and linear discrete operator $\mathcal{L}_h^a$, Ψ is Bayes consistent surrogate for l_{route} .*

Theorem 5.1 is a cost-sensitive analogue of the classical Bayes-consistency of multiclass cross-entropy for 0-1 loss: in the infinite-sample limit, minimizing cross-entropy recovers the true conditional class probabilities, so the induced decision is Bayes optimal. Our result extends this to cross-entropy with instance-dependent weights $\sum_{k \neq j}^K \tilde{c}_k(a_h, u_h^{(t)}, u_h)$. The uniform upper bound holds when all preconditioning functions are error-damping. It is also reasonable to assume at least one solver cannot annihilate the error in one step, yielding the lower bound. Under these conditions, minimizing Ψ recovers the Bayes-optimal router of Equation (9), i.e., the greedy solution of Equation (5). Following the work of Mao et al. [2024], the proof uses standard conditional-risk calibration: we relate the excess risks of l_{route} and Ψ and take the infinite-sample limit. **This is a per-decision statement on the state distribution the router is trained on: it says that minimising Ψ recovers the greedy decision at each visited state, not that the induced sequence, its time to tolerance, or its distance to the optimal sequence are controlled; the latter would require a sequential argument about the router’s own state distribution, which we do not provide and mitigate empirically with DAgger (Section E).**

6 Related Works

Hybrid PDE Solvers: Early data-driven solvers sought convergence guarantees by predicting parameters—e.g., preconditioning matrices, multi-grid smoothers or restriction matrices—within iterative schemes [Taghibakhshi et al., 2021, Caldana et al., 2024, Kopaničáková and Karniadakis, 2025, Huang et al., 2022, Katrutsa et al., 2020]. These works, however, do not leverage the neural surrogates’ ability to generalize across varying coefficients and forcings highlighted in Section 2.2 and thus, offer only modest speedups over classical solvers. HINTS [Zhang et al., 2024] introduced hybrid solvers that interleave a classical method with a pre-trained DeepONet on a fixed schedule (e.g., 24 Jacobi steps, then one DeepONet correction), achieving faster convergence than the standalone numerical solver. Subsequent works extend this idea to new geometries [Kahana et al., 2023] and analyze error mode damping, replacing DeepONet with alternatives such as MIONet [Jin et al., 2022] [Hu and Jin, 2025] or FNO [Li et al., 2020a, Cui et al., 2022]. However, these hybrid solvers are limited in two ways: they, firstly, only combine the trained surrogate with a *single* numerical solver, and secondly rely on a fixed, heuristic schedule. Our proposed method addresses these shortcomings, resulting in significant empirical improvements (Section 7).

Model Routing: Routing [Shnitzer et al., 2023, Hu et al., 2024, Ding et al., 2024, Huang et al., 2025] selects, for each input, a model from a fixed set to optimize a task metric under cost or latency constraints. Simple heuristics—e.g., thresholding a cheap model’s uncertainty estimates [Chuang et al., 2024, 2025]—often poorly balance the cost-accuracy tradeoff, motivating many systems to instead

learn a router that maps inputs to the best model [Hari and Thomson, 2023, Mohammadshahi et al., 2024, Šakota et al., 2024]. We similarly learn a router, but over numerical and neural solvers for PDEs at the iteration level. Our work is the first to apply routing to the problem of learning hybrid PDE solvers, in contrast to prior approaches that use fixed schedules. Additionally, by exploiting the algebraic structure of PDE solvers, we derive theoretical guarantees for our routing strategy (Section 4), unlike typical model-routing settings.

Mixture of Experts: Classical mixture-of-experts (MoE) [Jacobs et al., 1991, Jordan and Jacobs, 1994] uses a gating network to combine the outputs of multiple experts, trained jointly with the gate. In modern LLMs, MoE instead performs sparse, token-level routing to a subset of in-layer experts, enabling large capacity without proportional compute, typically with load-balancing mechanisms [Shazeer et al., 2017, Lepikhin et al., 2020, Fedus et al., 2022, Jiang et al., 2024]. Our method can be regarded as gating of a different kind: “experts” (solvers) are not jointly trained with the router, and a single router is reused across iterations, unlike MoE which commonly employs layer-specific gates.

7 Experiments

We evaluate the approximate greedy router on four PDEs posed on the unit domain $D = [0, 1]^2$ with periodic boundary conditions:

Poisson: $-\Delta u = f$, **ConvDiff:** $-\Delta u + 20 \partial_{x_1} u + 20 \partial_{x_2} u = f$, **AnisoDiff:** $-\epsilon \partial_{x_1}^2 u - \partial_{x_2}^2 u = f$, $\epsilon = 10^{-2}$, **VarCoef**

where f is centred to satisfy the compatibility condition $\int_D f(\mathbf{x}) d\mathbf{x} = 0$ and is drawn from a zero-mean hierarchical Gaussian random field with covariance operator $\alpha(-\Delta + \beta I)^{-\gamma}$ and sample-dependent (α, β, γ) (Section C). Anisotropic diffusion is the standard problem on which point relaxation fails to smooth: an error mode oscillatory in x_1 and smooth in x_2 is damped by only $(1 - \epsilon)/(1 + \epsilon) \approx 0.98$ per Jacobi sweep. Variable-coefficient diffusion uses a fixed smooth coefficient field $a(\mathbf{x}) = \exp(\kappa z(\mathbf{x}))$ with $\max a / \min a = 10$: its discrete operator is not diagonalised by the FFT, so no direct spectral solve exists, the corrector’s band-limited inverse has no closed form and must be learned from data, and the corrector is only approximately accurate (Section E.5). The domain is discretised on 128^2 , 256^2 and 512^2 grids; results with Dirichlet boundary conditions, obtained with an earlier version of the pipeline, are deferred to Section D.1. Because iteration counts favour methods whose iterations are expensive, our primary metric is the end-to-end *wall-clock time to a target relative error* $\varepsilon = \|e_h^{(t)}\|/\|u_h\|$ on 64, 32 and 16 test instances per grid, with $\varepsilon = h^2$ —a grid-dependent tolerance that lies below the measured discretisation error of every test instance, so that further algebraic refinement does not improve the computed solution (Section E.1)—highlighted; every comparison is paired over instances and tested with a one-sided Wilcoxon signed-rank test. We also report the iteration-based metrics of the original submission, the final relative error and the area under the error curve, $\text{AUC} = \sum_{t=1}^T \|e_h^{(t)}\|/\|u_h\|$ over $T = 300$ iterations. Section E gives the full protocol and collects every result per equation.

Setup. The classical solvers are Jacobi, damped Jacobi ($\omega = 0.67$), Gauss–Seidel (GS), symmetric GS (SymGS), SOR ($\omega = 1.5$) and geometric multigrid (V(2,2)-cycles with GS smoothing), all implemented as compiled single-threaded C stencil kernels (a GS sweep costs $41 \mu\text{s}$ and a V-cycle 0.34 ms at 128^2), so the classical baselines are not slowed by the implementation. The neural operator is a DeepONet corrector that acts on the residual through a 64×64 sensor grid with a Fourier trunk basis and is applied scale-equivariantly: its deployed form is a linear, Fourier-structured map (a least-squares fit of the band-limited inverse of the discrete operator, which it reproduces to $\approx 10^{-6}$), so one call solves the smooth band of the spectrum at the cost of about 12 Jacobi or 3 GS sweeps at 128^2 (and only 2.5 Jacobi sweeps at 512^2 , since its cost is nearly grid-independent) and never injects error outside its band. The router is made *cost-aware* through the cost-equalised macro-actions of Section 4 and selects from $6+K$ scalar features (log residual, its recent decay rate, the previous action and call counts) with a two-layer MLP whose decision costs $\approx 5 \mu\text{s}$ independently of N ; it is trained with the surrogate of Section 5.1 to imitate the cost-aware greedy oracle on oracle rollouts followed by DAGger rounds. We compare against the classical solver alone, HINTS with the published period $\tau=25$, the best fixed schedule chosen *a posteriori* on the test set for each cell from HINTS with $\tau \in \{2, 5, 10, 25, 50\}$, phase-shifted HINTS (first call at iteration 0) and the static one-shot schedule (corrector once at iteration 0, then the solver), the cost-aware greedy oracle (Algorithm 1 with the true

error; not deployable), and the classical methods a practitioner would use on these problems without a corrector, multigrid alone and multigrid-preconditioned CG or BiCGSTAB. All configurations were fixed on a development test set and every number reported is a single confirmatory evaluation on a fresh test set with three timed replays (Section E.9).

Paired solvers. Table 1 reports, for every pairing $\{\text{NO}, \text{solver}\}$, equation and grid, the median wall-clock time to $\varepsilon = h^2$ of HINTS, of the best fixed schedule, and of the learned router, together with the classical methods without corrector. Two regimes appear. *At truncation level* the whole solve is one corrector call followed by a few sweeps: the router, the cost-aware oracle and the best static schedule (the corrector at iteration 0, then smoothing) take the same decisions, so the router is within a few percent of the best fixed schedule found by a test-set search—paired ratios $0.93\times$ – $0.97\times$ at 128^2 , **[pending]**–**[pending]** at 256^2 and **[pending]**–**[pending]** at 512^2 , the difference being its own decision overhead—without any search, and it is $1.69\times$ – $5.94\times$ faster than HINTS with its published period $\tau=25$ (**[pending]**–**[pending]** at 256^2 , **[pending]**–**[pending]** at 512^2), whose first call comes $\tau - 1$ sweeps too late; the classical solver alone is $123\times$ – $707\times$ slower on the isotropic equations and never reaches h^2 on the anisotropic one. *Below truncation level* the schedule matters and the picture is mixed: down to $\varepsilon = 10^{-8}$ the router is $1.07\times$ – $3.18\times$ faster than HINTS ($\tau=25$) and one to two orders of magnitude faster than the one-shot schedule, because it calls the corrector again when the smooth error has re-emerged (Figure 1), but it beats the best fixed schedule in only 7 of the 15 pairings at 128^2 (paired ratios $0.77\times$ – $1.58\times$): the best static schedule there is a phase-shifted HINTS with a short period, which calls the corrector every 5–10 sweeps, more often than a cost-equalised macro-action of 12 Gauss–Seidel or 39 Jacobi sweeps allows the router to (Section E.6 examines this granularity limit). The router tracks the cost-aware oracle within $1.05\times$ in median time and its decisions are identical across five independent training seeds. Against the classical methods without corrector, the compiled multigrid is a strong competitor: at 128^2 the stationary hybrids match it at h^2 (router $0.94\times$ – $1.17\times$ of a stand-alone V-cycle solve) and are slower than it below h^2 ($0.05\times$ – $0.83\times$), whereas routing over $\{\text{NO}, \text{multigrid}\}$ is $0.92\times$ – $0.96\times$ and $1.09\times$ – $1.14\times$ of multigrid alone at h^2 and 10^{-8} ; the router is $1.55\times$ – $2.00\times$ faster than multigrid-preconditioned CG or BiCGSTAB, and on the constant-coefficient equations the FFT direct solve remains $2.97\times$ – $3.12\times$ faster than any iterative method including ours, whereas on variable-coefficient diffusion, which has no such solve, the router is compared with the sparse direct solve (Section E.5). The iteration-based metrics tell the same story: at 128^2 the router’s AUC over $T = 300$ iterations is orders of magnitude below HINTS’ and the solver-only baseline’s for every pairing and equation (paired t -tests, $p < 10^{-3}$; Tables 9, 15, 21 and 27).

Solver ensembles. The same construction routes over ensembles $\text{NO} \cup \mathcal{W}$ of several classical solvers. Since the greedy rule is a per-decision comparison, a larger ensemble can only help where the error passes through regimes that favour *different* members; an exhaustive oracle-level screening of every subset of up to three members (Section E.7) shows that this never happens at $\varepsilon = h^2$ —one corrector call and one macro-action of the cheapest smoother already reach truncation level—but does at tolerances far below h^2 , where the smoother must damp the whole band outside the corrector’s reach: undamped Jacobi is cheapest on the modes near the band edge but cannot damp the near-Nyquist modes, which damped Jacobi removes. We therefore evaluate the learned router over the nested sets $\{\text{Jacobi}\}$, $\{\text{Jacobi} (0.67)\}$, $\{\text{Jacobi}, \text{Jacobi} (0.67)\}$ and $\{\text{Jacobi}, \text{Jacobi} (0.67), \text{GS}\}$ on every equation at 128^2 and 256^2 , with the single-member routers evaluated in the same session on the same instances. Two properties emerge. At $\varepsilon = 10^{-8}$ the ensemble router is strictly faster than every single-member router in **[pending]** of the **[pending]** ensemble cells (six settings, two nested sets) (**[pending]**–**[pending]** over the best single solver, $p < 0.01$, largest on the 256^2 grids), reproducing its oracle: it learns a *nonstationary* damping schedule—undamped Jacobi while the band-edge modes dominate, damped Jacobi once the near-Nyquist modes do—that no single member can reproduce. And enlarging the set is nearly free: adding GS changes nothing, because the oracle ignores a member that never wins a cost-aware comparison and the router imitates it, and at h^2 every ensemble router is within a few percent of the best single-member router; the one exception (Poisson at 128^2 , 10^{-8} , where the ensemble router is 8% slower) is inherited from the greedy oracle itself, which is not monotone in its action set (Section E.7). Finally, we verify numerically that the assumptions of Sections 4 and 5 hold in all these settings, in the energy norm for the non-normal sweeps (Section E.10).

Table 1: Median wall-clock time to truncation-level relative error $\varepsilon = h^2$ for every solver pairing $\{\text{NO}, \text{solver}\}$, equation and grid ($64, 32$ and 16 test instances at $128^2, 256^2$ and 512^2): HINTS with the published period $\tau=25$, HINTS with the best period chosen per cell on the test set itself (subscript), and the learned router (paired median speedup over HINTS-25 / over the best period in parentheses; bold: faster than both). The classical solver alone is 10^2 – $10^3 \times$ slower than the router on the isotropic equations and never reaches h^2 on the anisotropic one (Section E). The last rows of each block give geometric multigrid alone and the best multigrid-preconditioned Krylov method, without corrector, on the same instances. Anisotropic diffusion is not run at 512^2 : its corrector keeps full resolution along x_1 , and the resulting $65,536 \times 65,536$ branch matrix does not fit in memory (Section E); multigrid is not run on it either (Section E.1). $\dagger n$: n instances did not reach h^2 within the iteration cap.

Equation	Pairing	128 $\times$ 128			256 $\times$ 256			512 $\times$ 512		
		HINTS-25	best schedule	router (ours)	HINTS-25	best schedule	router (ours)	HINTS-25	best schedule	router (ours)
Poisson	Jacobi	1.38 ms	774 $\mu\text{s}_{\tau=10, t_0=0}$	815 μs ($1.69 \times / 0.96 \times$)	–	–	–	–	–	–
	Jacobi (0.67)	1.35 ms	765 $\mu\text{s}_{\tau=10, t_0=0}$	802 μs ($1.69 \times / 0.96 \times$)	–	–	–	–	–	–
	GS	2.38 ms	769 $\mu\text{s}_{\tau=5, t_0=0}$	802 μs ($2.96 \times / 0.95 \times$)	–	–	–	–	–	–
	SymGS	3.13 ms	772 $\mu\text{s}_{\tau=25, t_0=0}$	809 μs ($3.90 \times / 0.96 \times$)	–	–	–	–	–	–
	SOR (1.5)	4.39 ms	708 $\mu\text{s}_{\text{one-shot}}$	733 μs ($4.70 \times / 0.96 \times$)	–	–	–	–	–	–
	Multigrid	1.02 ms	698 $\mu\text{s}_{\text{one-shot}}$	728 μs ($1.00 \times / 0.97 \times$)	–	–	–	–	–	–
	Multigrid alone (no corrector)			940 μs	–	–	–	–	–	–
	PCG (MG) (no corrector)			1.90 ms	–	–	–	–	–	–
	FFT direct solve (exact)			233 μs	–	–	–	–	–	–
ConvDiff	Jacobi	1.33 ms	727 $\mu\text{s}_{\tau=10, t_0=0}$	766 μs ($1.75 \times / 0.97 \times$)	–	–	–	–	–	–
	Jacobi (0.67)	1.27 ms	719 $\mu\text{s}_{\tau=25, t_0=0}$	747 μs ($1.69 \times / 0.96 \times$)	–	–	–	–	–	–
	GS	2.53 ms	751 $\mu\text{s}_{\tau=25, t_0=0}$	801 μs ($3.20 \times / 0.96 \times$)	–	–	–	–	–	–
	SymGS	3.11 ms	724 $\mu\text{s}_{\tau=25, t_0=0}$	750 μs ($4.09 \times / 0.95 \times$)	–	–	–	–	–	–
	SOR (1.5)	4.49 ms	729 $\mu\text{s}_{\tau=5, t_0=0}$	756 μs ($5.94 \times / 0.97 \times$)	–	–	–	–	–	–
	Multigrid	1.04 ms	715 $\mu\text{s}_{\tau=5, t_0=0}$	744 μs ($0.99 \times / 0.97 \times$)	–	–	–	–	–	–
	Multigrid alone (no corrector)			990 μs	–	–	–	–	–	–
	BiCGSTAB (MG) (no corrector)			1.38 ms	–	–	–	–	–	–
	FFT direct solve (exact)			250 μs	–	–	–	–	–	–
AnisoDiff	Jacobi	1.27 ms	705 $\mu\text{s}_{\tau=50, t_0=0}$	739 μs ($1.74 \times / 0.96 \times$)	–	–	–	–	–	–
	Jacobi (0.67)	1.29 ms	716 $\mu\text{s}_{\tau=50, t_0=0}$	735 μs ($1.77 \times / 0.96 \times$)	–	–	–	–	–	–
	GS	2.34 ms	749 $\mu\text{s}_{\tau=10, t_0=0}$	792 μs ($2.95 \times / 0.96 \times$)	–	–	–	–	–	–
	SymGS	3.08 ms	742 $\mu\text{s}_{\tau=25, t_0=0}$	787 μs ($3.91 \times / 0.93 \times$)	–	–	–	–	–	–
	SOR (1.5)	2.24 ms	689 $\mu\text{s}_{\tau=25, t_0=0}$	730 μs ($3.08 \times / 0.95 \times$)	–	–	–	–	–	–

Every result per equation and grid, the ensemble study, the accounting of overheads and training cost, and the verification of the theoretical assumptions are collected in Section E; Section D reports the Dirichlet experiment of the original submission.

8 Discussion

We have introduced an adaptive method for selecting solvers that efficiently minimizes the final error of a PDE in iterative methods, opening many directions for future work. Our current DeepONet is trained without accounting for its downstream role in correction term prediction; jointly training the ML solver and router could yield larger gains. Another avenue is to use reinforcement learning to learn a cost-aware routing strategy that optimizes error under compute budgets; Sections 4 and E show that a myopic cost-aware variant of the greedy rule—Algorithm 1 on cost-equalised macro-actions—already delivers uniform wall-clock gains, and RL could extend it to non-myopic budgeted routing. RL would additionally enable extending the ensemble routing to *continuous* parameterization of solvers. Finally, framing routing as an online learning problem could enable adaptation to unknown deployment conditions.

References

- Eugenia Kalnay. *Atmospheric modeling, data assimilation and predictability*. Cambridge university press, 2003.
- Peter Bauer, Alan Thorpe, and Gilbert Brunet. The quiet revolution of numerical weather prediction. *Nature*, 525(7567):47–55, 2015.
- Andrew Staniforth and Jean Côté. Semi-lagrangian integration schemes for atmospheric models—a review. *Monthly weather review*, 119(9):2206–2223, 1991.
- John David Anderson. *Hypersonic and high temperature gas dynamics*. Aiaa, 1989.
- Fredi Tröltzsch. *Optimal control of partial differential equations: theory, methods, and applications*, volume 112. American Mathematical Soc., 2010.

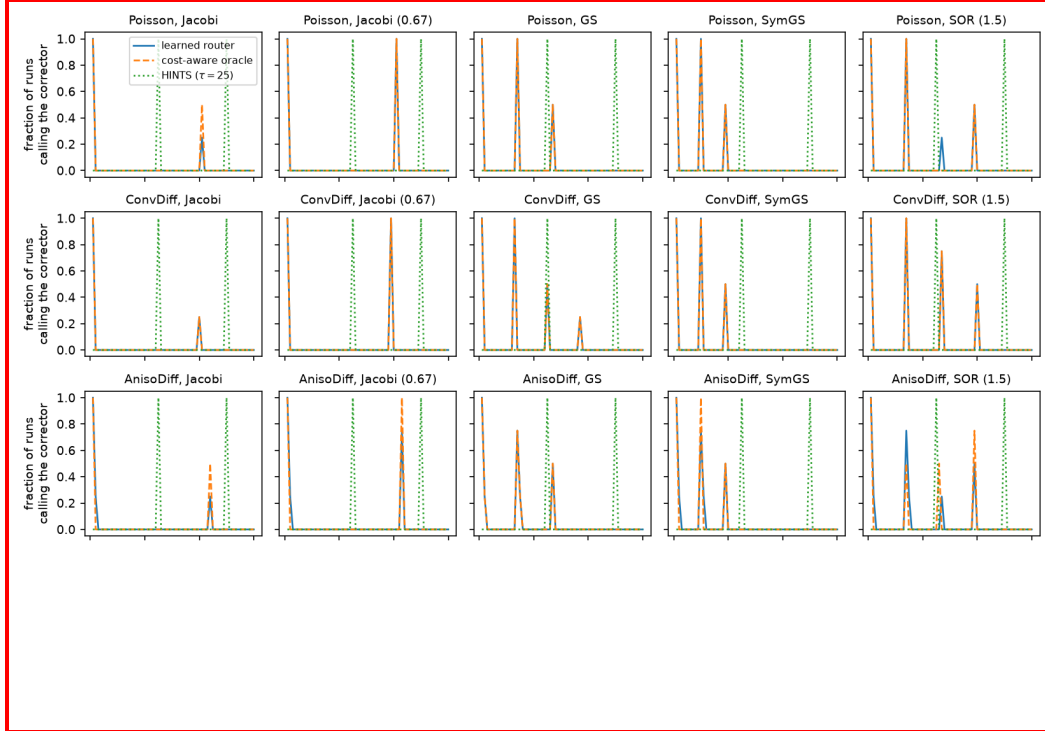


Figure 1: Corrector usage on the 128×128 grid: fraction of test runs that execute a corrector call at each iteration, for the learned router, the cost-aware oracle and HINTS ($\tau=25$), for every solver pairing (columns) and all three equations (rows); the same figure for the 256^2 and 512^2 grids is in Section E.

- 483 Sylvain Baillet, John C Mosher, and Richard M Leahy. Electromagnetic brain mapping. *IEEE Signal*
484 *processing magazine*, 18(6):14–30, 2001.
- 485 Roberta Grech, Tracey Cassar, Joseph Muscat, Kenneth P Camilleri, Simon G Fabri, Michalis
486 Zervakis, Petros Xanthopoulos, Vangelis Sakkalis, and Bart Vanrumste. Review on solving the
487 inverse problem in eeg source analysis. *Journal of neuroengineering and rehabilitation*, 5:1–33,
488 2008.
- 489 Gordon D Smith. *Numerical solution of partial differential equations: finite difference methods*.
490 Oxford university press, 1985.
- 491 Randall J LeVeque. *Finite difference methods for ordinary and partial differential equations: steady-*
492 *state and time-dependent problems*. SIAM, 2007.
- 493 Thomas JR Hughes. *The finite element method: linear static and dynamic finite element analysis*.
494 Courier Corporation, 2003.
- 495 Klaus-Jürgen Bathe. *Finite element procedures*. Klaus-Jurgen Bathe, 2006.
- 496 Susanne C Brenner. *The mathematical theory of finite element methods*. Springer, 2008.
- 497 Yousef Saad. *Iterative methods for sparse linear systems*. SIAM, 2003.
- 498 Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew
499 Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with
500 applications to pdes. *Journal of Machine Learning Research*, 24(89):1–97, 2023.
- 501 Tianping Chen and Hong Chen. Universal approximation to nonlinear operators by neural networks
502 with arbitrary activation functions and its application to dynamical systems. *IEEE transactions on*
503 *neural networks*, 6(4):911–917, 1995.

504 Lizuo Liu and Wei Cai. Multiscale deepnet for nonlinear operators in oscillatory function spaces for
505 building seismic wave responses. *arXiv preprint arXiv:2111.04860*, 2021.

506 Xinliang Liu, Bo Xu, Shuhao Cao, and Lei Zhang. Mitigating spectral bias for the multiscale operator
507 learning. *Journal of Computational Physics*, 506:112944, 2024.

508 Zhilin You, Zhenli Xu, and Wei Cai. Mscalefno: Multi-scale fourier neural operator learning for
509 oscillatory function spaces. *arXiv preprint arXiv:2412.20183*, 2024.

510 Siavash Khodakarami, Vivek Oommen, Aniruddha Bora, and George Em Karniadakis. Mitigating
511 spectral bias in neural operators via high-frequency scaling for physical systems. *arXiv preprint*
512 *arXiv:2503.13695*, 2025.

513 Zhi-Qin John Xu, Lulu Zhang, and Wei Cai. On understanding and overcoming spectral biases of
514 deep neural network learning methods for solving pdes. *arXiv preprint arXiv:2501.09987*, 2025.

515 Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua
516 Bengio, and Aaron Courville. On the spectral bias of neural networks. In *International conference*
517 *on machine learning*, pages 5301–5310. PMLR, 2019.

518 Zhi-Qin John Xu, Yaoyu Zhang, Tao Luo, Yanyang Xiao, and Zheng Ma. Frequency principle:
519 Fourier analysis sheds light on deep neural networks. *arXiv preprint arXiv:1901.06523*, 2019.

520 Tao Luo, Zheng Ma, Zhi-Qin John Xu, and Yaoyu Zhang. Theory of the frequency principle for
521 general deep neural networks. *arXiv preprint arXiv:1906.09235*, 2019.

522 Enrui Zhang, Adar Kahana, Alena Kopaničáková, Eli Turkel, Rishikesh Ranade, Jay Pathak, and
523 George Em Karniadakis. Blending neural operators and relaxation methods in pde numerical
524 solvers. *Nature Machine Intelligence*, pages 1–11, 2024.

525 John P Boyd. *Chebyshev and Fourier spectral methods*. Courier Corporation, 2001.

526 Claudio Canuto, M. Yousuff Hussaini, Alfio Quarteroni, and Thomas A. Zang. *Spectral Methods:*
527 *Fundamentals in Single Domains*. Springer, 2006. ISBN 9783540307264.

528 Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew
529 Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations.
530 *arXiv preprint arXiv:2010.08895*, 2020a.

531 Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Andrew Stuart, Kaushik
532 Bhattacharya, and Anima Anandkumar. Multipole graph neural operator for parametric partial
533 differential equations. *Advances in Neural Information Processing Systems*, 33:6755–6766, 2020b.

534 Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning
535 nonlinear operators via deepnet based on the universal approximation theorem of operators.
536 *Nature machine intelligence*, 3(3):218–229, 2021.

537 George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. An analysis of approximations for
538 maximizing submodular set functions—i. *Mathematical programming*, 14(1):265–294, 1978.

539 Edo Liberty and Maxim Sviridenko. Greedy minimization of weakly supermodular set functions.
540 In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques*
541 *(APPROX/RANDOM 2017)*, pages 19–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2017.

542 Louenas Bounia and Frederic Koriche. Approximating probabilistic explanations via supermodular
543 minimization. In *Uncertainty in Artificial Intelligence*, pages 216–225. PMLR, 2023.

544 Abhimanyu Das and David Kempe. Approximate submodularity and its applications: Subset selection,
545 sparse approximation and dictionary selection. *Journal of Machine Learning Research*, 19(3):
546 1–34, 2018.

547 Uriel Feige, Vahab S Mirrokni, and Jan Vondrák. Maximizing non-monotone submodular functions.
548 *SIAM Journal on Computing*, 40(4):1133–1153, 2011.

549 Andrew An Bian, Joachim M Buhmann, Andreas Krause, and Sebastian Tschiatschek. Guarantees for
550 greedy maximization of non-submodular functions with applications. In *International conference*
551 *on machine learning*, pages 498–507. PMLR, 2017.

552 Chris Harshaw, Moran Feldman, Justin Ward, and Amin Karbasi. Submodular maximization beyond
553 non-negativity: Guarantees, fast algorithms, and applications. In *International Conference on*
554 *Machine Learning*, pages 2634–2643. PMLR, 2019.

555 Matthew Streeter and Daniel Golovin. An online algorithm for maximizing submodular functions.
556 *Advances in Neural Information Processing Systems*, 21, 2008.

557 Saeed Alaei, Ali Makhdoumi, and Azarakhsh Malekian. Maximizing sequence-submodular functions
558 and its application to online advertising. *Management Science*, 67(10):6030–6054, 2021.

559 Zhenliang Zhang, Edwin KP Chong, Ali Pezeshki, and William Moran. String submodular functions
560 with curvature constraints. *IEEE Transactions on Automatic Control*, 61(3):601–616, 2015.

561 Sara Bernardini, Fabio Fagnani, and Chiara Piacentini. Through the lens of sequence submodularity.
562 In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 30,
563 pages 38–47, 2020.

564 Brandon Van Over, Bowen Li, Edwin KP Chong, and Ali Pezeshki. A performance bound for
565 the greedy algorithm in a generalized class of string optimization problems. *arXiv preprint*
566 *arXiv:2409.05020*, 2024.

567 Sebastian Tschiatschek, Adish Singla, and Andreas Krause. Selecting sequences of items via
568 submodular maximization. In *Proceedings of the AAAI Conference on Artificial Intelligence*,
569 volume 31, 2017.

570 Henry Gouk, Eibe Frank, Bernhard Pfahringer, and Michael J Cree. Regularisation of neural networks
571 by enforcing lipschitz continuity. *Machine Learning*, 110(2):393–416, 2021.

572 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
573 neural networks. *Neural computation*, 1(2):270–280, 1989.

574 Alex M Lamb, Anirudh Goyal ALIAS PARTH GOYAL, Ying Zhang, Saizheng Zhang, Aaron C
575 Courville, and Yoshua Bengio. Professor forcing: A new algorithm for training recurrent networks.
576 *Advances in neural information processing systems*, 29, 2016.

577 Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence
578 prediction with recurrent neural networks. *Advances in neural information processing systems*, 28,
579 2015.

580 Anqi Mao, Mehryar Mohri, and Yutao Zhong. Regression with multi-expert deferral. In *International*
581 *Conference on Machine Learning*, pages 34738–34759. PMLR, 2024.

582 Ali Taghibakhshi, Scott MacLachlan, Luke Olson, and Matthew West. Optimization-based algebraic
583 multigrid coarsening using reinforcement learning. *Advances in neural information processing*
584 *systems*, 34:12129–12140, 2021.

585 Matteo Caldana, Paola F Antonietti, et al. A deep learning algorithm to accelerate algebraic multigrid
586 methods in finite element solvers of 3d elliptic pdes. *Computers & Mathematics with Applications*,
587 167:217–231, 2024.

588 Alena Kopaničáková and George Em Karniadakis. Deepnet based preconditioning strategies for
589 solving parametric linear systems of equations. *SIAM Journal on Scientific Computing*, 47(1):
590 C151–C181, 2025.

591 Ru Huang, Ruipeng Li, and Yuanzhe Xi. Learning optimal multigrid smoothers via neural networks.
592 *SIAM Journal on Scientific Computing*, 45(3):S199–S225, 2022.

593 Alexandr Katrutsa, Talgat Daulbaev, and Ivan Oseledets. Black-box learning of multigrid parameters.
594 *Journal of Computational and Applied Mathematics*, 368:112524, 2020.

595 Adar Kahana, Enrui Zhang, Somdatta Goswami, George Karniadakis, Rishikesh Ranade, and Jay
596 Pathak. On the geometry transferability of the hybrid iterative numerical solver for differential
597 equations. *Computational Mechanics*, 72(3):471–484, 2023.

598 Pengzhan Jin, Shuai Meng, and Lu Lu. Mionet: Learning multiple-input operators via tensor product.
599 *SIAM Journal on Scientific Computing*, 44(6):A3490–A3514, 2022.

600 Jun Hu and Pengzhan Jin. A hybrid iterative method based on mionet for pdes: Theory and numerical
601 examples. *Mathematics of Computation*, 2025.

602 Chen Cui, Kai Jiang, Yun Liu, and Shi Shu. Fourier neural solver for large sparse linear algebraic
603 systems. *Mathematics*, 10(21):4014, 2022.

604 Tal Shnitzer, Anthony Ou, Mirian Silva, Kate Soule, Yuekai Sun, Justin Solomon, Neil Thompson,
605 and Mikhail Yurochkin. Large language model routing with benchmark datasets. *arXiv preprint*
606 *arXiv:2309.15789*, 2023.

607 Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt
608 Keutzer, and Shriyash Kaustubh Upadhyay. Routerbench: A benchmark for multi-llm routing
609 system. *arXiv preprint arXiv:2403.12031*, 2024.

610 Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks VS
611 Lakshmanan, and Ahmed Hassan Awadallah. Hybrid llm: Cost-efficient and quality-aware query
612 routing. *arXiv preprint arXiv:2404.14618*, 2024.

613 Zhongzhan Huang, Guoming Ling, Yupei Lin, Yandong Chen, Shanshan Zhong, Hefeng Wu, and
614 Liang Lin. Routereval: A comprehensive benchmark for routing llms to explore model-level
615 scaling up in llms. *arXiv preprint arXiv:2503.10657*, 2025.

616 Yu-Neng Chuang, Prathusha Kameswara Sarma, Parikshit Gopalan, John Boccio, Sara Bolouki,
617 Xia Hu, and Helen Zhou. Learning to route llms with confidence tokens. *arXiv preprint*
618 *arXiv:2410.13284*, 2024.

619 Yu-Neng Chuang, Leisheng Yu, Guanchu Wang, Lizhe Zhang, Zirui Liu, Xuanning Cai, Yang Sui,
620 Vladimir Braverman, and Xia Hu. Confident or seek stronger: Exploring uncertainty-based
621 on-device llm routing from benchmarking to generalization. *arXiv preprint arXiv:2502.04428*,
622 2025.

623 Surya Narayanan Hari and Matt Thomson. Tryage: Real-time, intelligent routing of user prompts to
624 large language models. *arXiv preprint arXiv:2308.11601*, 2023.

625 Alireza Mohammadshahi, Arshad Rafiq Shaikh, and Majid Yazdani. Routoo: Learning to route to
626 large language models effectively. *arXiv preprint arXiv:2401.13979*, 2024.

627 Marija Šakota, Maxime Peyrard, and Robert West. Fly-swat or cannon? cost-effective language
628 model choice via meta-modeling. In *Proceedings of the 17th ACM International Conference on*
629 *Web Search and Data Mining*, pages 606–615, 2024.

630 Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of
631 local experts. *Neural computation*, 3(1):79–87, 1991.

632 Michael I Jordan and Robert A Jacobs. Hierarchical mixtures of experts and the em algorithm. *Neural*
633 *computation*, 6(2):181–214, 1994.

634 Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and
635 Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv*
636 *preprint arXiv:1701.06538*, 2017.

637 Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang,
638 Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional
639 computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

640 William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter
641 models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39,
642 2022.

- 643 Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris
644 Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al.
645 Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- 646 Anqi Mao, Mehryar Mohri, and Yutao Zhong. Cross-entropy loss functions: Theoretical analysis and
647 applications. In *International conference on Machine learning*, pages 23803–23828. pmlr, 2023.
- 648 Michael C Mozer. A focused backpropagation algorithm for temporal pattern recognition. In
649 *Backpropagation*, pages 137–169. Psychology Press, 2013.
- 650 Anthony J Robinson and Frank Fallside. *The utility driven dynamic error propagation network*,
651 volume 11. University of Cambridge Department of Engineering Cambridge, 1987.
- 652 Paul J Werbos. Generalization of backpropagation with application to a recurrent gas market model.
653 *Neural networks*, 1(4):339–356, 1988.
- 654 Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and
655 George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with prac-
656 tical extensions) based on FAIR data. *Computer Methods in Applied Mechanics and Engineering*,
657 393:114778, 2022.
- 658 Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured
659 prediction to no-regret online learning. In *Proceedings of the Fourteenth International Confer-
660 ence on Artificial Intelligence and Statistics*, pages 627–635. JMLR Workshop and Conference
661 Proceedings, 2011.

662 A Proofs for Section 4

663 A.1 Proof of Proposition A.1

664 **Proposition A.1.** *Any prefix monotonically non-increasing sequence supermodular function g is*
665 *weakly supermodular with respect to all sequences $S \in \Omega^*$ with $\alpha(S) = 1$*

666 *Proof.* This proof is adapted from Liberty and Sviridenko [2017].

667 If g is sequence supermodular then,

$$\begin{aligned}
& g(S) - g(S \oplus S') \\
&= \sum_{i=1}^{|S'|} g(S \oplus (S'_1, \dots, S'_{i-1})) - g(S \oplus (S'_1, \dots, S'_i)) \\
&\stackrel{(a)}{\leq} \sum_{i=1}^{|S'|} g(S) - g(S \oplus S'_i) \\
&\leq |S'| \max_{i \in [|S'|]} g(S) - g(S \oplus S'_i)
\end{aligned}$$

668 (a) by supermodularity

669 □

670 A.2 Proof of Theorem 4.1

671 **Theorem 4.1.** *Let $g : \Omega^* \rightarrow \mathbb{R}$ be a weakly supermodular function with respect to the optimal*
672 *solution $O = \arg \min_{S \in \Omega^T} g(S)$ with a supermodularity ratio of $\alpha(O)$ and μ -postfix monotonicity*
673 *with respect to O along the greedy solution S^T of length T . If $\phi_T(\alpha) = (1 - \frac{1}{\alpha T})^T$, then*

$$g(S^T) \leq \mu (1 - \phi_T(\alpha(O))) g(O) + \phi_T(\alpha(O)) g(\emptyset)$$

674 *Proof.* This proof strategy is inspired by Streeter and Golovin [2008] and Liberty and Sviridenko
 675 [2017]

$$\begin{aligned}
 g(S^t) - \mu g(O) &\stackrel{(a)}{\leq} g(S^t) - g(S^t \oplus O) \\
 &= \sum_{i=1}^{|O|} g(S^t \oplus (o_1, \dots, o_{i-1})) - g(S^t \oplus (o_1, \dots, o_i)) \\
 &\stackrel{(b)}{\leq} \alpha(O) \sum_{i=1}^{|O|} g(S^t) - g(S^t \oplus o_i) \\
 &\leq \alpha(O) |O| \max_{i \in [|O|]} g(S^t) - g(S^t \oplus o_i) \\
 &\leq \alpha(O) T g(S^t) - \alpha(O) T \min_{\omega \in \Omega} g(S^t \oplus \omega) \\
 &= \alpha(O) T g(S^t) - \alpha(O) T g(S^{t+1})
 \end{aligned}$$

676 (a) by μ -postfix monotonicity along S^T , i.e., $g(S^t \oplus O) \leq \mu g(O)$; (b) by weak supermodularity with
 677 respect to O (applied to the prefix S^t).

678 After rearranging the inequality, we get:

$$\begin{aligned}
 g(S^{t+1}) &\leq \frac{1}{\alpha(O)T} (\mu g(O) + (\alpha(O)T - 1) g(S^t)) \\
 &= \frac{\mu}{\alpha(O)T} g(O) + \left(1 - \frac{1}{\alpha(O)T}\right) g(S^t)
 \end{aligned}$$

679 When recursively applying this inequality, we get:

$$\begin{aligned}
 g(S^T) &\leq \frac{\mu}{\alpha(O)T} g(O) \sum_{i=0}^{T-1} \left(1 - \frac{1}{\alpha(O)T}\right)^i + \left(1 - \frac{1}{\alpha(O)T}\right)^T g(\emptyset) \\
 &= \mu \left(1 - \left(1 - \frac{1}{\alpha(O)T}\right)^T\right) g(O) + \left(1 - \frac{1}{\alpha(O)T}\right)^T g(\emptyset)
 \end{aligned}$$

680

□

681 A.3 Proof of Proposition 4.2

682 **Proposition 4.2.** Suppose that for all $j \in [K]$, the error propagation function $I - C_j \circ \mathcal{L}_h^a$ is
 683 ρ_j -Lipschitz continuous with $\rho_j < 1$, and that $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$. Then, the function h is
 684 weakly supermodular with respect to the optimal solution O , with

$$\alpha(O) = \max \left\{ \frac{1}{T - \sum_{i=1}^T \rho_{O_i}^2}, 1 \right\}$$

685 *Furthermore, if the error propagation maps are linear and simultaneously diagonalizable by an*
 686 *orthogonal matrix with eigenvalues of modulus at most one (the setting of Theorem 4.3), h is also*
 687 *postfix non-increasing; invertibility alone does not suffice (Section A.3).*

688 *Proof.* For brevity, we use the notation $(g_1 \circ \dots \circ g_T)(x) = \circ_{t=1}^T g_t(x)$, where composition is
 689 applied from right to left so that g_T acts first. In this proof, we use a few properties of Lipschitz
 690 continuous functions:

691 • **Property 1:** If g is ρ -Lipschitz continuous and $g(0) = 0$, then $\|g(x)\|_2 = \|g(x) - 0\|_2 =$
 692 $\|g(x) - g(0)\|_2 \leq \rho \|x - 0\|_2 = \rho \|x\|_2$

- 693 • **Property 2:** If g_1 and g_2 are Lipschitz continuous functions with Lipschitz constants of ρ_1
694 and ρ_2 respectively, the Lipschitz constant of $g_1 + g_2$ and $g_1 - g_2$ is $\rho_1 + \rho_2$.
- 695 • **Property 3:** If g_1 and g_2 are Lipschitz continuous functions with Lipschitz constants of ρ_1
696 and ρ_2 respectively, the Lipschitz constant of $g_1 \circ g_2$ is $\rho_1 \rho_2$.

697 In order to prove weakly- α -supermodularity, we must first prove prefix monotonicity.

698 **Prefix monotonicity:** Let $S \preceq S'$ where $S' = S \oplus N$.

$$\begin{aligned}
h(S') &= \left\| \circ_{t=|S'|}^1 (I - C_{S'_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
&= \left\| \circ_{t=|S'|}^{|S|+1} (I - C_{S'_t} \circ \mathcal{L}_h^a) \circ \circ_{t=|S|}^1 (I - C_{S'_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
&\stackrel{(a)}{\leq} \left(\prod_{t=|S|+1}^{|S'|} \rho_{S'_t}^2 \right) \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
&\leq h(S)
\end{aligned}$$

699 (a) by Property 1

700 **Weak supermodularity:** We will upper bound $\alpha(O)$ by providing an upper bound for $h(S) - h(S \oplus$
701 $O)$ and a lower bound for $\sum_{i=1}^T h(S) - h(S \oplus O_i)$. Since $h \geq 0$, $h(S) - h(S \oplus O) \leq h(S) =$
702 $\left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2$. (An earlier version of this step used $\|x\|^2 - \|Gx\|^2 \leq \|x - Gx\|^2$,
703 which is false, e.g. for $G = \frac{1}{2}I$; the direct bound is sharper by a factor 4.)

704 To lower bound $\sum_{i=1}^{|O|} h(S) - h(S \oplus O_i)$

$$\begin{aligned}
\sum_{i=1}^{|O|} h(S) - h(S \oplus O_i) &= \sum_{i=1}^{|O|} \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 - \left\| (I - C_{O_i} \circ \mathcal{L}_h^a) \circ \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
&\geq \sum_{i=1}^{|O|} \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 - \rho_{O_i}^2 \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
&= \left\| \circ_{t=|S|}^1 (I_N - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \left(T - \sum_{i=1}^T \rho_{O_i}^2 \right)
\end{aligned}$$

705 Finally,

$$\begin{aligned}
\frac{h(S) - h(S \oplus O)}{\sum_{i=1}^T (h(S) - h(S \oplus O_i))} &\leq \max \left\{ \frac{\left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2}{\left\| \circ_{t=|S|}^1 (I_N - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 (T - \sum_{i=1}^T \rho_{O_i}^2)}, 1 \right\} \\
&= \max \left\{ \frac{1}{T - \sum_{i=1}^T \rho_{O_i}^2}, 1 \right\}
\end{aligned}$$

706 **Postfix monotonicity.** Let $S' = S \oplus N$ and suppose that $I - C_j \mathcal{L}_h^a = P \Lambda_j P^\top$ for all j with a
707 common orthogonal P and $|\lambda_{ji}| \leq 1$. By Theorem A.2,

$$h(S') = \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(N)} \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \leq \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(N)} = h(N),$$

708 since every factor $\lambda_{ji}^{2m_j(S)}$ lies in $[0, 1]$. The shared eigenbasis cannot be replaced by invertibility of
709 the individual maps: for

$$A = \begin{bmatrix} 0.0674240 & -0.1864331 \\ -0.1473150 & -0.8707252 \end{bmatrix}, \quad B = \begin{bmatrix} 0.7575738 & 0.4816284 \\ -0.1369844 & 0.3257283 \end{bmatrix}, \quad x = \begin{bmatrix} 0.2812107 \\ -0.5538228 \end{bmatrix},$$

710 both matrices are invertible with spectral norm 0.9, yet $\|BAx\|_2 = 0.330 > 0.225 = \|Bx\|_2$, i.e.,
 711 $h((A) \oplus (B)) > h((B))$. (An earlier version of this proof bounded $\|G^{-1}\|$ by the reciprocal of the
 712 Lipschitz constant of G , which is not valid.)

713

□

714 A.4 Proof of Theorem A.2

715 **Lemma A.2.** Let $(I - C_j \mathcal{L}_h^a) = P \Lambda_j P^{-1}$ where P is an orthogonal matrix and $\Lambda_j =$
 716 $\text{diag}(\lambda_{j1}, \dots, \lambda_{jN})$. If $P^{-1}e_h^{(0)} = z$, then the following equality holds:

$$h(S) = \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \quad (11)$$

717 where $m_j(S) = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=j}$

Proof.

$$\begin{aligned} h(S) &= \left\| \prod_{t=|S|}^1 (I_N - C_{S_t} \mathcal{L}_h^a) e_h^{(0)} \right\|^2 \\ &= \left\| \prod_{t=|S|}^1 (P \Lambda_{S_t} P^{-1}) e_h^{(0)} \right\|^2 \\ &= \left\| P \prod_{t=|S|}^1 \Lambda_{S_t} P^{-1} e_h^{(0)} \right\|^2 \\ &= \left\| \prod_{t=|S|}^1 \Lambda_{S_t} P^{-1} e_h^{(0)} \right\|^2 \\ &= \sum_{i=1}^N z_i^2 \prod_{t=|S|}^1 \lambda_{S_t i}^2 \\ &= \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \end{aligned}$$

718

□

719 A.5 Proof of Theorem 4.3

720 **Proposition 4.3.** Let $\|I_N - C_j \mathcal{L}_h^a\| \leq 1$ for all $j \in [K]$ and $(I - C_j \mathcal{L}_h^a) = P \Lambda_j P^{-1}$ with a common
 721 orthogonal matrix P . Then, h is supermodular.

722 *Proof.* Let $S \preceq S'$ where $S' = S \oplus B$. By Theorem A.2,

$$h(S) = \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)}$$

723 where $m_j(S) = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=j}$ be the number of times a sequence S calls the solver j . Recall that h is
 724 considered sequence supermodular if $\forall S', S \in \Omega^*$ such that $S \preceq S'$, it holds that

$$h(S) - h(S \oplus \omega) \geq h(S') - h(S' \oplus \omega)$$

$$\begin{aligned}
h(S) - h(S \oplus \omega) &= \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} - \sum_{i=1}^N z_i^2 \lambda_{\omega i}^2 \prod_{k=1}^K \lambda_{ji}^{2m_j(S)} \\
&= \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)}
\end{aligned}$$

726 Similarly,

$$\begin{aligned}
h(S') - h(S' \oplus \omega) &= \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S')} \\
&\stackrel{(a)}{=} \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2(m_j(S) + m_j(B))} \\
&\stackrel{(b)}{\leq} \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \\
&= h(S) - h(S \oplus \omega)
\end{aligned}$$

727 (a) Since $m_j(S') = \sum_{t=1}^{|S'|} \mathbf{1}_{S'_t=j} = \sum_{t=1}^{|S|} \mathbf{1}_{S'_t=j} + \sum_{t=|S|+1}^{|S'|} \mathbf{1}_{S'_t=j} = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=k} + \sum_{t=1}^{|B|} \mathbf{1}_{B_t=j} =$
728 $m_j(S) + m_j(B)$, (b) since $|\lambda_{ji}| \leq 1$ (as $\|I_N - C_j \mathcal{L}_h^a\| \leq 1$ and P is orthogonal) and $m_j(B) \geq 0$
729 for all $j \in [K]$

730 □

731 B Proofs for Section 5

732 B.1 Proof of Theorem B.1

733 **Lemma B.1.** *For any set of preconditioning functions $\mathcal{C}$, any discrete operator $\mathcal{L}_h^a$, any router r , any*
734 *$a_h, f_h, u_h^{(t)}, u_h \in \mathcal{A} \times \mathcal{F} \times \mathcal{U} \times \mathcal{U}$, the following equality holds true:*

$$\begin{aligned}
l_{route} \left(r, a_h, f_h, u_h^{(t)}, u_h \right) &= \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad - (K - 2) \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2
\end{aligned}$$

735 *Proof.* Note that $\sum_{j=1}^K \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} = K - 1$

$$\begin{aligned}
l_{\text{route}} \left(r, a_h, f_h, u_h^{(t)}, u_h \right) &= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) = j} \\
&= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad + (K - 1) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - (K - 1) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad + \sum_{j=1}^K \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&\quad - (K - 1) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2
\end{aligned}$$

736

$$\begin{aligned}
&= \sum_{j=1}^K \left(\sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \right) \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad - (K - 2) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&= \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad - (K - 2) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2
\end{aligned}$$

737

□

738 **B.2 Proof of Theorem B.2**

739 **Proposition B.2.** For any router r defined by $r(a, f, u^{(t)}) = \operatorname{argmax}_{j \in [K]} g_j(a, f, u^{(t)})$, any $a_h \in \mathcal{A}$,
740 $f_h \in \mathcal{F}$, and $u_h^{(t)}, u_h \in \mathcal{U}$, the routing loss l_{route} satisfies:

$$\log(2) l_{\text{route}} \left(r, a_h, f_h, u_h^{(t)}, u_h \right) \leq \Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)$$

741 *Proof.* By Theorem B.1, we know that

$$\begin{aligned}
\log(2)l_{\text{route}}\left(r, a_h, f_h, u_h^{(t)}, u_h\right) &= \log(2) \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad - \log(2) (K-2) \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&\leq \log(2) \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\stackrel{(a)}{\leq} - \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \log \left(\frac{\exp(g_j(a, f, u^{(t)}))}{\sum_{k=1}^K \exp(g_k(a, f, u^{(t)}))} \right) \\
&= \Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)
\end{aligned}$$

742 (a) if $r(a_h, f_h, u_h^{(t)}) \neq j$, $\frac{\exp(g_j(a, f, u^{(t)}))}{\sum_{k=1}^K \exp(g_k(a, f, u^{(t)}))} < 0.5$ which implies that

$$743 -\log \left(\frac{\exp(g_j(a, f, u^{(t)}))}{\sum_{k=1}^K \exp(g_k(a, f, u^{(t)}))} \right) \geq \log(2) \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j}$$

744

□

745 B.3 Proof of Theorem 5.1

746 **Theorem 5.1.** Let $\tilde{c}_j(a_h, u_h^{(t)}, u_h) < \bar{E} < \infty$ for all $j \in [K]$. If there exists $j \in [K]$ such that
747 $\tilde{c}_j(a_h, u_h^{(t)}, u_h) > E_{\min} > 0$, then, for any collection of solvers $\{C_j\}_{j=1}^K$ and linear discrete
748 operator $\mathcal{L}_h^a$, Ψ is Bayes consistent surrogate for l_{route} .

749 *Proof.* For a given a_h, f_h , let u_h be $\mathcal{G}_h(a_h, f_h)$ where $\mathcal{G}_h$ denotes the solution operator acting on the
750 grid G_h . Furthermore, let's consider routers of the form

$$r(a, f, u^{(t)}) = \operatorname{argmax}_{j \in [K]} g_j(a, f, u^{(t)})$$

751 For a given $a_h, f_h, u_h^{(t)} \in \mathcal{A} \times \mathcal{F} \times \mathcal{U}$, let the optimal loss under l_{route} be
752 $l_{\text{route}}^*(a_h, f_h, u_h^{(t)}) = \inf_{\tilde{r}} l_{\text{route}}(\tilde{r}, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h))$. Similarly, let the optimal loss un-
753 der Ψ be $\Psi^*(a_h, f_h, u_h^{(t)}) = \inf_{\tilde{\mathbf{g}}} \Psi(\tilde{\mathbf{g}}, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h))$. Let $B_j(a_h, f_h, u_h^{(t)}) =$
754 $\sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(\mathcal{G}_h(a_h, f_h) - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j}$.

$$\begin{aligned}
&l_{\text{route}}\left(r, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h)\right) - l_{\text{route}}^*\left(a_h, f_h, u_h^{(t)}\right) \\
&\stackrel{(a)}{=} \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} - (K-2) \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&\quad - \inf_{\tilde{r}} \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{\tilde{r}(a_h, f_h, u_h^{(t)}) \neq j} + (K-2) \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&= \sum_{j=1}^K B_j(a_h, f_h, u_h^{(t)}) \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} - \inf_{\tilde{r}} \sum_{j=1}^K B_j(a_h, f_h, u_h^{(t)}) \mathbf{1}_{\tilde{r}(a_h, f_h, u_h^{(t)}) \neq j} \\
&= \sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)}) \left(\sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} - \inf_{\tilde{r}} \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{\tilde{r}(a_h, f_h, u_h^{(t)}) \neq j} \right)
\end{aligned}$$

755 (a) by Theorem B.1

Let $\mathcal{X} = \mathcal{A} \times \mathcal{F} \times \mathcal{U}$ and $\mathcal{Y} = [K]$. Let $\mathcal{P}_{\mathcal{X}}$ denote the degenerate distribution supported at the point $(a_h, f_h, u_h^{(t)})$. We define the conditional distribution - $P(Y = j \mid X = (a_h, f_h, u_h^{(t)})) = \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})}$ for $j \in [K]$. The risk and optimal risk of 0 – 1 loss under this distribution can be written as:

$$\begin{aligned}\mathcal{R}_{0-1}(r) &= \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\ \mathcal{R}_{0-1}^* &= \inf_r \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j}\end{aligned}$$

If $r(a_h, f_h, u_h^{(t)}) = \arg \max_{j \in [k]} g_j(a_h, f_h, u_h^{(t)})$ for all $x \in \mathcal{X}$, then the risk and optimal risk of cross entropy loss ($l_{ce}(\mathbf{g}, x, y) - \log \left(\frac{\exp(g_y(x))}{\sum_{k=1}^K \exp(g_k(x))} \right)$) under this distribution can be written as:

$$\begin{aligned}\mathcal{R}_{ce}(\mathbf{g}) &= - \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \\ \mathcal{R}_{ce}^* &= \inf_{\mathbf{g}} - \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right)\end{aligned}$$

From Theorem 3.1 of [Mao et al., 2023], $\mathcal{R}_{0-1}(r) - \mathcal{R}_{0-1}^* \leq \Gamma^{-1}(\mathcal{R}_{ce}(\mathbf{g}) - \mathcal{R}_{ce}^*)$ if $r(a_h, f_h, u_h^{(t)}) = \arg \max_{j \in [k]} g_j(a_h, f_h, u_h^{(t)})$ where $\Gamma(z) = \frac{1+z}{2} \log(1+z) + \frac{1-z}{2} \log(1-z)$. Then,

$$\begin{aligned}
& l_{\text{route}} \left(r, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h) \right) - l_{\text{route}}^* \left(a_h, f_h, u_h^{(t)} \right) \\
&= \sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)}) \left(\sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} - \inf_{\tilde{r}} \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{\tilde{r}(a_h, f_h, u_h^{(t)}) \neq j} \right) \\
&\leq \sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)}) \Gamma^{-1} \left(- \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right. \\
&\quad \left. - \inf_{\mathbf{g}} - \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right) \\
&\stackrel{(a)}{\leq} \bar{E} K (K-1) \Gamma^{-1} \left(- \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right. \\
&\quad \left. - \inf_{\mathbf{g}} - \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right) \\
&\stackrel{(b)}{\leq} \bar{E} K (K-1) \Gamma^{-1} \left(- \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{(K-1)E_{\min}} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right. \\
&\quad \left. - \inf_{\mathbf{g}} - \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{(K-1)E_{\min}} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right) \\
&= \bar{E} K (K-1) \Gamma^{-1} \left(\frac{\Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h)) - \Psi^*(a_h, f_h, u_h^{(t)})}{(K-1)E_{\min}} \right)
\end{aligned}$$

765 (a) since $\left\| (I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)}) \right\|_2^2 < \bar{E}$ for all $j \in [K]$, (b) since Γ^{-1} is non-decreasing and

766 $\exists j \in [K]$ such that $\left\| (I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)}) \right\|_2^2 > E_{\min}$

767 Finally,

$$\begin{aligned}
& \lim_{n \rightarrow \infty} \mathcal{R}_{\text{route}}(r_n) - \mathcal{R}_{\text{route}}^* \\
&\stackrel{(a)}{=} \lim_{n \rightarrow \infty} \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} \left[l_{\text{route}} \left(r_n, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h) \right) - l_{\text{route}}^* \left(a_h, f_h, u_h^{(t)} \right) \right] \\
&\leq \lim_{n \rightarrow \infty} \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} \left[\bar{E} K (K-1) \Gamma^{-1} \left(\frac{\Psi(\tilde{\mathbf{g}}, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h)) - \Psi^*(a_h, f_h, u_h^{(t)})}{(K-1)E_{\min}} \right) \right] \\
&\stackrel{(b)}{\leq} \lim_{n \rightarrow \infty} \bar{E} K (K-1) \Gamma^{-1} \left(\frac{\mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} \left[\Psi(\mathbf{g}_n, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h)) - \Psi^*(a_h, f_h, u_h^{(t)}) \right]}{(K-1)E_{\min}} \right)
\end{aligned}$$

$$\begin{aligned}
&= \lim_{n \rightarrow \infty} \bar{E} K (K - 1) \Gamma^{-1} \left(\frac{\mathcal{R}_\Psi(\mathbf{g}_n) - \mathcal{R}_\Psi^*}{(K - 1)E_{min}} \right) \\
&\stackrel{(c)}{=} \bar{E} K (K - 1) \Gamma^{-1} \left(\frac{\lim_{n \rightarrow \infty} \mathcal{R}_\Psi(\mathbf{g}_n) - \mathcal{R}_\Psi^*}{(K - 1)E_{min}} \right) \\
&= \bar{E} K (K - 1) \Gamma^{-1}(0) \\
&\stackrel{(d)}{=} 0
\end{aligned}$$

769 (a) $\mathcal{R}_{\text{route}}^* = \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} \left[l_{\text{route}}^* \left(a_h, f_h, u_h^{(t)} \right) \right]$ since the infimum is taken over all measurable
770 functions, (b) by Jensen's inequality since Γ^{-1} is concave, (c) by continuity of Γ^{-1} at 0, (d) $\Gamma^{-1}(0) =$
771 0

772

□

773 C Training Details

774 The periodic experiments of Section 7 use the corrector and the lightweight router described in
775 Section E.1; the data distribution below is shared by all experiments, and the DeepONet and LSTM
776 router described afterwards are those of the Dirichlet experiment in Section D.1.

777 Data for both DeepONet and the routers is sampled from a zero-mean Hierarchical Gaussian Random
778 Field on the periodic domain with covariance operator $\alpha(-\Delta + \beta I)^{-\gamma}$, where α, β, γ are sampled
779 as:

$$\begin{aligned}
\alpha &\sim \text{Log-Uniform}(0.01, 100) \\
\beta &\sim \text{Log-Uniform}(0.1, 1000) \\
\gamma &\sim \text{Uniform}(\{0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 4.0\})
\end{aligned}$$

780 Given (α, β, γ) , samples are generated in the Fourier space. For each non-zero frequency mode
781 k , we draw an independent complex coefficient from a Gaussian distribution with mean 0 and
782 variance $\alpha(4\pi^2\|k\|_2^2 + \beta)^{-\gamma}$. We enforce a Hermitian symmetry to obtain a real-valued field, set the
783 zero-frequency (DC) mode to 0 to obtain a zero mean field, and apply the inverse Discrete Fourier
784 Transform to obtain the field in physical space.

785 For each sample, we compute reference solutions with a least squares solver and treat them as ground
786 truth.

787 This data is used to trained our DeepONet models and LSTM routers. All the models were imple-
788 mented using PyTorch and all the models were trained on one Nvidia A40 GPU.

789 Table 2 contains all hyperparameter details for the DeepONet. DeepONet took 30 minutes to train.
790 We then use the model with the best validation loss.

Table 2: Hyperparameter settings for DeepONet

Hyperparameter	Value
Learning rate	1e-3
Branch Dimension	128
Hidden dimension for branch net	256
No. of hidden layers in branch net	4
Hidden dimension for trunk net	256
No. of hidden layers in trunk net	4
Gradient Clipping Norm	1.0
Weight Decay	0.005
Batch size	256
Training samples	10000
Validation samples	2000
Epochs	1000

791 The routers are LSTM models. Along with the inputs specified in Section 5, namely $a_h, f_h, u_h^{(t)}$,
792 we also supply the routers with the iteration index t , current residual $f_h - \mathcal{L}_h^a u_h^{(t)}$, and the routing
793 decisions from the previous iteration. While the iteration index and residual are deterministic
794 functions of $(a_h, f_h, u_h^{(t)})$ and align with the theory presented in Section 5, incorporating previous
795 routing decisions is a mild deviation from the idealized setting. Nevertheless, this modification is
796 motivated by the same considerations that justify the use of scheduled sampling: under deployment,
797 routing decisions are predicted autoregressively and influence future inputs, whereas training under
798 teacher forcing assumes independence from past predictions. Including previous routing decisions
799 bridges the gap induced by exposure bias by providing the model with trajectory-level information.

800 All the routers are trained with scheduled sampling. We use a warm-up of e_w epochs with teacher-
801 forcing probability $p_{tf}(e) = ss_{start}$. After the warm-up, the p_{tf} decays geometrically by a factor of
802 $\gamma_{tf} < 1$ per epoch and is floored by ss_{end} :

$$p_{tf}(e) = \begin{cases} ss_{start} & e \leq e_w \\ \max(ss_{start}\gamma_{tf}^{e-e_w}, ss_{end}) & e > e_w \end{cases}$$

803 At each time step, with probability $p_{tf}(e)$, we feed the teacher-forced greedy iterate; otherwise, we
804 feed the router’s own predicted iterate.

805 Since LSTMs on long rollouts can suffer from exploding/vanishing gradients, we use truncated
806 backpropagation through time (TBPTT) [Mozier, 2013, Robinson and Fallside, 1987, Werbos, 1988]:
807 the forward pass unrolls the entire trajectory, but gradients are propagated only through the most
808 recent $w_{bptt}(e)$ steps at epoch e . Hidden states are passed forward between segments, while earlier
809 segments are treated as stop-gradient.

810 We employ a curriculum learning approach analogous to scheduled sampling. Let T_{max} be the
811 horizon (300 iterations). With a warm-up of e_w epochs,

$$w_{bptt}(e) = \begin{cases} w_{start} & e \leq e_w \\ \min\left(T_{max}, w_{start}\gamma_{bptt}^{\lfloor \frac{e-e_w}{f_{bptt}} \rfloor}\right) & e > e_w \end{cases} \quad (12)$$

812 so the window grows geometrically by a factor of $\gamma_{bptt} > 1$ every f_{bptt} epochs and is capped at the
813 full trajectory length.

814 Table 3 contains all hyperparameter details for the LSTM routers. The routers for the Paired Solver
815 experiment took a maximum of 4 hours and 30 minutes to train while the routers for the Solver
816 Ensemble experiment took a maximum of 6 hours to train. We then use the model with the best
817 validation loss for testing. The architecture and dataset size is larger for the Solver ensemble
818 experiments to encourage the model to learn some of the nuanced differences between the classes.

Table 3: Hyperparameter settings for routers in Paired solver and Solver Ensemble Experiments

Hyperparameter	Paired Solver	Solver Ensemble
Learning rate	1e-3	1e-3
Hidden dimension	256	512
No. of hidden layers	4	3
Gradient Clipping Norm	1.0	1.0
Weight Decay	0.005	0.005
Batch size	32	32
Training samples	256	1024
Validation samples	32	128
Epochs	200	200
ss_{start}	1.0	1.0
γ_{tf}	0.95	0.95
ss_{end}	0.0	0.0
w_{start}	50	50
γ_{bptt}	1.25	1.25
e_w	10	10
f_{bptt}	1	1

D Additional Experimental Results

This appendix keeps the one experiment of the original submission that the wall-clock study of Section E does not supersede: the paired-solver comparison under Dirichlet boundary conditions, which was run with the original pipeline (a 31×31 grid, a DeepONet with an MLP branch and an LSTM router trained with scheduled sampling; Section C). The other results of the original submission—iteration-based comparisons on 31^2 and 63^2 periodic grids with the LSTM router, residual and Fourier-mode analyses, and ensemble decision traces—are superseded by the per-equation results, the usage analyses and the ensemble study of Section E and have been removed.

D.1 Results with Dirichlet Boundary Conditions

We replicated the paired-solver experiment of the original submission for solutions with zero Dirichlet boundary conditions, with the original pipeline; the metrics are the final error and AUC over 300 iterations, with paired t -tests against the learned router. The dense symmetric-GS reference of that pipeline mis-implemented the SSOR scaling (it added the factor $\omega/(2 - \omega)$ to D^{-1} instead of multiplying it), so the SymGS rows below use a convergent but non-standard symmetric sweep; the implementation has since been corrected and the wall-clock study uses the standard SSOR.

In Table 4, we notice that, similar to PDEs with periodic boundary conditions, the learned greedy router closely matches the performance of the true greedy oracle and outperforms its single solver and HINTS baselines with great statistical significance. Despite the learned router matching the performance of the true greedy oracle, we observe that the learned router doesn't seem to provide much improvement over single-solver baseline in the case of SOR (1.5)-related solvers. This may be a result of the deepnet correction always being weaker relative to the SOR (1.5) correction in terms of error reduction.

Table 4: Final error and AUC of squared L^2 error (lower is better). Values are mean ($\pm$ standard deviation) over 128 test instances with Dirichlet boundaries; both are reported in $\times 10^{-3}$. If a standard error is not shown, it is $< 10^{-3}$ in the reported units (raw $< 10^{-6}$). Statistical significance is assessed via paired t -tests comparing each baseline (single-solver only and HINTS) against the learned greedy router, using a one-sided alternative that the learned router achieves lower error/AUC. Reported p -values correspond to these tests.

Equation	Poisson				ConvDiff			
Methods	$\ e_h^{(T)}\ \times 10^3$	$\ e_h^{(T)}\ $ p -value	AUC	AUC p -value	$\ e_h^{(T)}\ \times 10^3$	$\ e_h^{(T)}\ $ p -value	AUC	AUC p -value
Jacobi-related solvers								
Jacobi Only	0.704 (2.151)	0.001	0.703 (1.929)	$< 10^{-3}$	0.003 (0.006)	0.057	0.169 (0.430)	$< 10^{-3}$
HINTS-Jacobi	2.674 (7.315)	$< 10^{-3}$	0.869 (2.396)	$< 10^{-3}$	0.474 (0.001)	$< 10^{-3}$	0.199 (0.254)	$< 10^{-3}$
Learned Greedy-Jacobi	0.243 (0.851)	-	0.236 (0.719)	-	0.003 (0.006)	-	0.076 (0.162)	-
True-Greedy-Jacobi	0.042 (0.047)	-	0.099 (0.252)	-	0.003 (0.007)	-	0.042 (0.089)	-
GS-related solvers								
GS only	0.138 (0.431)	0.001	0.408 (1.139)	$< 10^{-3}$	0.003 (0.006)	0.028	0.057 (0.146)	$< 10^{-3}$
HINTS-GS	2.668 (7.316)	$< 10^{-3}$	0.841 (2.360)	$< 10^{-3}$	0.470 (0.001)	$< 10^{-3}$	0.115 (0.137)	$< 10^{-3}$
Learned Greedy-GS	0.032 (0.158)	-	0.115 (0.433)	-	0.003 (0.006)	-	0.024 (0.048)	-
True-Greedy-GS	0.010 (0.015)	-	0.063 (0.166)	-	0.003 (0.006)	-	0.017 (0.036)	-
SymGS-related solvers								
SymGS only	2.924 (8.865)	0.01	0.926 (2.740)	0.006	1.331 (3.687)	0.004	0.413 (1.133)	0.004
HINTS-SymGS	2.748 (7.375)	0.008	0.856 (2.427)	0.008	0.917 (1.015)	0.006	0.332 (0.697)	0.003
Learned Greedy-SymGS	1.333 (5.715)	-	0.416 (1.722)	-	0.628 (1.468)	-	0.196 (0.449)	-
True-Greedy-SymGS	0.342 (0.906)	-	0.147 (0.375)	-	0.184 (0.323)	-	0.067 (0.118)	-
Jacobi (0.67)-related solvers								
Jacobi (0.67) only	1.265 (3.768)	$< 10^{-3}$	0.910 (2.468)	$< 10^{-3}$	0.008 (0.021)	0.003	0.253 (0.642)	$< 10^{-3}$
HINTS-Jacobi (0.67)	2.686 (7.311)	$< 10^{-3}$	0.886 (2.412)	$< 10^{-3}$	0.487 (0.002)	$< 10^{-3}$	0.220 (0.279)	$< 10^{-3}$
Learned Greedy-Jacobi (0.67)	0.134 (0.275)	-	0.215 (0.660)	-	0.004 (0.010)	-	0.127 (0.393)	-
True-Greedy-Jacobi (0.67)	0.096 (0.220)	-	0.145 (0.408)	-	0.004 (0.013)	-	0.061 (0.131)	-
SOR (1.5)-related solvers								
SOR (1.5) only	0.004 (0.011)	0.012	0.145 (0.412)	$< 10^{-3}$	0.003 (0.006)	1.0	0.006 (0.014)	1.0
HINTS-SOR (1.5)	2.650 (7.321)	$< 10^{-3}$	0.791 (2.295)	$< 10^{-3}$	0.468 (0.001)	$< 10^{-3}$	0.014 (0.014)	$< 10^{-3}$
Learned Greedy-SOR (1.5)	0.004 (0.011)	-	0.039 (0.091)	-	0.003 (0.006)	-	0.006 (0.014)	-
True-Greedy-SOR (1.5)	0.004 (0.011)	-	0.028 (0.070)	-	0.003 (0.006)	-	0.006 (0.014)	-

E Cost-Aware Routing and Wall-Clock Evaluation

The experiments of Section 7 compare methods per iteration, which favors methods whose iterations are expensive. This appendix evaluates the end-to-end wall-clock time to a target accuracy of the learned router against HINTS, the classical solver alone and the classical methods a practitioner would reach for on these problems (geometric multigrid and preconditioned Krylov methods),

for all five classical solvers and three PDEs, on 128^2 , 256^2 and 512^2 grids, using the cost-aware instantiation of the greedy rule introduced at the end of Section 4. Besides the Poisson and convection–diffusion equations of Section 7 we add *variable-coefficient diffusion* (Section E.5), on which no Fourier diagonalisation exists and the corrector must be learned from data, and *anisotropic diffusion*, $-\epsilon \partial_{x_1}^2 u - \partial_{x_2}^2 u = f$ with $\epsilon = 10^{-2}$ (diffusion tensor $\text{diag}(\epsilon, 1)$), on the same periodic domain with the same forcing distribution. It is the standard model problem on which point-relaxation methods fail to smooth: an error component oscillatory in x_1 and smooth in x_2 is damped by only $(1 - \epsilon)/(1 + \epsilon) \approx 0.98$ per Jacobi sweep, and geometric multigrid with point smoothing loses its grid-independent convergence (line relaxation or semi-coarsening would be needed). Section E.1 describes the protocol; Sections E.2 to E.5 collect all results per equation (every grid, every tolerance, every baseline, with significance tests and seed trials); Section E.7 studies ensembles of one, two and three classical solvers; Section E.8 accounts for overheads and training cost; and Section E.10 verifies numerically that the assumptions of Theorem 4.1, Propositions 4.2–4.3 and Theorem 5.1 hold in these settings.

E.1 Protocol

Fast, matched implementations. A wall-clock comparison is meaningful only if the classical baseline is not handicapped by implementation. All classical operations are therefore implemented as compiled, single-threaded C kernels on the periodic 5-point stencil (`stencil.c`, built with `cc -O3`): the residual, fused (damped) Jacobi sweeps, in-place lexicographic Gauss–Seidel, SOR and symmetric SOR sweeps, and the full-weighting and bilinear transfers of geometric multigrid, whose V(2,2)-cycles use Gauss–Seidel smoothing, rediscrctised coarse operators and a direct solve on the 32×32 coarsest grid (error contraction ≈ 0.03 per cycle on the isotropic equations). At 128^2 a residual evaluation costs $10 \mu\text{s}$, a Gauss–Seidel sweep $41 \mu\text{s}$ and a V-cycle 0.34 ms , against 0.57 ms for one corrector call (a BLAS matrix–vector product), so the corrector is worth about 29 Jacobi sweeps, 11 Gauss–Seidel sweeps or 1.7 V-cycles there (Table 35); an earlier version of this study used numpy/scipy sweeps that were 4–5 $\times$ slower and correspondingly overstated the router’s margins (Section E.9). The compiled kernels agree with the numpy/scipy implementations they replace to 2×10^{-14} (relative), and their operators and one-step iterates match the dense reference implementations of the original submission to the reference’s own precision (relative error below 10^{-7} , the reference assembling its matrices in single precision; 10^{-12} where the reference is exact); the comparison is an assertion in `validate_fast_pde.py`. In the course of this check we found that the original dense symmetric-GS reference mis-implemented the SSOR scaling (the factor $\omega/(2 - \omega)$ was added to D^{-1} instead of multiplying it); it has been corrected, and the wall-clock study uses the standard SSOR throughout. Reference solutions are computed by FFT diagonalization of the same discrete operator, i.e., they are exact solutions of the discrete system, so true errors are available for evaluation at no charged cost; the FFT solve is also reported as a baseline in every table (it is a direct method that exists because these test problems have constant coefficients and periodic boundaries, and it is faster than every iterative method, including ours; the variable-coefficient problem of Section E.5 has no such solve). All timings are single-threaded CPU measurements (Apple M4 Pro, numpy/scipy/PyTorch, one thread) on an otherwise idle machine.

Corrector. The neural corrector is a DeepONet applied to the residual $r = f_h - \mathcal{L}_h^a u_h^{(t)}$ and evaluated on a 64×64 sensor grid, i.e., on the coarser grid obtained by band-limited (FFT) restriction to the modes $|k_1|, |k_2| \leq 31$ at every grid size (for the anisotropic equation the sensor grid is $N \times N/4$, i.e., full resolution along x_1 , so that the band contains every mode the point smoothers cannot damp; the sensor grid is the only PDE-specific hyperparameter of the pipeline; the variable-coefficient equation uses the isotropic 64×64 grid); its output is prolonged back to the fine grid by exact trigonometric interpolation, so the correction is band-limited by construction and never injects error into the high-frequency band that the classical smoother owns (Figure 2). The trunk net is the fixed orthonormal Fourier basis of that band ($p = 3969$ basis functions, as in POD-DeepONet [Lu et al., 2022]), and the branch net is a linear layer on the 4096 sensor values; it is fit by least squares on 32,000–64,000 exact (residual, error) pairs of the discrete operator drawn from a mixture of spectral shapes (forcing-like, residual-like, white and randomly tilted spectra), and reaches a validation relative error of $\approx 2 \times 10^{-6}$ on every mode of the band. The corrector is applied scale-equivariantly, $C_{\text{NO}}(r) = \|Rr\| \mathcal{G}_\theta(Rr/\|Rr\|)$, which preserves $C_{\text{NO}}(0) = 0$ (Proposition 4.2). Its deployed form is therefore a linear, Fourier-structured map—a least-squares approximation of the exact band-limited inverse $P(\mathcal{L}_h)_{\text{band}}^{-1} R$ of the discrete operator. Replacing it by that exact inverse changes nothing:

on the GS pairing at 128^2 the cost-aware oracle’s median work-unit time to 10^{-8} is 4.472 ms with either corrector and its decision sequence is identical on 13 of 16 instances. The gains reported below are thus those of an accurate band-limited spectral correction applied at the right moments; the learned corrector stands in for a spectral inverse that, on these constant-coefficient periodic problems, could also be computed directly. Its cost is dominated by a 4096×4096 matrix–vector product and is therefore almost independent of N (0.65 ms at 128^2 , 3.6 ms at 512^2 , Table 35), i.e., about 12 Jacobi or 3 GS sweeps at 128^2 and about 2.5 Jacobi sweeps at 512^2 (Tables 8, 14 and 20). The same corrector is used by HINTS and by all routing policies.

Cost-equalised macro-actions. Per-iteration costs c_j (residual evaluation plus the update) are measured once per pairing and grid on the benchmark machine (minimum over seven repeated blocks of the block median, cached), the unit of cost u is one corrector call, and every operation is applied $m_j = \max\{1, \text{round}(u/c_j)\}$ times per decision; for Jacobi, damped Jacobi, GS and SOR the measured $m_j c_j / u$ lies in $[0.77, 1.18]$ across all grids, while symmetric GS (1.15–1.83 units on the larger grids, $m_j = 1$) and multigrid (1.8–7.6 units) are dearer than the unit and are compared per unit of cost (Tables 8, 14 and 20). An operation dearer than the unit (a multigrid cycle) is applied once and compared per unit of cost, $\|e_j\|^{u/(m_j c_j)}$; the surrogate weights $\tilde{c}_j$ use the same exponent. Ensembles reuse the cached pairwise costs of their members, so every member has the same macro-action size in the ensemble as in its pairwise run. With the compiled kernels a multigrid cycle is cheaper than a corrector call at 128^2 and 256^2 , so two cycles form one macro-action there. The cost-aware oracle runs Algorithm 1 on these macro-actions with the true error; the *greedy oracle* rows run the cost-agnostic Algorithm 1 on single operations, as in the main text.

Lightweight router. The learned router selects macro-actions from $6 + K$ scalar features, all $O(1)$ given the residual norm that any stopping test computes: the log relative residual, its per-iteration change over the last macro-action and an exponential moving average thereof, the log epoch index, a one-hot encoding of the previous macro-action, and the log counts of corrector calls so far and of iterations since the last call. It is a two-hidden-layer MLP (64 units) evaluated in numpy, costing $\approx 5 \mu\text{s}$ per decision independently of N . It is trained with the surrogate of Section 5.1, with the per-state weights $\tilde{c}_k$ normalised by $\sum_k \tilde{c}_k$ (which does not change the Bayes decision), on states visited by oracle rollouts with ε -exploration (probability 0.1), followed by two to three DAgger rounds [Ross et al., 2011] in which the current router’s own rollouts are relabelled by the oracle—the batch analogue of the scheduled sampling used in Section C. Training uses 32–128 instances per round (fewer on the larger grids) drawn from the same GRF distribution as the test set but with a different seed; the router of every pairing is trained in seconds to a few minutes. Ensemble routers, which must additionally learn to choose among classical members, use a larger imitation budget (256 rollouts at 128^2 and 128 at 256^2 , six DAgger rounds, hidden width 128, 300 epochs): with the pairwise recipe the router over $\{\text{Jacobi}, \text{Jacobi} (0.67)\}$ imitated its oracle imperfectly at tight tolerances (convection–diffusion, 128^2 , $\varepsilon = 10^{-8}$: 3.34 against the oracle’s 2.67 work-unit ms), which the larger budget removes (2.70). At 512^2 the routers are trained from 32 oracle rollouts of at most 400 iterations (stopping error 10^{-8}); for the convection–diffusion Jacobi pairing, where the first such router missed h^2 within the cap on 4 of 16 instances, the reported router uses 64 rollouts of at most 800 iterations—the only per-cell change to the recipe.

Baselines and metric. HINTS applies the corrector every τ -th iteration, first at iteration τ (Section 3); we report $\tau = 25$, the schedule of Zhang et al. [2024], and the *best fixed schedule* selected *per cell and per tolerance* on the test instances themselves (an optimistic baseline) from the family of HINTS with $\tau \in \{2, 5, 10, 25, 50\}$, phase-shifted HINTS with the first call at iteration 0 and $\tau \in \{5, 10, 25, 50\}$, and the *one-shot schedule*—the corrector once at iteration 0, then the classical solver only—which is the static rule closest to what the router learns at truncation level and therefore the natural ablation for state-dependent routing ($\tau = 1$, the corrector at every iteration, never reduces the error outside the corrector’s band and is omitted). The classical baselines without corrector run on the same stencil with the same accounting: multigrid V(2,2) alone; conjugate gradients, CG preconditioned by symmetric GS and by a symmetric V-cycle for Poisson; BiCGSTAB, multigrid-preconditioned BiCGSTAB and restarted GMRES(20) for convection–diffusion (GMRES is accounted at the granularity of its restart cycles and was not run at 512^2 , where it was two orders of magnitude slower than the router on the smaller grids). They are not run for anisotropic diffusion, where point-smoothed multigrid loses its grid-independent contraction (≈ 0.85 per cycle) and would not be informative. The Krylov methods are timed in their SciPy implementations, which keep their

own residual bookkeeping instead of the benchmark’s per-iteration true-error check; their iteration counts come from the untimed pass with a per-iteration callback, so their timed runs carry less per-iteration overhead than the hybrid policies, not more. All configuration choices of the study were made on a development test set (GRF seed 72, with the earlier numpy kernels; archived as `results_dev/`); the results reported here are a single confirmatory evaluation of the frozen configurations on a fresh test set (seed 73) with three timed replays per instance, and Section E.9 places the two side by side. For each test instance (64 at 128^2 , 32 at 256^2 , 16 at 512^2) we record, after every iteration, the true relative error $\|e_h^{(t)}\|/\|u_h\|$ (untimed, benchmark-only) and the cumulative charged wall-clock time of a replay that executes only the chosen operations (residual, update, and—for the learned router—every feature and decision computation; oracle decisions are not charged). We report the median first time at which the error drops below a tolerance ε , paired speedups—the median over instances of the per-instance time ratio, which is the estimand behind every speedup, bold mark and test in this appendix (medians of raw times are shown for scale and can order differently from the paired median on small samples)—and one-sided Wilcoxon signed-rank tests on the paired log time ratios (alternative: the router is faster); the study performs several hundred such tests, so the speedup tables mark as significant only comparisons that survive a Holm correction at level 0.05 over all tests of the table; a run that does not reach ε within the iteration cap (60,000 at 128^2 , 40,000 at 256^2 , 8,000 at 512^2) is marked $\dagger$ with the number of censored instances, reported as a lower bound when it is the majority, counted against the router when it is the router’s, and entered at the time it spent up to the cap on the baseline side of the tests—a labelled lower-bound analysis rather than a survival analysis. Timed replays are repeated three times per instance in a random policy order, each replay preceded by an untimed warm-up (one corrector call and one sweep) so that every policy starts from the same cache state—the corrector’s matrix exceeds the CPU caches, and without the warm-up a policy’s first call was up to 0.1 ms slower or faster depending on which policy had run before it; the median over replays is used. The cached per-operation costs (minimum over blocks of the block median) are a lower envelope that is used only to set the macro-action sizes and to compute work units; all reported live times are measured directly. We highlight $\varepsilon = h^2$ as a nominal grid-dependent tolerance and verified that it lies below the discretisation error of the test instances: comparing the exact discrete solution with the exact discrete solution on a four times finer grid for the same forcing (`discretization_error.py`), the relative discretisation error has median 3.1×10^{-4} , 7.3×10^{-4} and 2.5×10^{-4} at 128^2 and 7.1×10^{-5} , 1.6×10^{-4} and 5.7×10^{-5} at 256^2 for Poisson, convection–diffusion and anisotropic diffusion (4 – $12 h^2$; the roughest instances reach $70 h^2$), so driving the algebraic error below h^2 does not improve the computed solution on any instance; we also report 10^{-3} and 10^{-8} , and the iteration-based metrics of the main text (final relative error and $\text{AUC} = \sum_{t=1}^T \|e_h^{(t)}\|/\|u_h\|$ over $T = 300$ iterations) on the 128^2 grid. Every router was additionally retrained from scratch with five independent seeds (different training instances, exploration and initialisation; the corrector and the test set are fixed) and re-run on the test set; because those trials ran in separate sessions, whose live timings are not comparable with the main run’s, they are compared in *work units* (measured per-iteration costs times executed operations, plus $5 \mu\text{s}$ per decision), which are determined by the decision sequence alone.

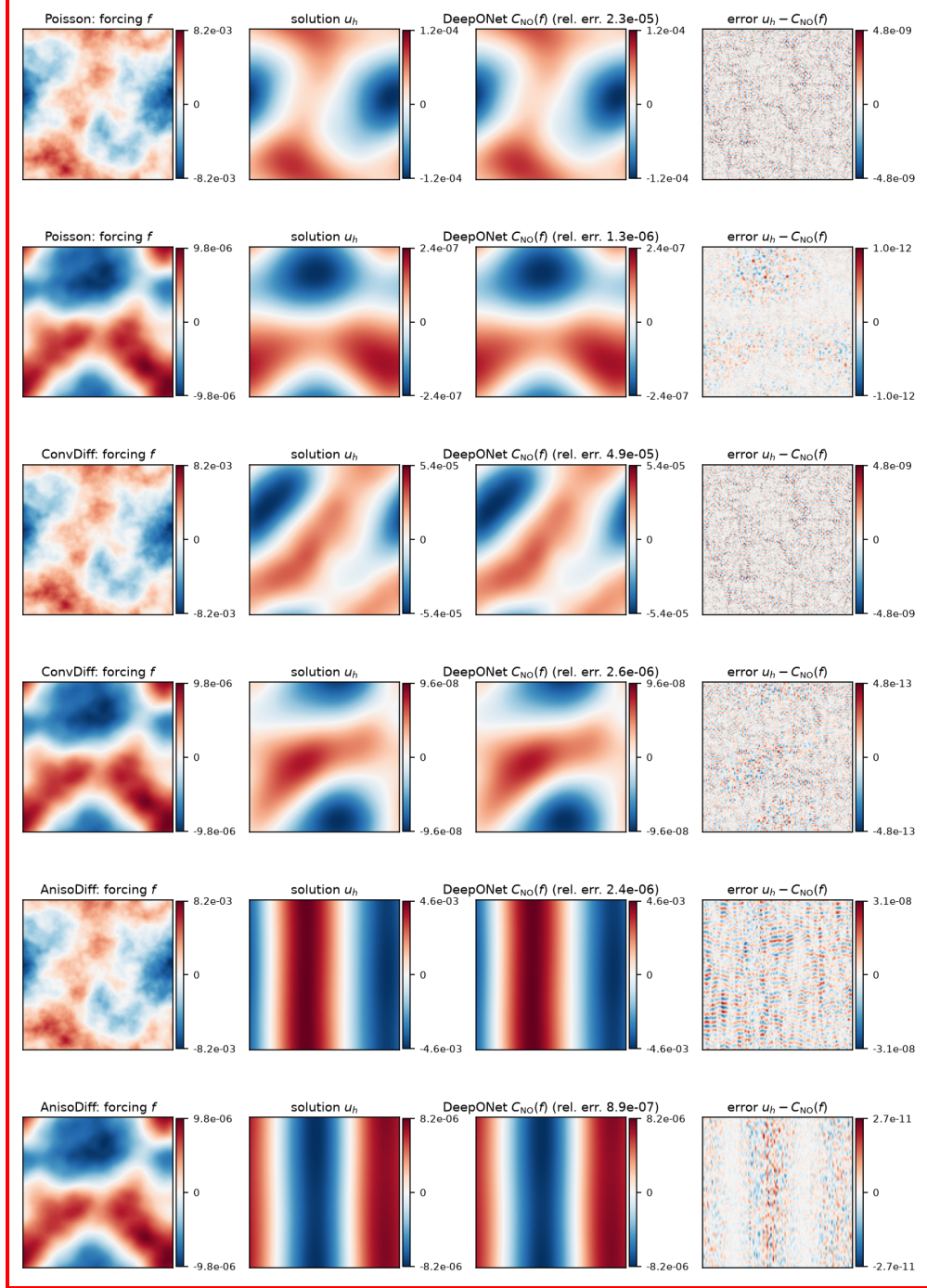


Figure 2: One-shot corrector predictions on the 128×128 grid (two test instances per equation): forcing, exact discrete solution, the DeepONet correction applied to the initial residual ($u_h^{(0)} = 0$), and the remaining error, which is confined to the modes outside the corrector's band (the relative error of the prediction is 10^{-6} – 10^{-5} on the isotropic equations and 10^{-6} on anisotropic diffusion, whose solution is dominated by modes that are smooth in x_2).

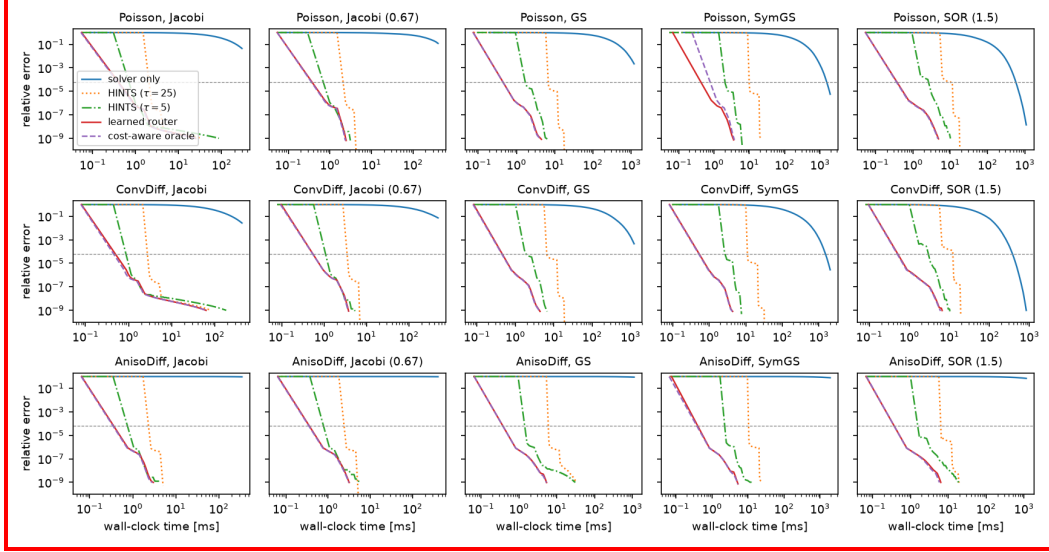


Figure 3: Relative error versus charged wall-clock time on the 128×128 grid for one representative test instance of every pairing (columns) and equation (rows); the dashed line is $\varepsilon = h^2$. The learned router and the cost-aware oracle are indistinguishable.

998 E.2 Poisson

999 Table 5 collects the median time to 10^{-3} , h^2 and 10^{-8} for every pairing, every policy and the classical
1000 baselines on all three grids; Table 6 the paired speedups of the learned router with their p -values;
1001 Figure 4 and table 7 where the iterations go; Table 8 the measured costs and macro-action sizes;
1002 Table 9 the iteration-based metrics; and Table 10 the seed trials.

1003 **Two regimes.** At $\varepsilon = h^2$ the whole solve consists of one corrector call and a few sweeps (Table 7):
1004 the router, the cost-aware oracle, phase-shifted HINTS and the one-shot schedule take the same
1005 decisions and their times coincide up to timer resolution, so the router’s paired ratio against the best
1006 fixed schedule is close to one (Table 6; bold marks the comparisons that remain significant after the
1007 Holm correction), while HINTS with its published period, whose first call comes after $\tau - 1$ wasted
1008 sweeps, is several times slower, and the solver alone hundreds to thousands of times slower. Below
1009 h^2 the remaining error is the band the corrector cannot see, and the schedules separate: the router
1010 calls the corrector again only when the smooth error has re-emerged (two or three calls down to
1011 10^{-8} ; Figure 4), the one-shot schedule never calls it again and pays for hundreds of extra sweeps,
1012 and every periodic schedule keeps paying for calls that no longer help, so at 10^{-8} the router is
1013 significantly faster than the best fixed schedule in every pairing. The cost-agnostic greedy oracle
1014 (Algorithm 1 on single operations) makes the same decisions as its cost-aware counterpart on the
1015 path to h^2 (they differ later, in when the corrector is called again); the learned router reproduces the
1016 cost-aware oracle’s decisions almost exactly (same median iteration and call counts, median times
1017 within a few percent), and the five retrained routers reproduce the main router’s decision sequence to
1018 h^2 on essentially every test instance (Table 10).

1019 **Classical baselines and larger grids.** Against the classical methods without corrector (bottom
1020 block of Table 5), the router with its best stationary pairing reaches truncation-level accuracy faster
1021 than multigrid alone and than multigrid-preconditioned CG at 128^2 and 256^2 (Table 6), because
1022 one corrector call removes the modes $|k| \leq 31$ to 10^{-6} at the price of one matrix–vector product
1023 whereas a V(2,2)-cycle contracts the whole spectrum by ≈ 0.03 ; routing over {NO, multigrid}
1024 calls the corrector once and lets the cycles remove the rest. The FFT direct solve, exact for this
1025 constant-coefficient periodic problem, is faster than every iterative method at every grid. At 256^2 and
1026 512^2 the same two regimes hold, with shrinking margins for the Jacobi and Gauss–Seidel pairings
1027 because the corrector’s fixed band $|k| \leq 31$ leaves the smoother all modes $k \geq 32$, whose slowest
1028 member contracts by only 0.98 per Jacobi sweep at $N = 512$ (against 0.71 at $N = 128$): every

1029 schedule then needs the same ≈ 80 sweeps after the first call, the wall-clock time is dominated by
1030 sweeps, and the corrector—whose cost is nearly grid-independent—is worth fewer and fewer of them,
1031 so the best classical method at 512^2 is multigrid, alone or with one corrector call. This is the expected
1032 limit of a corrector whose band does not grow with the grid.

Table 5: Poisson: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 , 256^2 and 512^2 grids (64, 32 and 16 test instances) for every pairing $\{\text{NO, solver}\}$ and policy, and for the classical baselines without corrector. “HINTS (best τ)” is the best of $\tau \in \{5, 10, 25, 50\}$ chosen per cell on the test set itself (the chosen τ is the subscript). The greedy oracle is the cost-agnostic Algorithm 1, the cost-aware oracle is Algorithm 1 on cost-equalised macro-actions; both use the true error and are not deployable. Bold marks the router where it is the fastest deployable method of its pairing; † n : n instances did not reach the tolerance within the iteration cap ($>$: lower bound).

Pairing	Method	128 × 128		
		$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$
{NO, Jacobi}	Solver only	272.2 ms	383.3 ms	729.7 ms
	HINTS ($\tau=25$)	1.37 ms	1.38 ms	51.1 ms
	Best fixed schedule	763 $\mu\text{s}_{\text{one-shot}}$	774 $\mu\text{s}_{\tau=10, t_0=0}$	37.0 $\text{ms}_{\tau=50}$
	One-shot schedule	763 μs	779 μs	67.3 ms
	Greedy oracle (Alg. 1)	767 μs	775 μs	24.1 ms
	Cost-aware oracle	773 μs	784 μs	24.2 ms
	Learned router (ours)	794 μs	815 μs	24.1 ms
{NO, Jacobi (0.67)}	Solver only	392.7 ms	554.1 ms	1.06 s
	HINTS ($\tau=25$)	1.35 ms	1.35 ms	2.66 ms
	Best fixed schedule	750 $\mu\text{s}_{\tau=10, t_0=0}$	765 $\mu\text{s}_{\tau=10, t_0=0}$	1.91 $\text{ms}_{\tau=10, t_0=0}$
	One-shot schedule	752 μs	769 μs	82.8 ms
	Greedy oracle (Alg. 1)	747 μs	759 μs	1.83 ms
	Cost-aware oracle	749 μs	766 μs	2.42 ms
	Learned router (ours)	784 μs	802 μs	2.51 ms
{NO, GS}	Solver only	345.5 ms	485.8 ms	924.8 ms
	HINTS ($\tau=25$)	2.33 ms	2.38 ms	4.99 ms
	Best fixed schedule	748 $\mu\text{s}_{\tau=5, t_0=0}$	769 $\mu\text{s}_{\tau=5, t_0=0}$	2.14 $\text{ms}_{\tau=10, t_0=0}$
	One-shot schedule	760 μs	781 μs	101.9 ms
	Greedy oracle (Alg. 1)	749 μs	781 μs	2.41 ms
	Cost-aware oracle	745 μs	763 μs	2.27 ms
	Learned router (ours)	776 μs	802 μs	2.37 ms
{NO, SymGS}	Solver only	254.0 ms	357.4 ms	680.6 ms
	HINTS ($\tau=25$)	3.07 ms	3.13 ms	6.13 ms
	Best fixed schedule	749 $\mu\text{s}_{\tau=25, t_0=0}$	772 $\mu\text{s}_{\tau=25, t_0=0}$	2.02 $\text{ms}_{\tau=5, t_0=0}$
	One-shot schedule	756 μs	787 μs	70.6 ms
	Greedy oracle (Alg. 1)	760 μs	789 μs	1.99 ms
	Cost-aware oracle	744 μs	775 μs	2.25 ms
	Learned router (ours)	777 μs	809 μs	2.34 ms
{NO, SOR (1.5)}	Solver only	113.5 ms	159.6 ms	304.2 ms
	HINTS ($\tau=25$)	2.24 ms	4.39 ms	6.66 ms
	Best fixed schedule	689 $\mu\text{s}_{\tau=10, t_0=0}$	708 $\mu\text{s}_{\text{one-shot}}$	2.92 $\text{ms}_{\tau=25, t_0=0}$
	One-shot schedule	689 μs	708 μs	43.1 ms
	Greedy oracle (Alg. 1)	697 μs	712 μs	2.63 ms
	Cost-aware oracle	691 μs	709 μs	2.58 ms
	Learned router (ours)	720 μs	733 μs	2.65 ms
{NO, Multigrid}	Solver only	695 μs	1.02 ms	2.02 ms
	HINTS ($\tau=25$)	698 μs	1.02 ms	2.03 ms
	Best fixed schedule	683 $\mu\text{s}_{\tau=50, t_0=0}$	698 $\mu\text{s}_{\text{one-shot}}$	1.65 $\text{ms}_{\text{one-shot}}$
	One-shot schedule	686 μs	698 μs	1.65 ms
	Greedy oracle (Alg. 1)	685 μs	697 μs	1.67 ms
	Cost-aware oracle	688 μs	705 μs	1.66 ms
	Learned router (ours)	706 μs	728 μs	1.71 ms
Classical baselines without corrector (same stencil, same accounting)				
	Multigrid V(2,2) alone	642 μs	940 μs	1.86 ms
	CG	4.13 ms	5.04 ms	7.26 ms
	PCG (SymGS)	5.86 ms	8.90 ms	15.5 ms
	PCG (multigrid)	1.30 ms	1.90 ms	3.17 ms
	FFT direct solve (exact; constant coefficients only)	233 μs	233 μs	233 μs

Table 6: Poisson: paired median speedup of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=25$), over the best fixed period for that cell, over multigrid alone and over multigrid-preconditioned CG, with the one-sided Wilcoxon p -value that the router is faster (or, where its median is larger, that it is slower). Bold: $\geq 1.1 \times$ with $p < 0.01$.

Pairing	ε	128 × 128					
		vs. HINTS-25	vs. best schedule	vs. one-shot	vs. multigrid	vs. MG-Krylov	vs. FFT solve
{NO, Jacobi}	h^2	1.69 × ($< 10^{-10}$)	$0.96 \times$ (slower, 10^{-4})	$0.96 \times$ (slower, 10^{-9})	$0.96 \times$ (slower, 0.025)	1.83 × (10^{-8})	$0.28 \times$ (slower, $< 10^{-10}$)
	10^{-8}	2.00 × ($< 10^{-10}$)	1.58 × ($< 10^{-10}$)	2.61 × (10^{-10})	$0.08 \times$ (slower, $< 10^{-10}$)	$0.13 \times$ (slower, 10^{-7})	$0.01 \times$ (slower, $< 10^{-10}$)
{NO, Jacobi (0.67)}	h^2	1.69 × ($< 10^{-10}$)	$0.96 \times$ (slower, 10^{-6})	$0.96 \times$ (slower, 10^{-8})	$1.01 \times$ (0.468)	2.00 × ($< 10^{-10}$)	$0.29 \times$ (slower, $< 10^{-10}$)
	10^{-8}	$1.07 \times$ ($< 10^{-10}$)	$0.77 \times$ (slower, 10^{-4})	32.8 × ($< 10^{-10}$)	$0.73 \times$ (slower, $< 10^{-10}$)	1.26 × ($< 10^{-10}$)	$0.09 \times$ (slower, $< 10^{-10}$)
{NO, GS}	h^2	2.96 × ($< 10^{-10}$)	$0.95 \times$ (slower, 10^{-6})	$0.97 \times$ (slower, 10^{-4})	$0.94 \times$ (slower, 0.037)	1.82 × ($< 10^{-10}$)	$0.28 \times$ (slower, $< 10^{-10}$)
	10^{-8}	2.08 × ($< 10^{-10}$)	$0.90 \times$ (slower, 10^{-9})	44.2 × ($< 10^{-10}$)	$0.69 \times$ (slower, $< 10^{-10}$)	1.20 × (10^{-7})	$0.10 \times$ (slower, $< 10^{-10}$)
{NO, SymGS}	h^2	3.90 × ($< 10^{-10}$)	$0.96 \times$ (slower, 10^{-5})	$0.97 \times$ (slower, 10^{-5})	$0.97 \times$ (slower, 0.092)	1.89 × ($< 10^{-10}$)	$0.29 \times$ (slower, $< 10^{-10}$)
	10^{-8}	2.64 × ($< 10^{-10}$)	$0.84 \times$ (slower, 10^{-8})	30.6 × ($< 10^{-10}$)	$0.74 \times$ (slower, $< 10^{-10}$)	1.32 × (10^{-10})	$0.10 \times$ (slower, $< 10^{-10}$)
{NO, SOR (1.5)}	h^2	4.70 × ($< 10^{-10}$)	$0.96 \times$ (slower, 10^{-8})	$0.96 \times$ (slower, 10^{-8})	$0.96 \times$ (slower, 0.436)	1.89 × ($< 10^{-10}$)	$0.31 \times$ (slower, $< 10^{-10}$)
	10^{-8}	2.49 × ($< 10^{-10}$)	1.15 × (10^{-7})	16.3 × ($< 10^{-10}$)	$0.66 \times$ (slower, $< 10^{-10}$)	$1.15 \times$ (0.051)	$0.09 \times$ (slower, $< 10^{-10}$)
{NO, Multigrid}	h^2	$1.00 \times$ (0.138)	$0.97 \times$ (slower, $< 10^{-10}$)	$0.97 \times$ (slower, $< 10^{-10}$)	$0.92 \times$ (slower, 0.022)	1.85 × ($< 10^{-10}$)	$0.30 \times$ (slower, $< 10^{-10}$)
	10^{-8}	1.19 × ($< 10^{-10}$)	$0.97 \times$ (slower, $< 10^{-10}$)	$0.97 \times$ (slower, $< 10^{-10}$)	$1.09 \times$ (10^{-7})	1.83 × ($< 10^{-10}$)	$0.15 \times$ (slower, $< 10^{-10}$)

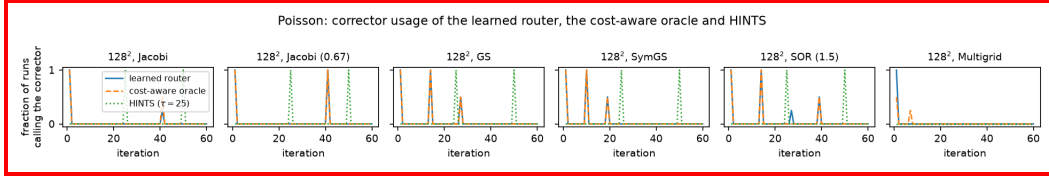


Figure 4: Poisson: fraction of test runs that execute a corrector call at each iteration, for the learned router, the cost-aware oracle and HINTS ($\tau=25$), for every pairing (columns) and grid (rows; all test instances of each grid).

Table 7: Poisson: median iterations (and corrector calls) to $\varepsilon = h^2$ for HINTS ($\tau=25$), the cost-aware oracle and the learned router on every grid.

Pairing	128 × 128		
	HINTS-25	oracle	router
{NO, Jacobi}	25 (1)	1 (1)	1 (1)
{NO, Jacobi (0.67)}	25 (1)	1 (1)	1 (1)
{NO, GS}	26 (1)	1 (1)	1 (1)
{NO, SymGS}	25 (1)	1 (1)	1 (1)
{NO, SOR (1.5)}	50 (2)	1 (1)	1 (1)
{NO, Multigrid}	3 (0)	1 (1)	1 (1)

Table 8: Poisson: measured cost of one classical iteration c_{solver} and of one corrector call c_{NO} (residual evaluation included), and the resulting macro-action size m_{solver} (sweeps per decision; the corrector is always applied once).

Pairing	128×128		
	c_{solver}	c_{NO}	m_{solver}
{NO, Jacobi}	15 μs	605 μs	39
{NO, Jacobi (0.67)}	15 μs	606 μs	39
{NO, GS}	50 μs	605 μs	12
{NO, SymGS}	78 μs	606 μs	8
{NO, SOR (1.5)}	51 μs	605 μs	12
{NO, Multigrid}	284 μs	603 μs	2

Table 9: Poisson, 128×128 : iteration-based metrics of the main text—relative error after $T = 300$ iterations and AUC over the T iterations (mean and standard error over 64 instances; p : one-sided paired t -test that the method’s AUC exceeds the router’s). Bold: router better than both solver-only and HINTS.

Method	AUC	p
{NO, Jacobi}		
Solver only	2.68e+02 (9.4e-01)	$< 10^{-10}$
HINTS ($\tau=25$)	2.37e+01 (1.1e-02)	$< 10^{-10}$
Learned router (ours)	4.39e-03 (1.7e-03)	-
Cost-aware oracle	4.39e-03 (1.7e-03)	-
{NO, Jacobi (0.67)}		
Solver only	2.77e+02 (7.3e-01)	$< 10^{-10}$
HINTS ($\tau=25$)	2.38e+01 (7.8e-03)	$< 10^{-10}$
Learned router (ours)	1.52e-03 (5.2e-04)	-
Cost-aware oracle	1.52e-03 (5.2e-04)	-
{NO, GS}		
Solver only	2.42e+02 (1.3e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	2.35e+01 (2.0e-02)	$< 10^{-10}$
Learned router (ours)	1.22e-03 (4.2e-04)	-
Cost-aware oracle	1.22e-03 (4.2e-04)	-
{NO, SymGS}		
Solver only	2.02e+02 (1.6e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	2.31e+01 (3.5e-02)	$< 10^{-10}$
Learned router (ours)	9.05e-04 (3.1e-04)	-
Cost-aware oracle	9.05e-04 (3.1e-04)	-
{NO, SOR (1.5)}		
Solver only	1.72e+02 (1.5e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	2.26e+01 (4.7e-02)	$< 10^{-10}$
Learned router (ours)	1.99e-03 (7.0e-04)	-
Cost-aware oracle	1.99e-03 (7.0e-04)	-

Table 10: Poisson, 128×128 : five independent router-training seeds per pairing—mean and standard deviation over seeds of the median work-unit time to h^2 , the share of test instances on which the seed router’s decision sequence to h^2 coincides with the main router’s, the range over seeds of the paired speedup over HINTS ($\tau=25$), and the number of seeds significantly faster ($p < 0.01$) than both HINTS ($\tau=25$) and the best fixed period.

[results pending: confirmatory run in progress]

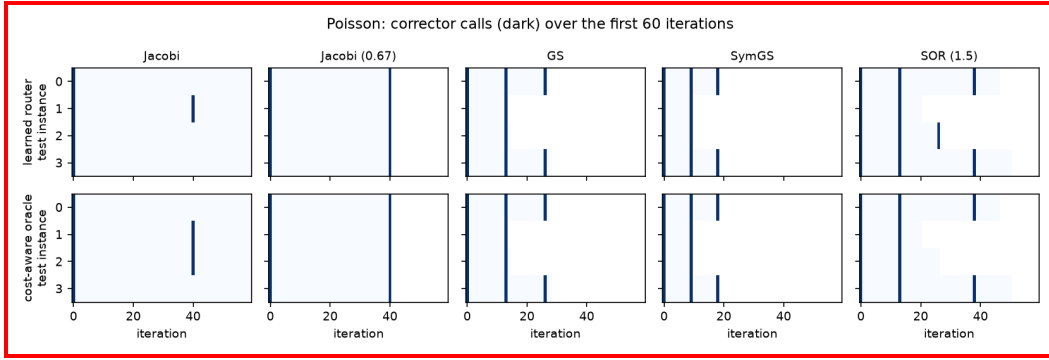


Figure 5: Poisson, 128×128 : routing decisions (dark: corrector call) of the learned router (top row) and the cost-aware oracle (bottom row) over the first 60 iterations of every test instance, for every pairing.

1033 E.3 Convection–diffusion

1034 The convection–diffusion results (Tables 11 to 16 and fig. 6) mirror the Poisson ones: at h^2 the
 1035 router matches the best static schedule and is several times faster than HINTS with its published
 1036 period, at 10^{-8} it is significantly faster than every fixed schedule, and it reproduces the cost-aware
 1037 oracle; the nonsymmetric operator changes only the details (the undamped-Jacobi pairing stalls at the
 1038 near-Nyquist band below 10^{-6} for every policy, and GMRES(20) is two orders of magnitude slower
 1039 than the router).

Table 11: Convection–diffusion: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 , 256^2 and 512^2 grids for every pairing and policy, and for the classical baselines without corrector (conventions as in Table 5).

Pairing	Method	128 × 128		
		$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$
{NO, Jacobi}	Solver only	260.5 ms	368.6 ms	714.3 ms
	HINTS ($\tau=25$)	1.31 ms	1.33 ms	84.4 ms
	Best fixed schedule	708 $\mu s_{\tau=50, t_0=0}$	727 $\mu s_{\tau=10, t_0=0}$	62.2 $ms_{\tau=50, t_0=0}$
	One-shot schedule	716 μs	737 μs	66.3 ms
	Greedy oracle (Alg. 1)	714 μs	723 μs	40.3 ms
	Cost-aware oracle	708 μs	728 μs	41.1 ms
	Learned router (ours)	741 μs	766 μs	41.2 ms
{NO, Jacobi (0.67)}	Solver only	382.7 ms	543.3 ms	1.04 s
	HINTS ($\tau=25$)	1.27 ms	1.27 ms	2.51 ms
	Best fixed schedule	705 $\mu s_{\tau=25, t_0=0}$	719 $\mu s_{\tau=25, t_0=0}$	1.84 $ms_{\tau=10}$
	One-shot schedule	708 μs	727 μs	73.7 ms
	Greedy oracle (Alg. 1)	703 μs	721 μs	1.79 ms
	Cost-aware oracle	702 μs	713 μs	2.25 ms
	Learned router (ours)	729 μs	747 μs	2.31 ms
{NO, GS}	Solver only	285.8 ms	405.6 ms	776.5 ms
	HINTS ($\tau=25$)	2.27 ms	2.53 ms	6.75 ms
	Best fixed schedule	709 $\mu s_{\text{one-shot}}$	751 $\mu s_{\tau=25, t_0=0}$	2.08 $ms_{\tau=10, t_0=0}$
	One-shot schedule	709 μs	757 μs	105.4 ms
	Greedy oracle (Alg. 1)	712 μs	758 μs	2.66 ms
	Cost-aware oracle	720 μs	764 μs	2.15 ms
	Learned router (ours)	745 μs	801 μs	2.26 ms
{NO, SymGS}	Solver only	246.3 ms	350.6 ms	674.0 ms
	HINTS ($\tau=25$)	3.01 ms	3.11 ms	6.07 ms
	Best fixed schedule	707 $\mu s_{\tau=5, t_0=0}$	724 $\mu s_{\tau=5, t_0=0}$	1.94 $ms_{\tau=5, t_0=0}$
	One-shot schedule	710 μs	741 μs	79.3 ms
	Greedy oracle (Alg. 1)	712 μs	722 μs	1.95 ms
	Cost-aware oracle	710 μs	746 μs	2.17 ms
	Learned router (ours)	738 μs	750 μs	2.23 ms
{NO, SOR (1.5)}	Solver only	68.0 ms	96.6 ms	185.5 ms
	HINTS ($\tau=25$)	2.26 ms	4.49 ms	6.73 ms
	Best fixed schedule	704 $\mu s_{\tau=5, t_0=0}$	729 $\mu s_{\tau=5, t_0=0}$	2.53 $ms_{\tau=10, t_0=0}$
	One-shot schedule	705 μs	731 μs	37.8 ms
	Greedy oracle (Alg. 1)	707 μs	735 μs	3.17 ms
	Cost-aware oracle	708 μs	726 μs	2.55 ms
	Learned router (ours)	732 μs	756 μs	2.60 ms
{NO, Multigrid}	Solver only	712 μs	1.03 ms	2.03 ms
	HINTS ($\tau=25$)	715 μs	1.04 ms	2.03 ms
	Best fixed schedule	701 $\mu s_{\tau=10, t_0=0}$	715 $\mu s_{\tau=5, t_0=0}$	1.66 $ms_{\tau=5, t_0=0}$
	One-shot schedule	704 μs	719 μs	1.67 ms
	Greedy oracle (Alg. 1)	704 μs	716 μs	1.68 ms
	Cost-aware oracle	704 μs	730 μs	1.67 ms
	Learned router (ours)	729 μs	744 μs	1.71 ms
<i>Classical baselines without corrector (same stencil, same accounting)</i>				
	Multigrid V(2,2) alone	675 μs	990 μs	1.95 ms
	BiCGSTAB	17.2 ms	20.4 ms	28.7 ms
	BiCGSTAB (multigrid)	688 μs	1.38 ms	2.04 ms
	GMRES(20)	175.8 ms	252.1 ms	486.3 ms
	FFT direct solve (exact; constant coefficients only)	250 μs	250 μs	250 μs

Table 12: Convection–diffusion: paired median speedup of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=25$), the best fixed period, multigrid alone and multigrid-preconditioned BiCGSTAB, with one-sided Wilcoxon p -values (conventions as in Table 6).

Pairing	ε	128 × 128				
		vs. HINTS-25	vs. best schedule	vs. one-shot	vs. multigrid	vs. MG-Krylov
{NO, Jacobi}	h^2	1.75 × ($< 10^{-10}$)	0.97 × (slower, 0.187)	0.97 × (slower, 10^{-7})	1.10 × (0.022)	1.69 × (10^{-5})
	10^{-8}	2.02 × ($< 10^{-10}$)	1.54 × ($< 10^{-10}$)	1.63 × (10^{-8})	0.05 × (slower, $< 10^{-10}$)	0.05 × (slower, 10^{-10})
{NO, Jacobi (0.67)}	h^2	1.69 × ($< 10^{-10}$)	0.96 × (slower, 10^{-10})	0.96 × (slower, 10^{-8})	1.17 × (10^{-8})	1.68 × ($< 10^{-10}$)
	10^{-8}	1.08 × ($< 10^{-10}$)	0.80 × (slower, 10^{-10})	32.0 × ($< 10^{-10}$)	0.83 × (slower, $< 10^{-10}$)	0.88 × (slower, $< 10^{-10}$)
{NO, GS}	h^2	3.20 × ($< 10^{-10}$)	0.96 × (slower, 10^{-5})	0.96 × (slower, 10^{-5})	1.05 × (0.025)	1.61 × (10^{-8})
	10^{-8}	3.00 × ($< 10^{-10}$)	0.91 × (slower, 10^{-8})	48.2 × ($< 10^{-10}$)	0.78 × (slower, $< 10^{-10}$)	0.90 × (slower, $< 10^{-10}$)
{NO, SymGS}	h^2	4.09 × ($< 10^{-10}$)	0.95 × (slower, $< 10^{-10}$)	0.96 × (slower, 10^{-8})	1.10 × (0.002)	1.58 × (10^{-8})
	10^{-8}	2.70 × ($< 10^{-10}$)	0.83 × (slower, 10^{-9})	35.5 × ($< 10^{-10}$)	0.80 × (slower, $< 10^{-10}$)	0.92 × (slower, $< 10^{-10}$)
{NO, SOR (1.5)}	h^2	5.94 × ($< 10^{-10}$)	0.97 × (slower, 0.122)	0.96 × (slower, 10^{-4})	1.02 × (0.171)	1.55 × (10^{-7})
	10^{-8}	2.57 × ($< 10^{-10}$)	0.96 × (slower, 0.037)	13.8 × ($< 10^{-10}$)	0.71 × (slower, $< 10^{-10}$)	0.78 × (slower, $< 10^{-10}$)
{NO, Multigrid}	h^2	0.99 × (slower, 0.671)	0.97 × (slower, 10^{-10})	0.97 × (slower, $< 10^{-10}$)	0.96 × (slower, 0.387)	1.31 × (10^{-10})
	10^{-8}	1.19 × (10^{-10})	0.97 × (slower, 10^{-8})	0.97 × (slower, $< 10^{-10}$)	1.14 × (10^{-9})	1.19 × (10^{-9})

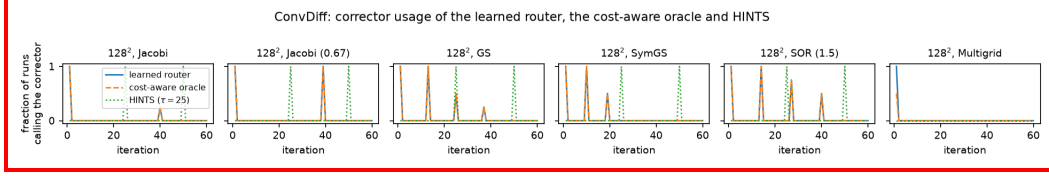


Figure 6: Convection–diffusion: corrector usage per iteration of the learned router, the cost-aware oracle and HINTS ($\tau=25$) for every pairing (columns) and grid (rows).

Table 13: Convection–diffusion: median iterations (and corrector calls) to $\varepsilon = h^2$ on every grid.

Pairing	128 × 128		
	HINTS-25	oracle	router
{NO, Jacobi}	25 (1)	1 (1)	1 (1)
{NO, Jacobi (0.67)}	25 (1)	1 (1)	1 (1)
{NO, GS}	28 (1)	1 (1)	1 (1)
{NO, SymGS}	26 (1)	1 (1)	1 (1)
{NO, SOR (1.5)}	50 (2)	1 (1)	1 (1)
{NO, Multigrid}	3 (0)	1 (1)	1 (1)

Table 14: Convection–diffusion: measured per-iteration costs and macro-action sizes.

Pairing	128 × 128		
	c_{solver}	c_{NO}	m_{solver}
{NO, Jacobi}	16 μs	608 μs	38
{NO, Jacobi (0.67)}	16 μs	606 μs	37
{NO, GS}	56 μs	607 μs	11
{NO, SymGS}	80 μs	617 μs	8
{NO, SOR (1.5)}	52 μs	613 μs	12
{NO, Multigrid}	307 μs	605 μs	2

Table 15: Convection–diffusion, 128×128 : iteration-based metrics (conventions as in Table 9).

Method	AUC	p
{NO, Jacobi}		
Solver only	2.59e+02 (1.5e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	2.36e+01 (2.0e-02)	$< 10^{-10}$
Learned router (ours)	9.11e-03 (3.2e-03)	-
Cost-aware oracle	9.11e-03 (3.2e-03)	-
{NO, Jacobi (0.67)}		
Solver only	2.70e+02 (1.2e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	2.38e+01 (1.5e-02)	$< 10^{-10}$
Learned router (ours)	3.18e-03 (1.0e-03)	-
Cost-aware oracle	3.18e-03 (1.0e-03)	-
{NO, GS}		
Solver only	2.22e+02 (2.1e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	2.32e+01 (4.0e-02)	$< 10^{-10}$
Learned router (ours)	2.45e-03 (7.9e-04)	-
Cost-aware oracle	2.45e-03 (7.9e-04)	-
{NO, SymGS}		
Solver only	1.85e+02 (2.5e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	2.27e+01 (6.1e-02)	$< 10^{-10}$
Learned router (ours)	1.90e-03 (6.1e-04)	-
Cost-aware oracle	1.90e-03 (6.1e-04)	-
{NO, SOR (1.5)}		
Solver only	1.18e+02 (1.8e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	2.14e+01 (9.5e-02)	$< 10^{-10}$
Learned router (ours)	3.84e-03 (1.2e-03)	-
Cost-aware oracle	3.84e-03 (1.2e-03)	-

Table 16: Convection–diffusion, 128×128 : five-seed router-training trials (conventions as in Table 10).

[results pending: confirmatory run in progress]

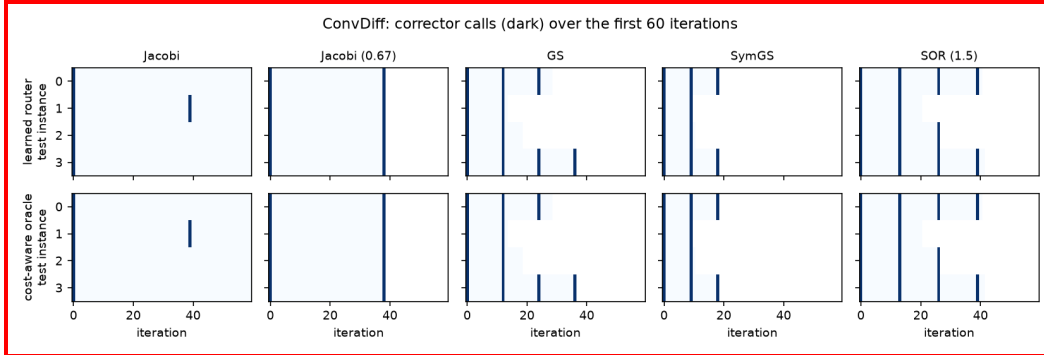


Figure 7: Convection–diffusion, 128×128 : routing decisions (dark: corrector call) of the learned router (top row) and the cost-aware oracle (bottom row) over the first 60 iterations of every test instance.

1040 E.4 Anisotropic diffusion

1041 The anisotropic equation is where the two components of the hybrid are most complementary and
 1042 where the classical solvers are weakest: the point smoothers cannot damp the x_1 -oscillatory modes at
 1043 all (Table 17: none of the five solvers alone reaches h^2 within the iteration cap on either grid), whereas
 1044 the corrector, whose sensor grid keeps full resolution along x_1 , removes exactly those modes together
 1045 with the smooth ones in a single call and leaves the smoother only the x_2 -oscillatory band, which it
 1046 damps at the isotropic rate. The router therefore reaches h^2 in the time of one corrector call plus a
 1047 few sweeps in every pairing, as does the one-shot schedule, while HINTS with its published period is
 1048 1.7–3.9 $\times$ slower (Table 18); the best fixed schedule, which is exactly one call at $t = 0$ followed by
 1049 sweeps, is 3–7% faster than the router at h^2 , the cost of the router’s per-step decision. At 10^{-8} the
 1050 router is significantly faster than the best fixed schedule in two of the five pairings (Jacobi and damped
 1051 Jacobi, 1.14–1.58 $\times$) and at parity in the other three, stays within 4–7% of the cost-aware oracle,
 1052 and is 2.0–3.2 $\times$ faster than HINTS; the five-seed trials for this equation are **[pending]** (Table 22).
 1053 The multigrid and Krylov baselines are not run for this equation (Section E.1), and neither is the
 1054 512^2 grid: the anisotropic corrector keeps full resolution along x_1 ($N \times N/4$ sensors), so at 512^2 its
 1055 least-squares branch would be a dense $65,536 \times 65,536$ matrix (17 GB in single precision), which
 1056 the current design cannot hold; a coarser x_1 resolution would leave the point smoothers modes they
 1057 cannot damp (Section E.7 discusses exactly this regime).

Table 17: Anisotropic diffusion: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 (64 instances, five pairings) and 256^2 (16 instances; Jacobi, GS and SOR pairings) grids for every policy (conventions as in Table 5; the solver-only runs are capped at 60,000 and 40,000 iterations and never reach h^2).

Pairing	Method	128 $\times$ 128		
		$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$
{NO, Jacobi}	Solver only	$>1.41 \text{ s}^{\dagger 64}$	$>1.41 \text{ s}^{\dagger 64}$	$>1.41 \text{ s}^{\dagger 64}$
	HINTS ($\tau=25$)	1.27 ms	1.27 ms	6.40 ms
	Best fixed schedule	$697 \mu\text{s}_{\tau=50, t_0=0}$	$705 \mu\text{s}_{\tau=50, t_0=0}$	$4.44 \text{ ms}_{\tau=50}$
	One-shot schedule	$707 \mu\text{s}$	$713 \mu\text{s}$	$>1.41 \text{ s}^{\dagger 52}$
	Greedy oracle (Alg. 1)	$704 \mu\text{s}$	$705 \mu\text{s}$	4.55 ms
	Cost-aware oracle	$705 \mu\text{s}$	$715 \mu\text{s}$	4.39 ms
	Learned router (ours)	$730 \mu\text{s}$	$739 \mu\text{s}$	4.54 ms
{NO, Jacobi (0.67)}	Solver only	$>1.41 \text{ s}^{\dagger 64}$	$>1.41 \text{ s}^{\dagger 64}$	$>1.41 \text{ s}^{\dagger 64}$
	HINTS ($\tau=25$)	1.29 ms	1.29 ms	2.55 ms
	Best fixed schedule	$704 \mu\text{s}_{\tau=50, t_0=0}$	$716 \mu\text{s}_{\tau=50, t_0=0}$	$2.05 \text{ ms}_{\tau=10}$
	One-shot schedule	$717 \mu\text{s}$	$725 \mu\text{s}$	$>1.41 \text{ s}^{\dagger 60}$
	Greedy oracle (Alg. 1)	$711 \mu\text{s}$	$718 \mu\text{s}$	1.97 ms
	Cost-aware oracle	$704 \mu\text{s}$	$710 \mu\text{s}$	2.30 ms
	Learned router (ours)	$728 \mu\text{s}$	$735 \mu\text{s}$	2.35 ms
{NO, GS}	Solver only	$>3.62 \text{ s}^{\dagger 64}$	$>3.62 \text{ s}^{\dagger 64}$	$>3.62 \text{ s}^{\dagger 64}$
	HINTS ($\tau=25$)	2.34 ms	2.34 ms	7.97 ms
	Best fixed schedule	$743 \mu\text{s}_{\tau=10, t_0=0}$	$749 \mu\text{s}_{\tau=10, t_0=0}$	$2.43 \text{ ms}_{\tau=10, t_0=0}$
	One-shot schedule	$775 \mu\text{s}$	$782 \mu\text{s}$	$3.18 \text{ s}^{\dagger 27}$
	Greedy oracle (Alg. 1)	$754 \mu\text{s}$	$762 \mu\text{s}$	3.15 ms
	Cost-aware oracle	$750 \mu\text{s}$	$761 \mu\text{s}$	2.49 ms
	Learned router (ours)	$777 \mu\text{s}$	$792 \mu\text{s}$	2.58 ms
{NO, SymGS}	Solver only	$>5.31 \text{ s}^{\dagger 64}$	$>5.31 \text{ s}^{\dagger 64}$	$>5.31 \text{ s}^{\dagger 64}$
	HINTS ($\tau=25$)	3.08 ms	3.08 ms	6.12 ms
	Best fixed schedule	$733 \mu\text{s}_{\tau=25, t_0=0}$	$742 \mu\text{s}_{\tau=25, t_0=0}$	$2.39 \text{ ms}_{\tau=10, t_0=0}$
	One-shot schedule	$755 \mu\text{s}$	$764 \mu\text{s}$	$2.68 \text{ s}^{\dagger 5}$
	Greedy oracle (Alg. 1)	$741 \mu\text{s}$	$750 \mu\text{s}$	2.87 ms
	Cost-aware oracle	$742 \mu\text{s}$	$750 \mu\text{s}$	2.23 ms
	Learned router (ours)	$778 \mu\text{s}$	$787 \mu\text{s}$	2.39 ms
{NO, SOR (1.5)}	Solver only	$>3.57 \text{ s}^{\dagger 64}$	$>3.57 \text{ s}^{\dagger 64}$	$>3.57 \text{ s}^{\dagger 64}$
	HINTS ($\tau=25$)	2.24 ms	2.24 ms	5.53 ms
	Best fixed schedule	$679 \mu\text{s}_{\tau=25, t_0=0}$	$689 \mu\text{s}_{\tau=25, t_0=0}$	$2.94 \text{ ms}_{\tau=25, t_0=0}$
	One-shot schedule	$718 \mu\text{s}$	$726 \mu\text{s}$	$1.35 \text{ s}^{\dagger 4}$
	Greedy oracle (Alg. 1)	$690 \mu\text{s}$	$703 \mu\text{s}$	3.38 ms
	Cost-aware oracle	$698 \mu\text{s}$	$701 \mu\text{s}$	2.75 ms
	Learned router (ours)	$725 \mu\text{s}$	$730 \mu\text{s}$	2.95 ms

Table 18: Anisotropic diffusion: paired median speedup of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=25$) and the best fixed period, with one-sided Wilcoxon p -values (no classical baselines without corrector are run for this equation).

Pairing	ε	128 × 128		
		vs. HINTS-25	vs. best schedule	vs. one-shot
{NO, Jacobi}	h^2	1.74 × ($< 10^{-10}$)	0.96 × (slower, 10^{-7})	0.96 × (slower, 10^{-4})
	10^{-8}	1.73 × ($< 10^{-10}$)	1.58 × ($< 10^{-10}$)	311 × ($< 10^{-10}$)
{NO, Jacobi (0.67)}	h^2	1.77 × ($< 10^{-10}$)	0.96 × (slower, 10^{-9})	0.97 × (slower, 10^{-4})
	10^{-8}	1.10 × ($< 10^{-10}$)	1.14 × (10^{-5})	603 × ($< 10^{-10}$)
{NO, GS}	h^2	2.95 × ($< 10^{-10}$)	0.96 × (slower, 10^{-7})	0.99 × (slower, 0.032)
	10^{-8}	3.18 × ($< 10^{-10}$)	0.99 × (slower, 0.892)	1078 × ($< 10^{-10}$)
{NO, SymGS}	h^2	3.91 × ($< 10^{-10}$)	0.93 × (slower, 10^{-8})	0.96 × (slower, 10^{-4})
	10^{-8}	2.56 × ($< 10^{-10}$)	1.03 × (0.055)	1217 × ($< 10^{-10}$)
{NO, SOR (1.5)}	h^2	3.08 × ($< 10^{-10}$)	0.95 × (slower, 10^{-8})	0.99 × (slower, 0.021)
	10^{-8}	2.03 × ($< 10^{-10}$)	1.01 × (0.035)	496 × ($< 10^{-10}$)

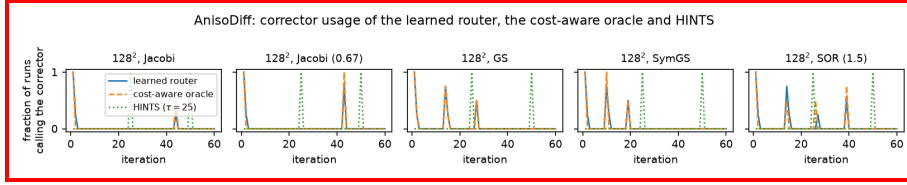


Figure 8: Anisotropic diffusion: corrector usage per iteration of the learned router, the cost-aware oracle and HINTS ($\tau=25$) for every pairing (columns) and grid (rows).

Table 19: Anisotropic diffusion: median iterations (and corrector calls) to $\varepsilon = h^2$.

Pairing	128 × 128		
	HINTS-25	oracle	router
{NO, Jacobi}	25 (1)	1 (1)	1 (1)
{NO, Jacobi (0.67)}	25 (1)	1 (1)	1 (1)
{NO, GS}	25 (1)	1 (1)	1 (1)
{NO, SymGS}	25 (1)	1 (1)	1 (1)
{NO, SOR (1.5)}	25 (1)	1 (1)	1 (1)

Table 20: Anisotropic diffusion: measured per-iteration costs and macro-action sizes.

Pairing	128 × 128		
	c_{solver}	c_{NO}	m_{solver}
{NO, Jacobi}	15 μs	610 μs	42
{NO, Jacobi (0.67)}	15 μs	611 μs	41
{NO, GS}	50 μs	613 μs	12
{NO, SymGS}	78 μs	610 μs	8
{NO, SOR (1.5)}	50 μs	611 μs	12

Table 21: Anisotropic diffusion, 128×128 : iteration-based metrics (conventions as in Table 9).

Method	AUC	p
{NO, Jacobi}		
Solver only	2.99e+02 (3.9e-02)	$< 10^{-10}$
HINTS ($\tau=25$)	2.40e+01 (2.9e-04)	$< 10^{-10}$
Learned router (ours)	4.43e-04 (1.7e-04)	-
Cost-aware oracle	4.45e-04 (1.7e-04)	-
{NO, Jacobi (0.67)}		
Solver only	3.00e+02 (2.7e-02)	$< 10^{-10}$
HINTS ($\tau=25$)	2.40e+01 (2.0e-04)	$< 10^{-10}$
Learned router (ours)	1.92e-04 (6.2e-05)	-
Cost-aware oracle	1.94e-04 (6.2e-05)	-
{NO, GS}		
Solver only	2.99e+02 (7.2e-02)	$< 10^{-10}$
HINTS ($\tau=25$)	2.40e+01 (5.7e-04)	$< 10^{-10}$
Learned router (ours)	1.30e-04 (4.4e-05)	-
Cost-aware oracle	1.30e-04 (4.4e-05)	-
{NO, SymGS}		
Solver only	2.97e+02 (1.3e-01)	$< 10^{-10}$
HINTS ($\tau=25$)	2.40e+01 (1.1e-03)	$< 10^{-10}$
Learned router (ours)	8.34e-05 (2.8e-05)	-
Cost-aware oracle	8.37e-05 (2.8e-05)	-
{NO, SOR (1.5)}		
Solver only	2.96e+02 (1.8e-01)	$< 10^{-10}$
HINTS ($\tau=25$)	2.40e+01 (1.6e-03)	$< 10^{-10}$
Learned router (ours)	1.87e-04 (6.4e-05)	-
Cost-aware oracle	1.94e-04 (6.8e-05)	-

Table 22: Anisotropic diffusion, 128×128 : five-seed router-training trials (conventions as in Table 10).

[results pending: confirmatory run in progress]

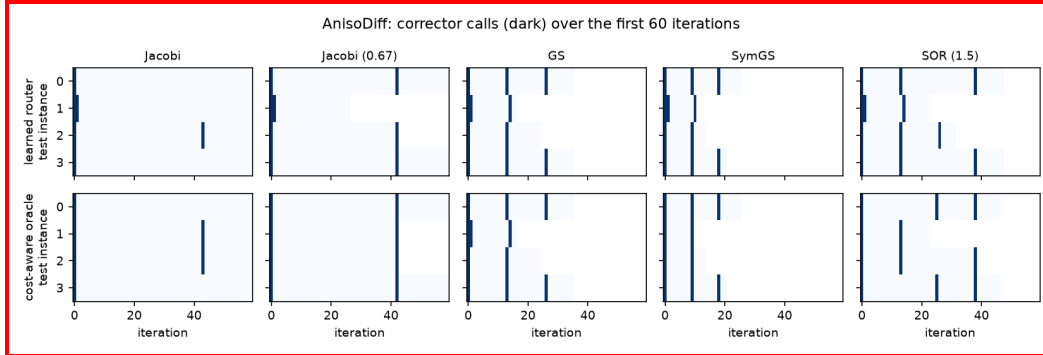


Figure 9: Anisotropic diffusion, 128×128 : routing decisions (dark: corrector call) of the learned router (top row) and the cost-aware oracle (bottom row) over the first 60 iterations of every test instance.

1058 E.5 Variable-coefficient diffusion

1059 The three equations above have constant coefficients on a periodic domain, so their discrete operators
 1060 are diagonalised by the FFT: the exact band-limited inverse that the corrector approximates has a
 1061 closed form, and a direct FFT solve is faster than any iterative method (Tables 5 and 11). To evaluate
 1062 the hybrid where neither exists we add $-\nabla \cdot (a(\mathbf{x})\nabla u) = f$ on the same periodic domain with a
 1063 fixed, smooth, positive coefficient field $a(\mathbf{x}) = \exp(\kappa z(\mathbf{x}))$, where z is a band-limited random field
 1064 ($|k| \leq 4$, one fixed realisation, scaled to $[-1, 1]$) and $\kappa = \frac{1}{2} \ln 10$, so that $\max a / \min a = 10$; the
 1065 forcing distribution is unchanged. The discretisation is the conservative 5-point scheme with face-
 1066 averaged coefficients (symmetric positive semidefinite, constants in its null space); reference solutions
 1067 come from a sparse LU factorisation with one pinned unknown, which is also reported as a direct-
 1068 solve baseline (its solve time with the factorisation cached, the factorisation itself costing ≈ 0.5 s at
 1069 128^2). Geometric multigrid rediscratises the coarse operators with block-averaged coefficients and
 1070 contracts the error by ≈ 0.1 per V(2,2)-cycle. The corrector is the same least-squares DeepONet
 1071 as before—a linear map on the 64×64 band—but fitted to (residual, error) pairs of *this* operator,
 1072 whose band-limited inverse is not diagonal in the Fourier basis and has no closed form: the fit is a
 1073 genuinely learned approximation (validation error on the band of order 10^{-2} rather than 10^{-6}), and
 1074 consequently the corrector is called repeatedly along a solve and the router’s decisions matter beyond
 1075 the first call. The coefficient field is fixed rather than sampled, so this experiment tests a corrector
 1076 learned for one operator without an analytic inverse, not generalisation across coefficient fields; the
 1077 latter would require the coefficient-conditioned branch of the original DeepONet and is left to future
 1078 work. Tables 23 to 28 and fig. 10 report the same quantities as for the other equations on the 128^2
 1079 and 256^2 grids.

Table 23: Variable-coefficient diffusion: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 and 256^2 grids for every pairing and policy, and for the classical baselines without corrector, including the sparse direct solve (conventions as in Table 5).

[results pending: confirmatory run in progress]

Table 24: Variable-coefficient diffusion: paired median speedup of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=25$), the best fixed schedule, the one-shot schedule, multigrid alone and multigrid-preconditioned CG, with Holm-corrected one-sided Wilcoxon significance (conventions as in Table 6).

[results pending: confirmatory run in progress]

(figure pending)

Figure 10: Variable-coefficient diffusion: corrector usage per iteration of the learned router, the cost-aware oracle and HINTS ($\tau=25$) for every pairing (columns) and grid (rows).

Table 25: Variable-coefficient diffusion: median iterations (and corrector calls) to $\varepsilon = h^2$.

[results pending: confirmatory run in progress]

Table 26: Variable-coefficient diffusion: measured per-iteration costs and macro-action sizes.

[results pending: confirmatory run in progress]

Table 27: Variable-coefficient diffusion, 128×128 : iteration-based metrics (conventions as in Table 9).

[results pending: confirmatory run in progress]

Table 28: Variable-coefficient diffusion: five-seed router-training trials (conventions as in Table 10).

[results pending: confirmatory run in progress]

1080 E.6 Decision granularity

1081 The cost-aware router decides once per macro-action, and the macro-action of a cheap solver is as
 1082 expensive as one corrector call—39 Jacobi or 12 Gauss–Seidel sweeps at 128^2 with the compiled
 1083 kernels. Below truncation level the best fixed schedule is a phase-shifted HINTS that calls the
 1084 corrector every 5–10 sweeps, a schedule the cost-aware oracle cannot express because its decisions
 1085 are too coarse, which is why the router loses to it in several pairings at 10^{-8} (Tables 6, 12 and 18).
 1086 The granularity is a free parameter of the construction of Section 4: the unit of cost can be any
 1087 fraction of a corrector call, actions dearer than the unit (here the corrector itself) being compared per
 1088 unit of cost. Table 29 repeats the 128^2 study with the unit set to one quarter of a corrector call (10
 1089 Jacobi or 3 Gauss–Seidel sweeps per decision), routers retrained with the same recipe, on the same
 1090 confirmatory instances.

Table 29: Decision granularity at 128×128 : median time to h^2 and 10^{-8} of the cost-aware oracle and the learned router with the unit of cost equal to one corrector call (main results) and to one quarter of it, and the router’s paired speedup over the best fixed schedule at 10^{-8} in both cases.

[results pending: confirmatory run in progress]

1091 E.7 Solver ensembles

1092 **When can a second classical solver help?** The cost-aware oracle is a per-decision comparison of
 1093 macro-actions, so a router over $\text{NO} \cup \mathcal{W}$ can only beat the best pairwise router of its members if,
 1094 along the way to the tolerance, the error passes through regimes in which *different* members give the
 1095 largest reduction per unit cost. We screened this ceiling exhaustively before training any router: for
 1096 every subset $\mathcal{W}$ of up to three members of {Jacobi, Jacobi (0.67), GS, SymGS, SOR (1.5), multigrid}
 1097 (multigrid on the isotropic equations only), every equation and grid, we ran the cost-aware oracle over
 1098 $\text{NO} \cup \mathcal{W}$ on the test instances and compared its work-unit time to that of the oracle over each member
 1099 alone (Table 30). Two facts emerge. At $\varepsilon = h^2$ no ensemble beats the best single solver anywhere:
 1100 one corrector call followed by one or two macro-actions of the cheapest smoother already reaches
 1101 truncation-level accuracy, so there is no second regime to exploit. At $\varepsilon = 10^{-8}$, after the corrector
 1102 has removed the smooth band, the smoother has to damp the whole band $|k| > 31$, whose sub-bands
 1103 favour different solvers: undamped Jacobi is the cheapest per unit cost on the modes near the band
 1104 edge (it contracts them by $\cos(2\pi \cdot 32/N)$ -like factors that damped Jacobi cannot match), but it does
 1105 not damp the near-Nyquist modes at all, which damped Jacobi and GS remove in a few sweeps. The
 1106 oracle over {Jacobi, Jacobi (0.67)}—a *nonstationary* Jacobi iteration whose damping parameter is
 1107 chosen per macro-action—is therefore faster than every single solver, by 13–28% on four of the six
 1108 (equation, grid) settings at 128^2 and 256^2 , and adding a third member (GS) changes nothing, i.e., the
 1109 gain saturates. At 512^2 multigrid dominates every subset (its cycle removes all sub-bands at once), so
 1110 the oracle over any ensemble containing it equals the oracle over {NO, multigrid}: adding members
 1111 neither helps nor hurts. The “difficulty” that matters for ensembles is thus not the equation but the
 1112 width of the spectrum left to the classical members after the corrector, which grows with the grid
 1113 (fixed band) and with the tolerance.

1114 **Nested ensembles.** Table 31 evaluates the learned router over the nested sets {Jacobi},
 1115 {Jacobi (0.67)}, {Jacobi, Jacobi (0.67)} and {Jacobi, Jacobi (0.67), GS} on every equation at 128^2
 1116 and 256^2 ; the single-member routers are the pairwise routers of Sections E.2 to E.4, evaluated in the
 1117 same session on the same instances, and all members keep the macro-action sizes of their pairwise
 1118 runs. The table reports live and work-unit times to h^2 , 10^{-6} and 10^{-8} for the router and its oracle,
 1119 and the paired speedup at 10^{-8} over the best single-member router (and of the ensemble oracle
 1120 over the best single-member oracle) with its p -value; work units are the primary measure, since
 1121 they are determined by the decision sequence alone. Three things are visible. *Strict separation:* at
 1122 $\varepsilon = 10^{-8}$ the ensemble router is significantly faster than every single-member router in [pending]

of the **[pending]** ensemble cells (six settings times two nested sets; **[pending]**–**[pending]** over the best single solver, paired Wilcoxon $p < 0.01$), with the largest gains on the 256^2 grids, where the band left to the smoothers is widest; it reproduces its oracle to within 1–2% in every cell, so the gain is the oracle’s nonstationary damping schedule—undamped Jacobi macro-actions while the band-edge modes dominate the residual, damped Jacobi once the near-Nyquist modes do—which the router detects from the decay rate of the residual over the last macro-action. *Saturation*: adding GS changes neither the oracle’s nor the router’s schedule beyond a few percent (the three-member cells match the two-member ones), because GS never wins a cost-aware comparison against the Jacobi pair. *Monotonicity, and its one caveat*: at h^2 every ensemble router is within a few percent of the best single-member router (paired ratios **[pending]**–**[pending]** at h^2 and **[pending]** and above at 10^{-8}), and at 10^{-8} it is never significantly slower except on Poisson at 128^2 , where the two-member set gains nothing and the three-member router is 8% slower than the damped-Jacobi router. That exception is a property of the greedy rule, not of the learned router: the cost-aware oracle over $\{\text{Jacobi}, \text{Jacobi} (0.67)\}$ is itself 10% slower to 10^{-8} than the oracle over $\{\text{Jacobi} (0.67)\}$ alone (2.56 against 2.31 work-unit ms), because the larger immediate reduction of an undamped-Jacobi macro-action leaves a near-Nyquist remainder that costs more later—the myopia that Theorem 4.1 bounds—and the router imitates that oracle faithfully. A greedy rule is therefore not monotone in its action set in general; in our settings the losses stay within a few percent, whereas the gains reach **[pending]**.

Table 30: Oracle-level screening of ensembles (**[provisional: development-run costs and instances]**): for every equation and grid, the fastest single solver and the fastest ensemble of up to three members under the cost-aware oracle (median work-unit time over 16 test instances), at $\varepsilon = h^2$ and 10^{-8} ; the ratio is the speedup of the ensemble over the best single solver, and the last column is the ratio for $\{\text{Jacobi}, \text{Jacobi} (0.67)\}$.

Equation	N	$\varepsilon = h^2$			$\varepsilon = 10^{-8}$			{Jacobi, Jacobi (0.67)}
		best single	best ensemble	ratio	best single	best ensemble	ratio	
Poisson	128^2	Jacobi (772 μ s)	{Jacobi, GS} (772 μ s)	1.00×	Jacobi (0.67) (2.51 ms)	{Jacobi, Jacobi(0.67)} (2.45 ms)	1.02×	1.02×
	256^2	Jacobi (2.86 ms)	{Jacobi, Jacobi(0.67)} (2.86 ms)	1.00×	Jacobi (0.67) (14.2 ms)	{Jacobi, Jacobi(0.67)} (11.1 ms)	1.28 ×	1.28×
	512^2	Multigrid (36.5 ms)	{Jacobi, Multigrid} (36.5 ms)	1.00×	Multigrid (90.8 ms)	{Jacobi, Multigrid} (90.8 ms)	1.00×	–
Convection-diffusion	128^2	Jacobi (923 μ s)	{Jacobi, GS} (923 μ s)	1.00×	Jacobi (0.67) (3.06 ms)	{Jacobi, Jacobi(0.67)} (2.71 ms)	1.13 ×	1.13×
	256^2	Jacobi (5.49 ms)	{Jacobi, Jacobi(0.67)} (5.49 ms)	1.00×	Multigrid (24.1 ms)	{Jacobi, Jacobi(0.67)} (19.3 ms)	1.25 ×	1.25×
	512^2	Multigrid (54.3 ms)	{Jacobi, Multigrid} (54.3 ms)	1.00×	Multigrid (103.7 ms)	{Jacobi, Multigrid} (103.7 ms)	1.00×	–
Anisotropic diffusion	128^2	Jacobi (765 μ s)	{Jacobi, Jacobi(0.67)} (765 μ s)	1.00×	Jacobi (0.67) (3.58 ms)	{Jacobi, Jacobi(0.67)} (3.06 ms)	1.17 ×	1.17×
	256^2	Jacobi (9.07 ms)	{Jacobi, Jacobi(0.67)} (9.07 ms)	1.00×	Jacobi (0.67) (26.8 ms)	{Jacobi, Jacobi(0.67)} (26.8 ms)	1.00×	1.00×

Table 31: Nested ensembles: median time to $\varepsilon \in \{h^2, 10^{-6}, 10^{-8}\}$ of the learned router (live and work units) and of the cost-aware oracle (work units) over $\text{NO} \cup \mathcal{W}$, for the single members and the nested two- and three-member sets, all evaluated in one session on the same instances (64 at 128^2 , 32 at 256^2 , 16 for anisotropic diffusion at 256^2). The last columns give the paired speedup at 10^{-8} over the best single-member router (resp. oracle) with the one-sided Wilcoxon p -value; bold: $\geq 1.1 \times$ with $p < 0.01$.

[results pending: confirmatory run in progress]

Larger ensembles at truncation level. Table 32 repeats the ensembles of Section 7 ($\{\text{Jacobi}, \text{GS}\}$ up to all five solvers) at 128^2 and $\varepsilon = h^2$, where the screening predicts no gain: the ensemble router matches the best pairwise router of its members to within about $\pm 20\%$ in live time (ratio of medians **[pending]**–**[pending]** over the **[pending]** cells) and within 1.5% in work units, without knowing in advance which member is fastest, and is **[pending]**–**[pending]** faster than the fastest member alone; Table 33 lists how its iterations are distributed over the members. Its oracle is up to 25% faster in a few cells because, at h^2 , several cost-equalised smoothers are near-ties and the oracle’s choice among them depends on the spectral content of the current error, which the router’s residual-norm features reflect only indirectly (agreement with the oracle 55–83% on its own rollouts against 88–99% for pairwise routers). Deciding *when* to call the corrector—the decision that dominates the wall-clock gains—is learned equally well in the ensembles.

Table 32: Ensembles of Section 7 at 128×128 and $\varepsilon = h^2$: median live time of the best member solver alone, of the best pairwise router among the members (paired speedup of the ensemble router over it in parentheses), and of the router and the cost-aware oracle over $\text{NO} \cup \mathcal{W}$ (64 instances; pairwise baselines from the pairwise runs on the same instances).

[results pending: confirmatory run in progress]

Table 33: Ensembles at 128×128 : mean (standard deviation over instances) fraction of the iterations up to the h^2 crossing that the ensemble router spends in each member and in the corrector.

[results pending: confirmatory run in progress]

Table 34: Ensembles at 128×128 : one-sided Wilcoxon p -values at $\varepsilon = h^2$ for the ensemble router against the fastest member solver alone (censored runs at their lower bound), against the best pairwise router in both directions, and against the cost-aware oracle over the same ensemble (alternative: the ensemble router is slower).

[results pending: confirmatory run in progress]

1153 E.8 Overheads and amortisation

1154 Table 35 lists the measured cost of every operation as a function of N , including one decision of the
 1155 LSTM router of Section 7 (full-state input of size $4N^2$, hidden size 256, 4 layers): at 128^2 one LSTM
 1156 decision costs **[pending]** ms, about **[pending]** Jacobi sweeps, which is why the wall-clock study uses
 1157 the scalar-feature router ($\approx 5 \mu\text{s}$, independent of N). The learned router’s decisions therefore account
 1158 for well under 1% of its solve time, and its corrector calls for 50–70% of the time to h^2 (one or two
 1159 calls). Table 36 accounts for training: the corrector (shared with HINTS) is fit in minutes, the router
 1160 in seconds to a few minutes per pairing, and the break-even number of solves after which the router’s
 1161 own training cost is recovered relative to HINTS ($\tau=25$) with the same corrector is in the thousands
 1162 at 128^2 and in the hundreds on the larger grids.

Table 35: Measured single-thread cost of one operation as a function of the grid size (Poisson; median of repeated calls). Multigrid uses a direct solve below 64^2 and is not listed there.

[results pending: confirmatory run in progress]

Table 36: Training-cost amortisation for the GS pairing: corrector training (data generation and least-squares fit, CPU), router training (oracle rollouts and DAgger rounds), median wall-clock saved per solve at $\varepsilon = h^2$ relative to HINTS ($\tau=25$) and to the solver alone, and the number of solves after which the router’s training cost is recovered relative to HINTS.

[results pending: confirmatory run in progress]

1163 E.9 Development versus confirmatory runs

1164 Several choices of this study were made after looking at test results: the corrector band and the
 1165 anisotropic sensor grid, the router of the convection–diffusion Jacobi pairing at 512^2 (retrained with
 1166 more rollouts after the first router missed h^2 on 4 of 16 development instances, Section E.1), and
 1167 the imitation budget of the ensemble routers (raised after the pairwise recipe imitated its oracle
 1168 imperfectly on one development cell, Section E.7); the optimistic best-schedule baseline is tuned on
 1169 the test set by design and labelled as such. All of these were made on the development set (seed 72),
 1170 with the earlier numpy/scipy sweeps. Every result in this appendix comes from a single evaluation

of the frozen configurations on the confirmatory set (seed 73) with the compiled kernels and three timed replays. Table 37 lists, for every cell present in both, the development and the confirmatory speedups: the two differ both through the instances and through the kernels—the compiled sweeps are 4–5× faster, so the corrector is worth 2–3× more sweeps than in the development runs and the margins over fixed schedules at h^2 shrink accordingly—and the table shows exactly how much of the development-phase margin survives.

Table 37: Development (seed 72, numpy/scipy sweeps, one timed replay) versus confirmatory (seed 73, compiled kernels, three timed replays) paired median speedups of the learned router, for every pairwise cell at $\varepsilon = h^2$ against HINTS ($\tau=25$) and against the best fixed schedule available in that run, and for every nested ensemble at 10^{-8} in work units against the best single-member router and against its oracle; one-sided Wilcoxon p -values in parentheses.

Equation	N	Pairing / ensemble	development (seed 72, numpy kernels)		confirmatory (seed 73, compiled kernels)	
			vs. HINTS-25 or best single	vs. best schedule or oracle	vs. HINTS-25 or best single	vs. best schedule or oracle
Poisson	128^2	{NO, Jacobi}	$2.68 \times (< 10^{-10})$	$1.21 \times (10^{-8})$	$1.69 \times (< 10^{-10})$	$0.96 \times (\text{slower}, 10^{-4})$
	128^2	{NO, Jacobi (0.67)}	$2.90 \times (< 10^{-10})$	$1.28 \times (10^{-9})$	$1.69 \times (< 10^{-10})$	$0.96 \times (\text{slower}, 10^{-6})$
	128^2	{NO, GS}	$6.75 \times (< 10^{-10})$	$1.94 \times (< 10^{-10})$	$2.96 \times (< 10^{-10})$	$0.95 \times (\text{slower}, 10^{-6})$
	128^2	{NO, SymGS}	$9.21 \times (< 10^{-10})$	$2.34 \times (< 10^{-10})$	$3.90 \times (< 10^{-10})$	$0.96 \times (\text{slower}, 10^{-6})$
	128^2	{NO, SOR (1.5)}	$10.3 \times (< 10^{-10})$	$3.22 \times (< 10^{-10})$	$4.70 \times (< 10^{-10})$	$0.96 \times (\text{slower}, 10^{-6})$
	128^2	{NO, Multigrid}	$1.96 \times (< 10^{-10})$	$1.88 \times (< 10^{-10})$	$1.00 \times (0.138)$	$0.97 \times (\text{slower}, < 10^{-10})$
Convection–diffusion	128^2	{NO, Jacobi}	$2.95 \times (< 10^{-10})$	$1.34 \times (< 10^{-10})$	$1.75 \times (< 10^{-10})$	$0.97 \times (\text{slower}, 0.187)$
	128^2	{NO, Jacobi (0.67)}	$3.14 \times (< 10^{-10})$	$1.32 \times (10^{-10})$	$1.69 \times (< 10^{-10})$	$0.96 \times (\text{slower}, 10^{-10})$
	128^2	{NO, GS}	$6.76 \times (< 10^{-10})$	$2.60 \times (< 10^{-10})$	$3.20 \times (< 10^{-10})$	$0.96 \times (\text{slower}, 10^{-5})$
	128^2	{NO, SymGS}	$8.70 \times (< 10^{-10})$	$2.40 \times (< 10^{-10})$	$4.09 \times (< 10^{-10})$	$0.95 \times (\text{slower}, < 10^{-10})$
	128^2	{NO, SOR (1.5)}	$8.89 \times (< 10^{-10})$	$2.40 \times (< 10^{-10})$	$5.94 \times (< 10^{-10})$	$0.97 \times (\text{slower}, 0.122)$
	128^2	{NO, Multigrid}	$2.07 \times (< 10^{-10})$	$2.01 \times (< 10^{-10})$	$0.99 \times (\text{slower}, 0.671)$	$0.97 \times (\text{slower}, 10^{-10})$
Anisotropic diffusion	128^2	{NO, Jacobi}	$3.16 \times (< 10^{-10})$	$1.25 \times (10^{-9})$	$1.74 \times (< 10^{-10})$	$0.96 \times (\text{slower}, 10^{-7})$
	128^2	{NO, Jacobi (0.67)}	$3.31 \times (< 10^{-10})$	$1.34 \times (< 10^{-10})$	$1.77 \times (< 10^{-10})$	$0.96 \times (\text{slower}, 10^{-9})$
	128^2	{NO, GS}	$7.61 \times (< 10^{-10})$	$1.99 \times (< 10^{-10})$	$2.95 \times (< 10^{-10})$	$0.96 \times (\text{slower}, 10^{-7})$
	128^2	{NO, SymGS}	$11.8 \times (< 10^{-10})$	$2.72 \times (< 10^{-10})$	$3.91 \times (< 10^{-10})$	$0.93 \times (\text{slower}, 10^{-8})$
	128^2	{NO, SOR (1.5)}	$7.36 \times (< 10^{-10})$	$2.04 \times (< 10^{-10})$	$3.08 \times (< 10^{-10})$	$0.95 \times (\text{slower}, 10^{-8})$

E.10 Verification of the theoretical assumptions

Theorem 4.1 and Propositions 4.2–4.3 assume that every error propagation map $G_j = I - C_j \circ \mathcal{L}_h^a$ is Lipschitz with constant $\rho_j < 1$ (respectively ≤ 1 with a shared eigenbasis), zero-preserving, and—for postfix monotonicity—invertible; Theorem 5.1 assumes that the per-step errors $\tilde{c}_j$ are bounded above and that at least one solver cannot annihilate the error in one step. Table 38 verifies these conditions numerically for every operation of the study on the mean-free subspace (the constant mode is the null space of the periodic operator and is excluded from the data and from every iteration): the Euclidean Lipschitz constant $\|G_j\|_2$ (power iteration on $G_j^\top G_j$; exact from the Fourier symbol for Jacobi), the Lipschitz constant $\|G_j\|_A = \|A_s^{1/2} G_j A_s^{-1/2}\|_2$ in the energy norm of the symmetric part A_s of the discrete operator, the same for the macro-action $G_j^{m_j}$, the spectral radius, the smallest observed contraction $\sigma_{\min} = \min \|G_j x\| / \|x\|$, $\|C_j(0)\|$, and the commutator $\|G_j G_{\text{NO}} - G_{\text{NO}} G_j\|$ on random vectors. Table 40 evaluates the quantities of Theorem 4.1 and Theorem 5.1 along the cost-aware oracle’s own path to $\varepsilon = 10^{-8}$ (a horizon of tens of macro-actions; to h^2 the path has one or two steps).

What holds, and in which norm. (i) Every classical solver is zero-preserving exactly and contracts in the sense that matters for convergence: the spectral radius of G_j is below 1 in every setting (at most 0.999997), and so is the energy-norm Lipschitz constant of the solver and of its macro-action (at most 0.999912). The *Euclidean* Lipschitz constant, however, exceeds 1 for Gauss–Seidel, SOR and symmetric GS (up to 1.89 for SOR), whose iteration matrices are non-normal; the proof of Proposition 4.2 uses only the triangle inequality and the composition rule for Lipschitz constants, so it holds verbatim with $\|\cdot\|_2$ replaced by the energy norm, in which the condition is satisfied by all classical solvers in all settings—but that statement concerns the greedy rule and objective defined with the energy norm, whereas the oracle and the router of this study minimise the Euclidean error. For the evaluated Euclidean rule, the strict-contraction premise of Proposition 4.2 therefore holds for the Jacobi-type solvers and fails for the non-normal sweeps, whose Euclidean expansion is confined to a single sweep (their spectral radii and macro-action contraction show the iteration is convergent). For (damped) Jacobi both constants agree (the maps are Fourier-diagonal); the values are close to 1 because the slowest modes (the smoothest ones, and the near-Nyquist mode for undamped Jacobi) are barely damped by a point smoother—which is exactly why the corrector is needed. (ii) The corrector’s map is zero-preserving, contracts every band mode to 1.7×10^{-3} or

1207 better (on average to 10^{-4} – 10^{-6}) and leaves the modes outside the band untouched, so its Lipschitz
 1208 constant is exactly 1 (not < 1): Theorem 4.2 in its strict form does not apply to the corrector, but
 1209 Theorem 4.3 does, and its second condition—a shared orthogonal eigenbasis—holds exactly for the
 1210 exact band-limited inverse together with (damped) Jacobi, all of which are diagonal in the Fourier
 1211 basis, and holds for the learned corrector up to its fit error (measured commutators $\approx 10^{-6}$, at the
 1212 corrector’s single-precision accuracy; replacing the learned corrector by the exact inverse does not
 1213 change the oracle’s decisions, Section E.1), so for the ensembles $\{\text{NO}, \text{Jacobi}\}$, $\{\text{NO}, \text{Jacobi (0.67)}\}$
 1214 and $\{\text{NO}, \text{Jacobi}, \text{Jacobi (0.67)}\}$ the objective is supermodular and postfix monotone up to that error
 1215 and Theorem 4.1 applies with $\alpha = 1$; for GS, SOR, symmetric GS and multigrid the commutators are
 1216 small but nonzero (10^{-3} – 10^{-1}) and no guarantee is claimed. (iii) Invertibility fails for most maps in
 1217 exact arithmetic: a Gauss–Seidel, SOR or symmetric-GS sweep annihilates an error concentrated
 1218 at the first grid point ($\sigma_{\min} = 0$), undamped Jacobi annihilates the mode $(N/4, N/4)$, and the
 1219 corrector annihilates the band; postfix monotonicity is therefore not guaranteed, which Section 4
 1220 anticipates (“our experiments show that greedy routers remain effective even when invertibility is not
 1221 met”). (iv) Table 39 verifies Theorem 4.1 *exactly* for short horizons on the evaluated ensembles, in
 1222 the Euclidean objective the oracle optimises: for every pairing, for $\{\text{Jacobi}, \text{Jacobi (0.67)}\}$, for the
 1223 two non-commuting sweeps $\{\text{GS}, \text{SOR}\}$, for $\{\text{GS}, \text{multigrid}\}$ and for all five stationary solvers, the
 1224 optimal sequence O of T macro-actions ($T = 4$, or 3 for $K = 6$) is found by exhaustive search on
 1225 8 test instances, and μ (over the greedy prefixes), $\alpha(O)$ (over the greedy prefixes, and over every
 1226 sequence of length below T) and the bound itself are computed. In every ensemble $\mu \leq 1.01$ and
 1227 $\alpha(O) = 1$ along the greedy prefixes, the bound holds on all 208 instance–ensemble pairs, and the
 1228 greedy sequence coincides with the exact optimum or is within a few percent of it on most instances
 1229 (the exceptions are myopic misses of up to an order of magnitude on single instances, most often for
 1230 $\{\text{GS}, \text{SOR}\}$ on convection–diffusion); over all short prefixes the ratio exceeds 1 only for the SOR
 1231 pairing, and only on prefixes the greedy rule never visits. The worst-case $\alpha(O)$ of Theorem 4.2, which
 1232 only uses the Lipschitz constants, is loose in comparison: along the oracle’s path to 10^{-8} (Table 40)
 1233 $\sum_i \rho_{O_i}^2$ is close to T because each ρ is dominated by the slowest mode, so the bound ranges from 1
 1234 (the multigrid pairing, whose cycle contracts everything) to 10^2 – 10^4 for the point smoothers, and
 1235 is infinite on the few instances where $\sum_i \rho_{O_i}^2 \geq T$. (The last two columns of Table 40 evaluate the
 1236 ratio of Equation (7) with the oracle’s own suffix in place of the unknown long-horizon O ; this is
 1237 only a diagnostic, and its maximum is attained trivially at the last prefix.) (v) For Theorem 5.1, the
 1238 per-step errors are bounded by the current error ($\max_j \tilde{c}_j / \|e^{(t)}\|^2 \leq 1$ at every visited state) and at
 1239 least one solver leaves a positive fraction of it ($\min_t \max_j \tilde{c}_j / \|e^{(t)}\|^2 > 0$), so the surrogate’s Bayes
 1240 consistency applies.

Table 38: Numerical verification of the assumptions of Propositions 4.2–4.3 for every operation of the study (mean-free subspace). **[Provisional: development run; the confirmatory run regenerates this table with the compiled kernels’ macro-action sizes.]** $\|\cdot\|_2$: Euclidean Lipschitz constant of the error propagation map; $\|\cdot\|_A$: Lipschitz constant in the energy norm of the symmetric part of $\mathcal{L}_h$, for one iteration and for the macro-action of m_j iterations; ρ : spectral radius; $\sigma_{\min}$: smallest observed contraction (a unit error at the first grid point for the triangular sweeps, the minimum over Fourier modes otherwise; for the corrector, the largest remaining fraction of a band mode); $\|C_j(0)\|$: zero preservation; last column: commutator with the corrector’s map on random unit vectors (Proposition 4.3). For (damped) Jacobi the constants are exact from the Fourier symbol (the value with the Nyquist modes included; without them undamped Jacobi has $\|G\|_2 = \rho = 0.9994$ at 128^2).

Equation	Operation	m_j	$\ I - C_j \mathcal{L}_h\ _2$	$\ I - C_j \mathcal{L}_h\ _A$	$\ (I - C_j \mathcal{L}_h)^{m_j}\ _A$	$\rho(I - C_j \mathcal{L}_h)$	$\sigma_{\min}$	$\ C_j(0)\ $	$\ [G_j, G_{\text{NO}}]\ $
Poisson, 128^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	4.4e-05 (band)	0	–
	Jacobi	12	0.999398	0.999398	0.992797	0.999398	0	0	5.7e-06
	Jacobi (0.67)	12	0.999596	0.999596	0.995168	0.999596	1.4e-04	0	4.1e-06
	GS	3	1.021515	0.998595	0.996368	0.998797	0	0	3.1e-02
	SymGS	2	1.006607	0.997396	0.995187	0.997623	0	0	2.2e-02
	SOR (1.5)	3	1.449417	0.996467	0.989336	0.996411	7.2e-01	0	8.5e-02
	Multigrid	1	0.098203	0.097860	0.097860	0.096877	1.9e-03	0	8.8e-03
Poisson, 256^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	4.4e-05 (band)	0	–
	Jacobi	6	0.999849	0.999849	0.999097	0.999849	0	0	1.3e-06
	Jacobi (0.67)	6	0.999899	0.999899	0.999395	0.999899	5.6e-05	0	1.2e-06
	GS	1	1.022053	0.998925	0.998925	0.999699	0	0	1.1e-02
	SymGS	1	1.005470	0.998839	0.998839	0.999401	0	0	9.4e-03
	SOR (1.5)	1	1.451367	0.998352	0.998352	0.999098	7.2e-01	0	3.8e-02
	Multigrid	1	0.102361	0.102263	0.102263	0.101344	1.4e-03	0	4.2e-03
Poisson, 512^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	2.1e-05 (band)	0	–
	Jacobi	3	0.999962	0.999962	0.999887	0.999962	0	0	6.1e-07
	GS	1	1.022202	0.998983	0.998983	0.999925	0	0	4.5e-03
	SOR (1.5)	1	1.451920	0.999043	0.999043	0.999774	7.2e-01	0	1.7e-02
	Multigrid	1	0.103470	0.103434	0.103434	0.102511	1.0e-03	0	1.3e-03
Convection–diffusion, 128^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	1.5e-05 (band)	0	–
	Jacobi	9	0.999398	0.999398	0.994593	0.999398	0	0	3.1e-06
	Jacobi (0.67)	9	0.999596	0.999596	0.996374	0.999596	1.4e-04	0	2.2e-06
	GS	3	1.025726	0.998460	0.995816	0.998589	0	0	3.1e-02
	SymGS	2	1.010112	0.997353	0.995168	0.997571	0	0	2.3e-02
	SOR (1.5)	3	1.554544	0.996981	0.983568	0.993973	7.5e-01	0	9.1e-02
	Multigrid	1	0.076967	0.074905	0.074905	0.074609	1.5e-03	0	6.9e-03
Convection–diffusion, 256^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	1.1e-05 (band)	0	–
	Jacobi	4	0.999849	0.999849	0.999398	0.999849	0	0	7.6e-07
	Jacobi (0.67)	4	0.999899	0.999899	0.999596	0.999899	5.6e-05	0	6.8e-07
	GS	1	1.024025	0.998934	0.998934	0.999674	0	0	1.2e-02
	SymGS	1	1.008004	0.998846	0.998846	0.999398	0	0	9.8e-03
	SOR (1.5)	1	1.497088	0.998192	0.998192	0.998846	7.3e-01	0	4.1e-02
	Multigrid	1	0.084356	0.083809	0.083809	0.083459	1.2e-03	0	3.7e-03
Convection–diffusion, 512^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	9.3e-06 (band)	0	–
	Jacobi	3	0.999962	0.999962	0.999887	0.999962	0	0	3.4e-07
	GS	1	1.023154	0.998983	0.998983	0.999922	0	0	4.6e-03
	SOR (1.5)	1	1.473282	0.999075	0.999075	0.999745	7.3e-01	0	1.8e-02
	Multigrid	1	0.091267	0.091201	0.091201	0.090840	9.4e-04	0	1.2e-03
Anisotropic diffusion, 128^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	4.4e-04 (band)	0	–
	Jacobi	11	0.999988	0.999988	0.999869	0.999988	1.1e-16	0	2.4e-05
	Jacobi (0.67)	11	0.999992	0.999992	0.999912	0.999992	4.4e-03	0	1.6e-05
	GS	3	1.131668	0.999528	0.999537	0.999976	0	0	5.7e-02
	SymGS	2	1.102789	0.999489	0.999255	0.999953	0	0	5.5e-02
	SOR (1.5)	3	1.889774	0.999393	0.999471	0.999928	8.3e-01	0	1.5e-01
Anisotropic diffusion, 256^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	1.7e-03 (band)	0	–
	Jacobi	38	0.999997	0.999997	0.999887	0.999997	1.1e-16	0	9.7e-05
	GS	11	1.131845	0.999245	0.999485	0.999994	0	0	4.0e-02
	SOR (1.5)	11	1.890225	0.999352	0.999473	0.999982	8.3e-01	0	1.1e-01

Table 39: Exact verification of Theorem 4.1 for short horizons (128×128 , 8 test instances, T macro-actions, K actions including the corrector; **[provisional: development-run macro-action sizes and instances]**): the optimal sequence O is found by exhaustive search; $\mu = \max_{t < T} g(S^t \oplus O)/g(O)$ over the greedy prefixes; $\alpha(O)$ over the greedy prefixes and over all sequences of length below T ; whether $g(S^T) \leq \mu(1 - \phi_T)g(O) + \phi_T g(\emptyset)$ holds; the ratio $g(S^T)/g(O)$ of the greedy to the optimal error (median and maximum over instances); and the number of instances on which the greedy sequence is the optimum. Errors below 10^{-11} relative are clipped to that floor.

Equation	Ensemble	K	T	μ	$\alpha(O)$ greedy prefixes	$\alpha(O)$ all $ S < T$	bound holds	$g(S^T)/g(O)$ med (max)	greedy = O
Poisson, 128^2	$\{Jacobi\}$	2	4	1.000	1.000	1.000	8/8	1.000 (1.06)	5/8
	$\{Jacobi(0.67)\}$	2	4	1.000	1.000	1.000	8/8	1.000 (1.00)	5/8
	$\{GS\}$	2	4	1.000	1.000	1.000	8/8	1.000 (1.00)	8/8
	$\{SymGS\}$	2	4	1.000	1.000	1.000	8/8	1.000 (1.02)	5/8
	$\{SOR(1.5)\}$	2	4	1.002	1.000	1.000	8/8	1.000 (1.00)	7/8
	$\{Multigrid\}$	2	4	1.001	1.000	1.000	8/8	1.000 (1.00)	8/8
	$\{Jacobi, Jacobi(0.67)\}$	3	4	1.000	1.000	1.000	8/8	1.000 (1.00)	0/8
	$\{GS, SOR(1.5)\}$	3	4	1.000	1.000	1.000	8/8	1.000 (7.24)	7/8
	$\{GS, Multigrid\}$	3	4	1.001	1.000	1.000	8/8	1.000 (1.00)	8/8
	$\{Jacobi, Jacobi(0.67), GS, SymGS, SOR(1.5)\}$	6	3	1.000	1.000	1.000	4/4	1.032 (7.16)	2/4
Convection-diffusion, 128^2	$\{Jacobi\}$	2	4	1.000	1.000	1.000	8/8	1.000 (1.00)	7/8
	$\{Jacobi(0.67)\}$	2	4	1.000	1.000	1.000	8/8	1.000 (1.05)	3/8
	$\{GS\}$	2	4	1.001	1.000	1.000	8/8	1.000 (1.00)	6/8
	$\{SymGS\}$	2	4	1.002	1.000	1.000	8/8	1.000 (1.01)	7/8
	$\{SOR(1.5)\}$	2	4	1.012	1.000	1.199	8/8	1.000 (1.00)	8/8
	$\{Multigrid\}$	2	4	1.001	1.000	1.000	8/8	1.000 (1.00)	8/8
	$\{Jacobi, Jacobi(0.67)\}$	3	4	1.000	1.000	1.000	8/8	1.000 (1.00)	2/8
	$\{GS, SOR(1.5)\}$	3	4	1.001	1.000	1.000	8/8	6.703 (8.12)	2/8
	$\{GS, Multigrid\}$	3	4	1.001	1.000	1.000	8/8	1.000 (1.00)	8/8
	$\{Jacobi, Jacobi(0.67), GS, SymGS, SOR(1.5)\}$	6	3	1.000	1.000	1.000	4/4	1.000 (1.07)	2/4
Anisotropic diffusion, 128^2	$\{Jacobi\}$	2	4	1.000	1.000	1.000	8/8	1.000 (2.90)	1/8
	$\{Jacobi(0.67)\}$	2	4	1.000	1.000	1.000	8/8	1.000 (11.85)	2/8
	$\{GS\}$	2	4	1.000	1.000	1.000	8/8	1.000 (1.09)	6/8
	$\{SymGS\}$	2	4	1.000	1.000	1.000	8/8	1.000 (1.33)	5/8
	$\{SOR(1.5)\}$	2	4	1.000	1.000	135.837	8/8	1.000 (1.05)	6/8
	$\{Jacobi, Jacobi(0.67)\}$	3	4	1.000	1.000	1.000	8/8	1.000 (2.90)	2/8
	$\{GS, SOR(1.5)\}$	3	4	1.000	1.000	1.000	8/8	1.000 (2.13)	5/8
	$\{GS, Multigrid\}$	3	4	1.001	1.000	1.000	4/4	1.000 (6.02)	3/4
	$\{Jacobi, Jacobi(0.67), GS, SymGS, SOR(1.5)\}$	6	3	1.000	1.000	1.000	4/4	1.000 (6.02)	3/4

Table 40: Quantities of Theorem 4.1, Proposition 4.2 and Theorem 5.1 along the cost-aware oracle’s path to $\varepsilon = 10^{-8}$ (8 test instances; medians unless stated): number of macro-actions T , $\sum_i \rho_{O_i}^2$ with the energy-norm constants of the chosen macro-actions, the resulting $\alpha(O)$ of Proposition 4.2 (median over the instances on which $T - \sum_i \rho_{O_i}^2 > 0$; ∞ otherwise), the empirical supermodularity ratio $\hat{\alpha}$ of Equation (7) evaluated with S the oracle’s prefix and S' its suffix (maximum and median over prefixes and instances), and the normalised bounds of Theorem 5.1: the largest ratio $\max_j \tilde{c}_j / \|e^{(t)}\|^2$ over visited states ($\bar{E}$) and the smallest value of $\max_j \tilde{c}_j / \|e^{(t)}\|^2$ ($E_{\min}$).

Equation	Ensemble	T	$\sum_i \rho_{O_i}^2$	$\alpha(O)$ (Prop. 4.2)	$\hat{\alpha}$ max	$\hat{\alpha}$ median	$\max_i \max_j \tilde{c}_j / \ e^{(t)}\ ^2$	$\min_i \max_j \tilde{c}_j / \ e^{(t)}\ ^2$
Poisson, 128 ²	{Jacobi}	52	51.775	1	1.000	0.420	1.000	6.3e-02
	{Jacobi(0.67)}	4	3.981	52	1.000	0.598	1.000	9.0e-02
	{GS}	6	5.971	34	1.000	0.498	1.000	1.4e-01
	{SymGS}	5	4.971	35	1.000	0.501	1.000	1.1e-01
	{SOR(1.5)}	9	8.852	7	1.000	0.376	1.000	1.2e-01
	{Multigrid}	4	1.029	1	1.000	0.417	1.000	9.6e-06
	{Jacobi, Jacobi(0.67)}	4	3.976	42	1.000	0.612	1.000	9.0e-02
	{Jacobi, Jacobi(0.67), GS, SymGS, SOR(1.5)}	4	3.976	42	1.000	0.612	1.000	1.4e-01
Poisson, 256 ²	{Jacobi}	10	9.486	74	1.000	0.459	1.000	3.6e-01
	{Jacobi(0.67)}	12	11.988	83	1.000	0.302	1.000	4.7e-01
	{GS}	26	26.447	19	1.000	0.201	1.000	6.3e-01
	{SymGS}	14	14.471	35	1.000	0.255	1.000	5.1e-01
	{SOR(1.5)}	24	23.928	14	1.000	0.207	1.000	5.4e-01
	{Multigrid}	4	1.037	1	1.000	0.418	1.000	1.1e-05
	{Jacobi, Jacobi(0.67)}	9	8.988	80	1.000	0.389	1.000	4.3e-01
	{Jacobi, Jacobi(0.67), GS, SymGS, SOR(1.5)}	9	8.988	80	1.000	0.389	1.000	4.3e-01
Poisson, 512 ²	{Jacobi}	55	54.988	80 (∞ on 1/8)	1.000	0.236	1.000	7.8e-01
	{GS}	86	86.328	6 (∞ on 1/8)	1.000	0.154	1.000	8.4e-01
	{SOR(1.5)}	41	40.925	13 (∞ on 1/8)	1.000	0.166	1.000	7.4e-01
	{Multigrid}	4	1.043	1 (∞ on 1/8)	1.000	0.419	1.000	1.0e-05
Convection-diffusion, 128 ²	{Jacobi}	156	154.834	1	1.000	0.391	1.000	7.9e-02
	{Jacobi(0.67)}	5	4.978	46	1.000	0.503	1.000	1.2e-01
	{GS}	6	5.967	30	1.000	0.494	1.000	1.2e-01
	{SymGS}	5	4.971	35	1.000	0.501	1.000	1.0e-01
	{SOR(1.5)}	9	8.772	4	1.000	0.359	1.000	7.5e-02
	{Multigrid}	4	1.017	1	1.000	0.417	1.000	8.2e-06
	{Jacobi, Jacobi(0.67)}	4	3.982	55	1.000	0.659	1.000	1.2e-01
	{Jacobi, Jacobi(0.67), GS, SymGS, SOR(1.5)}	4	3.982	55	1.000	0.615	1.000	1.2e-01
Convection-diffusion, 256 ²	{Jacobi}	26	25.971	46	1.000	0.464	1.000	5.0e-01
	{Jacobi(0.67)}	20	19.486	71	1.000	0.226	1.000	5.2e-01
	{GS}	28	28.444	18	1.000	0.186	1.000	5.9e-01
	{SymGS}	16	16.467	30	1.000	0.221	1.000	5.1e-01
	{SOR(1.5)}	24	23.921	13	1.000	0.207	1.000	5.2e-01
	{Multigrid}	4	1.021	1	1.000	0.418	1.000	2.8e-06
	{Jacobi, Jacobi(0.67)}	14	13.986	71	1.000	0.276	1.000	5.2e-01
	{Jacobi, Jacobi(0.67), GS, SymGS, SOR(1.5)}	14	13.986	71	1.000	0.276	1.000	5.2e-01
Convection-diffusion, 512 ²	{Jacobi}	62	62.486	73	1.000	0.196	1.000	7.6e-01
	{GS}	94	93.813	5	1.000	0.138	1.000	8.4e-01
	{SOR(1.5)}	42	41.926	14	1.000	0.158	1.000	6.8e-01
	{Multigrid}	4	1.033	1	1.000	0.377	1.000	2.0e-06
Anisotropic diffusion, 128 ²	{Jacobi}	10	9.998	347 (∞ on 1/8)	1.000	0.481	1.000	3.5e-01
	{Jacobi(0.67)}	5	4.999	1896 (∞ on 1/8)	1.000	0.504	1.000	3.5e-01
	{GS}	7	6.995	216 (∞ on 1/8)	1.000	0.500	1.000	3.5e-01
	{SymGS}	6	6.494	168 (∞ on 1/8)	1.000	0.435	1.000	3.5e-01
	{SOR(1.5)}	9	8.993	135 (∞ on 1/8)	1.000	0.397	1.000	3.5e-01
	{Jacobi, Jacobi(0.67)}	4	4.000	2282 (∞ on 1/8)	1.000	0.652	1.000	3.5e-01
	{Jacobi, Jacobi(0.67), GS, SymGS, SOR(1.5)}	4	4.000	2282 (∞ on 1/8)	1.000	0.639	1.000	3.5e-01
	{Jacobi, Jacobi(0.67), GS, SymGS, SOR(1.5), Multigrid}	4	4.000	2282 (∞ on 1/8)	1.000	0.639	1.000	3.5e-01
Anisotropic diffusion, 256 ²	{Jacobi}	3	3.000	4413 (∞ on 2/8)	1.000	0.608	1.000	2.9e-01
	{GS}	3	2.999	972 (∞ on 2/8)	1.000	0.658	1.000	3.9e-01
	{SOR(1.5)}	4	3.498	474 (∞ on 2/8)	1.000	0.578	1.000	2.2e-01

F Limitations

A primary limitation of our approach is the higher on-time computational cost requires to learn the greedy routing strategy. In contrast, the simpler baselines such as HINTS incur no training overhead. While this cost is amortized during inference time and leads to improved solution quality, it may be prohibitive in settings with limited computational resources or when rapid deployment is required. On the deployment side, Section E shows that the gains survive cost accounting: with a corrector that covers the smooth half of the spectrum and a classical smoother that covers the rest, one or two corrector calls suffice and the router’s advantage over fixed schedules is uniform across solvers, equations and tolerances. The corrector we use is specific to the constant-coefficient periodic setting (a fixed Fourier trunk basis on a coarse sensor grid), its cost grows with the fourth power of the sensor-grid size, and undamped Jacobi remains limited far below truncation level by the near-Nyquist modes that neither component damps. Our wall-clock study is restricted to single-thread CPU execution; with a corrector band that does not grow with the grid, the gains over fixed schedules shrink at 512² for the point smoothers, whereas pairing the corrector with geometric multigrid restores them (Section E). Ensembles of classical solvers pay off only where the error passes through regimes that favour different members; at truncation level a single well-chosen smoother suffices. For the constant-coefficient periodic test problems the discrete operator is diagonalised by the FFT, and the resulting direct solve, reported in every table, is faster than every iterative method we measured, including ours ($2.97 \times - 3.12 \times$ against the best router); the hybrid is meant for problems on which no

1260 such direct diagonalisation exists and the corrector must be learned rather than computed, which is
1261 why Section E also reports a variable-coefficient diffusion problem.

1262 Additionally, the effectiveness of the learned router depends on the neural operator exhibiting
1263 heterogeneous performance across samples. When the neural operators produces predictions of
1264 relatively uniform quality, the benefit of adaptive routing diminishes and it is more practical to set a
1265 fixed schedule based on the quality of the neural operator. In practice, achieving such heterogeneity
1266 can be sensitive to training choices, and neural operators themselves can be finicky to train.

1267 **G LLM Usage**

1268 LLMs, specifically ChatGPT and Gemini, supported the writing process in an iterative manner. We
1269 drafted paragraphs and asked the models for feedback on grammar and clarity. We then incorporated
1270 selected suggestions into the writing and repeated this process until we were satisfied with the writing.

1271 The code developed for the experiments was written by the authors with the help of occasional code
1272 completions. The central components (e.g., the hybrid solver implementation and the greedy-router
1273 training pipelines) were implemented exclusively by the authors.

1274 All substantive intellectual contributions, which include ideas, theorems, and analyses, are our own.
1275 LLMs were occasionally used to verify the correctness of proofs, but all proof strategies originated
1276 from the authors and relevant literature.

1277 NeurIPS Paper Checklist

1278 The checklist is designed to encourage best practices for responsible machine learning research,
1279 addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove
1280 the checklist: **The papers not including the checklist will be desk rejected.** The checklist should
1281 follow the references and follow the (optional) supplemental material. The checklist does NOT count
1282 towards the page limit.

1283 Please read the checklist guidelines carefully for information on how to answer these questions. For
1284 each question in the checklist:

- 1285 • You should answer [Yes], [No], or [N/A].
- 1286 • [N/A] means either that the question is Not Applicable for that particular paper or the
1287 relevant information is Not Available.
- 1288 • Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

1289 **The checklist answers are an integral part of your paper submission.** They are visible to the
1290 reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it
1291 (after eventual revisions) with the final version of your paper, and its final version will be published
1292 with the paper.

1293 The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation.
1294 While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a
1295 proper justification is given (e.g., error bars are not reported because it would be too computationally
1296 expensive” or “we were unable to find the license for the dataset we used”). In general, answering
1297 [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we
1298 acknowledge that the true answer is often more nuanced, so please just use your best judgment and
1299 write a justification to elaborate. All supporting evidence can appear either in the main paper or the
1300 supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification
1301 please point to the section(s) where related material for the question can be found.

1302 IMPORTANT, please:

- 1303 • **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”,**
- 1304 • **Keep the checklist subsection headings, questions/answers and guidelines below.**
- 1305 • **Do not modify the questions and only use the provided macros for your answers.**

1306 1. Claims

1307 Question: Do the main claims made in the abstract and introduction accurately reflect the
1308 paper’s contributions and scope?

1309 Answer: [Yes]

1310 Justification: The abstract clearly delineates the paper’s main contributions and scope, and
1311 the introduction reiterates these claims in a structured, bullet-point form. All stated claims
1312 are directly supported by the theoretical and experimental results presented in the paper.

1313 Guidelines:

- 1314 • The answer [N/A] means that the abstract and introduction do not include the claims
1315 made in the paper.
- 1316 • The abstract and/or introduction should clearly state the claims made, including the
1317 contributions made in the paper and important assumptions and limitations. A [No] or
1318 [N/A] answer to this question will not be perceived well by the reviewers.
- 1319 • The claims made should match theoretical and experimental results, and reflect how
1320 much the results can be expected to generalize to other settings.
- 1321 • It is fine to include aspirational goals as motivation as long as it is clear that these goals
1322 are not attained by the paper.

1323 2. Limitations

1324 Question: Does the paper discuss the limitations of the work performed by the authors?

1325 Answer: [Yes]

Justification: The paper discusses key limitations in Appendix E.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: All theoretical results include clearly stated assumptions in the statement of theorems. Complete proofs are provided in Appendices A and B with appropriate cross-referencing from the main text.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper provides sufficient detail to reproduce all experiments, including training procedures and implementation details described in the main text and Appendix C. Additionally, we release code with a comprehensive README to support the reproducibility of our results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We provide open access to code with detailed instructions in a comprehensive README, including environment setup and commands to train routers and reproduce the main experimental results. These files will automatically generate the data needed for training. Data generation has also been documented in Appendix C.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.

- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [\[Yes\]](#)

Justification: The paper specifies all key training and evaluation details in the Section 6 and Appendix C.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [\[Yes\]](#)

Justification: The paper reports standard deviations in the main results tables, and further provides statistical significance via t-test p values in Appendix D.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not include experiments.
- The authors should answer [\[Yes\]](#) if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

1485 Answer: [Yes]

1486 Justification: The paper provides details on compute resources in Appendix C.

1487 Guidelines:

- 1488 • The answer [N/A] means that the paper does not include experiments.
- 1489 • The paper should indicate the type of compute workers CPU or GPU, internal cluster,
- 1490 or cloud provider, including relevant memory and storage.
- 1491 • The paper should provide the amount of compute required for each of the individual
- 1492 experimental runs as well as estimate the total compute.
- 1493 • The paper should disclose whether the full research project required more compute
- 1494 than the experiments reported in the paper (e.g., preliminary or failed experiments that
- 1495 didn't make it into the paper).

1496 **9. Code of ethics**

1497 Question: Does the research conducted in the paper conform, in every respect, with the

1498 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

1499 Answer: [Yes]

1500 Justification: This paper does not involve any human subjects and all data has been simu-

1501 lated. This research focuses on numerical methods for physical systems and does not raise

1502 concerns related to safety, security, discrimination, surveillance, deception & harrassment,

1503 environment, human rights, and bias & fairness.

1504 Guidelines:

- 1505 • The answer [N/A] means that the authors have not reviewed the NeurIPS Code of
- 1506 Ethics.
- 1507 • If the authors answer [No], they should explain the special circumstances that require a
- 1508 deviation from the Code of Ethics.
- 1509 • The authors should make sure to preserve anonymity (e.g., if there is a special consid-
- 1510 eration due to laws or regulations in their jurisdiction).

1511 **10. Broader impacts**

1512 Question: Does the paper discuss both potential positive societal impacts and negative

1513 societal impacts of the work performed?

1514 Answer: [Yes]

1515 Justification: The paper discusses broader impacts in the introduction section. No direct

1516 negative societal impacts are anticipated.

1517 Guidelines:

- 1518 • The answer [N/A] means that there is no societal impact of the work performed.
- 1519 • If the authors answer [N/A] or [No], they should explain why their work has no societal
- 1520 impact or why the paper does not address societal impact.
- 1521 • Examples of negative societal impacts include potential malicious or unintended uses
- 1522 (e.g., disinformation, generating fake profiles, surveillance), fairness considerations
- 1523 (e.g., deployment of technologies that could make decisions that unfairly impact specific
- 1524 groups), privacy considerations, and security considerations.
- 1525 • The conference expects that many papers will be foundational research and not tied
- 1526 to particular applications, let alone deployments. However, if there is a direct path to
- 1527 any negative applications, the authors should point it out. For example, it is legitimate
- 1528 to point out that an improvement in the quality of generative models could be used to
- 1529 generate Deepfakes for disinformation. On the other hand, it is not needed to point out
- 1530 that a generic algorithm for optimizing neural networks could enable people to train
- 1531 models that generate Deepfakes faster.
- 1532 • The authors should consider possible harms that could arise when the technology is
- 1533 being used as intended and functioning correctly, harms that could arise when the
- 1534 technology is being used as intended but gives incorrect results, and harms following
- 1535 from (intentional or unintentional) misuse of the technology.

- 1536 • If there are negative societal impacts, the authors could also discuss possible mitigation
1537 strategies (e.g., gated release of models, providing defenses in addition to attacks,
1538 mechanisms for monitoring misuse, mechanisms to monitor how a system learns from
1539 feedback over time, improving the efficiency and accessibility of ML).

1540 11. Safeguards

1541 Question: Does the paper describe safeguards that have been put in place for responsible
1542 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
1543 image generators, or scraped datasets)?

1544 Answer: [N/A]

1545 Justification: The paper does not release any models or datasets at all. It focuses on method-
1546 ological contributions for numerical solvers using simulated data. Therefore, safeguards are
1547 not applicable.

1548 Guidelines:

- 1549 • The answer [N/A] means that the paper poses no such risks.
1550 • Released models that have a high risk for misuse or dual-use should be released with
1551 necessary safeguards to allow for controlled use of the model, for example by requiring
1552 that users adhere to usage guidelines or restrictions to access the model or implementing
1553 safety filters.
1554 • Datasets that have been scraped from the Internet could pose safety risks. The authors
1555 should describe how they avoided releasing unsafe images.
1556 • We recognize that providing effective safeguards is challenging, and many papers do
1557 not require this, but we encourage authors to take this into account and make a best
1558 faith effort.

1559 12. Licenses for existing assets

1560 Question: Are the creators or original owners of assets (e.g., code, data, models), used in
1561 the paper, properly credited and are the license and terms of use explicitly mentioned and
1562 properly respected?

1563 Answer: [N/A]

1564 Justification: The paper does not use any external datasets or pretrained models. All
1565 experiments are conducted using simulated data and models trained in-house.

1566 Guidelines:

- 1567 • The answer [N/A] means that the paper does not use existing assets.
1568 • The authors should cite the original paper that produced the code package or dataset.
1569 • The authors should state which version of the asset is used and, if possible, include a
1570 URL.
1571 • The name of the license (e.g., CC-BY 4.0) should be included for each asset.
1572 • For scraped data from a particular source (e.g., website), the copyright and terms of
1573 service of that source should be provided.
1574 • If assets are released, the license, copyright information, and terms of use in the
1575 package should be provided. For popular datasets, `paperswithcode.com/datasets`
1576 has curated licenses for some datasets. Their licensing guide can help determine the
1577 license of a dataset.
1578 • For existing datasets that are re-packaged, both the original license and the license of
1579 the derived asset (if it has changed) should be provided.
1580 • If this information is not available online, the authors are encouraged to reach out to
1581 the asset's creators.

1582 13. New assets

1583 Question: Are new assets introduced in the paper well documented and is the documentation
1584 provided alongside the assets?

1585 Answer: [Yes]

1586 Justification: We release code for our method which is documented with a detailed
1587 README.

1588 Guidelines:

1589 • The answer [N/A] means that the paper does not release new assets.

1590 • Researchers should communicate the details of the dataset/code/model as part of their

1591 submissions via structured templates. This includes details about training, license,

1592 limitations, etc.

1593 • The paper should discuss whether and how consent was obtained from people whose

1594 asset is used.

1595 • At submission time, remember to anonymize your assets (if applicable). You can either

1596 create an anonymized URL or include an anonymized zip file.

1597 **14. Crowdsourcing and research with human subjects**

1598 Question: For crowdsourcing experiments and research with human subjects, does the paper

1599 include the full text of instructions given to participants and screenshots, if applicable, as

1600 well as details about compensation (if any)?

1601 Answer: [N/A]

1602 Justification: The paper does not involve crowdsourcing or any research with human subjects.

1603 All experiments are conducted using simulated data.

1604 Guidelines:

1605 • The answer [N/A] means that the paper does not involve crowdsourcing nor research

1606 with human subjects.

1607 • Including this information in the supplemental material is fine, but if the main contribu-

1608 tion of the paper involves human subjects, then as much detail as possible should be

1609 included in the main paper.

1610 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,

1611 or other labor should be paid at least the minimum wage in the country of the data

1612 collector.

1613 **15. Institutional review board (IRB) approvals or equivalent for research with human**

1614 **subjects**

1615 Question: Does the paper describe potential risks incurred by study participants, whether

1616 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)

1617 approvals (or an equivalent approval/review based on the requirements of your country or

1618 institution) were obtained?

1619 Answer: [N/A]

1620 Justification: This paper does not involve human subjects. All experiments are conducted

1621 using simulated data. Therefore, IRB approval is not applicable.

1622 Guidelines:

1623 • The answer [N/A] means that the paper does not involve crowdsourcing nor research

1624 with human subjects.

1625 • Depending on the country in which research is conducted, IRB approval (or equivalent)

1626 may be required for any human subjects research. If you obtained IRB approval, you

1627 should clearly state this in the paper.

1628 • We recognize that the procedures for this may vary significantly between institutions

1629 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the

1630 guidelines for their institution.

1631 • For initial submissions, do not include any information that would break anonymity (if

1632 applicable), such as the institution conducting the review.

1633 **16. Declaration of LLM usage**

1634 Question: Does the paper describe the usage of LLMs if it is an important, original, or

1635 non-standard component of the core methods in this research? Note that if the LLM is used

1636 only for writing, editing, or formatting purposes and does *not* impact the core methodology,

1637 scientific rigor, or originality of the research, declaration is not required.

1638 Answer: [N/A]

1639 Justification: LLMs are primarily used for writing and editing. Further details are provided
1640 in Appendix F.
1641 Guidelines:
1642 • The answer [N/A] means that the core method development in this research does not
1643 involve LLMs as any important, original, or non-standard components.
1644 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
1645 be described.